TITLE :

Systems and Methods for Preventing Artificial-Intelligence-Generated Hallucinations, Unsupported Outputs, Stale Outputs, and Unsafe Agentic Acts from Becoming External Consequences Using Candidate-Act Finality, Consequence Simulation, Escalated Conditional Finality, and Cryptographic Execution-Dependency Non-Completability

The Missing Protocol Layer of the Internet

The internet was built with protocols governing how data moves — TCP/IP for transmission, TLS for confidentiality, DNS for naming, OAuth for delegated identity. Each protocol solved a specific boundary problem: how packets route, how channels are secured, how identities are asserted. What no existing protocol addresses is the boundary at which a computational output becomes an externally effective act. As artificial-intelligence systems assume increasing operational authority — executing payments, mutating databases, controlling infrastructure, issuing communications, managing supply chains, and directing physical systems — the absence of a finality protocol at the computation-to-consequence boundary becomes a structural gap in internet architecture. Existing protocols govern the transmission of instructions; none governs whether a computationally generated instruction has satisfied the machine-verifiable predicates required to become a consequence.

The disclosed architecture addresses this gap by introducing a protected finality layer positioned at the output-to-consequence boundary — a protocol-level enforcement mechanism that does for artificial-intelligence-generated acts what TLS did for data in transit and what OAuth did for delegated access: it converts an uncontrolled technical boundary into a machine-verifiable, cryptographically enforced, and sink-verified governance checkpoint through which no artificial-intelligence-generated output may pass into external consequence without satisfying the required finality predicates.

TECHNICAL FIELD

This disclosure relates to artificial intelligence governance, hallucination-resistant artificial-intelligence control, autonomous-agent execution control, protected execution finality, machine-verifiable compliance, cryptographic capability release, hardware-rooted execution control, secure distributed computing, trusted enforcement domains, and technical systems for controlling the boundary at which computational outputs become externally effective acts.

More particularly, this disclosure relates to systems and methods in which an artificial-intelligence-generated output is treated as a non-effective Candidate Act and is prevented from becoming an external consequence unless a protected finality pipeline validates output-level, provenance-level, factual-support-level, consequence-level, jurisdiction-level, epoch-level, and sink-level predicates.

The disclosure further relates to advanced non-completability embodiments in which a Finality Sink is technically unable to complete a Candidate Act unless a protected Execution Handle or other sink-bound capability enables completion.

BACKGROUND OF THE INVENTION

Artificial intelligence systems increasingly generate outputs that are not merely informational. Modern artificial intelligence systems, autonomous agents, enterprise copilots, orchestration systems, cloud-management systems, software-development agents, database agents, financial agents, telecom controllers, robotic systems, model-management agents, and decision-support systems may produce outputs that trigger tool calls, payments, data exports, database commits, memory writes, model updates, communications, software deployments, network-configuration changes, settlement events, legal commitments, physical commands, or other externally effective acts.

Existing artificial-intelligence governance approaches commonly focus on model training, alignment, prompt filtering, output moderation, identity checks, access control, policy review, ordinary human approval, logging, monitoring, or post-hoc audit. These approaches may reduce risk, but they do not reliably control the precise technical boundary at which an artificial-intelligence-generated output becomes an external consequence.

A specific technical problem arises because an artificial-intelligence model may be approved, a workflow may be approved, a prompt policy may be approved, a tool policy may be approved, and observed runtime behavior may remain within an expected envelope, yet the specific generated output may still be incorrect, unsupported, stale, unsafe, confidential, Technically undesired, jurisdictionally improper, or otherwise unsuitable for effectuation.

Thus, approval of the model is not approval of the output. Approval of the workflow is not approval of the consequence. Approval of runtime behavior is not approval of the specific act becoming externally effective.

Another technical problem arises because many artificial-intelligence-agent systems operate through chains of tools, APIs, queues, plugins, workflow engines, databases, storage layers, payment modules, communication systems, network controllers, and downstream agents. A moderation layer or application-layer policy decision may be bypassed, separated from the actual effectuation interface, or applied before the final state of the act is known.

A further technical problem arises from state drift and time-of-check-to-time-of-use risk. A Candidate Act may appear permissible at a first time but become impermissible before effectuation due to a change in policy state, revocation state, source validity, recipient status, jurisdictional condition, Finality Sink trust state, model approval, data classification, human approval, operational mode, or risk classification. A system that checks an artificial-intelligence-generated act at one time and executes it later may therefore effectuate a consequence that is no longer valid under current protected state conditions.

A further Technical deployment problem arises when systems provide only binary allow-or-deny outcomes. In practical enterprise, financial, operational, infrastructure, communications, and regulated environments, many acts are neither prohibited nor safe for ordinary release. Such acts may be Technically necessary but should be allowed only under stricter safeguards. Therefore, practical artificial-intelligence finality governance should support graduated outcomes, including escalated but still allowable effectuation.

The technical problem is therefore not merely whether an artificial-intelligence system was allowed to generate an output. The technical problem is whether the generated output should be permitted to

cross the machine boundary from computation into consequence, under current protected state conditions, and under what scope, safeguards, and sink-verifiable authority.

DIFFERENCE FROM EXISTING AND PRIOR TECHNICAL APPROACHES

1. Difference From Artificial-Intelligence Moderation and Output Filtering

Existing artificial-intelligence safety systems often focus on prompt filtering, response moderation, toxicity classification, refusal policies, output scoring, content filtering, or guardrail models. Such systems may classify or suppress certain generated content, but they do not necessarily control the final machine boundary where an artificial-intelligence-generated output becomes an external consequence.

The disclosed architecture is different because the artificial-intelligence-generated output is converted into a Candidate Act and held in a non-effective state before consequence. The Candidate Act cannot become externally effective merely because a moderation layer permitted the text, a model generated the output, or an application accepted the response.

Instead, the Candidate Act must satisfy protected output-level, provenance-level, factual-support-level, consequence-level, jurisdiction-level, epoch-level, and sink-level finality predicates. Only then may a scoped non-bearer capability or Execution Handle be released and verified by the Finality Sink.

Accordingly, the invention does not merely moderate artificial-intelligence output. It governs whether the artificial-intelligence output may become consequence.

2. Difference From Identity, Access-Control, and API Authorization Systems

Identity systems, access-control systems, role-based access control, attribute-based access control, API gateways, and ordinary authorization layers may determine whether a user, service, application, or artificial-intelligence agent is permitted to access a resource or call an interface.

Such systems generally answer the question:

Who or what is allowed to request an operation?

The disclosed architecture answers a different technical question:

Whether this specific artificial-intelligence-generated Candidate Act may become this specific external consequence at this specific Finality Sink under current protected state conditions.

A user, model, service, or agent may be authenticated and authorized, yet the specific Candidate Act may still be blocked if it lacks factual support, exceeds the Result-Consequence Acceptance Envelope, relies on stale provenance, conflicts with jurisdiction, fails consequence simulation, uses a stale policy epoch, uses a stale revocation epoch, or is not accepted by the Finality Sink.

Thus, identity or access authority is not treated as execution authority.

3. Difference From Ordinary Bearer Tokens and Permission Tokens

Ordinary permission tokens or bearer tokens may permit access when presented by a holder. Such tokens may be misused if copied, stolen, forwarded, replayed, or presented outside the intended context.

The disclosed scoped non-bearer finality capability is different. Possession of the capability data alone is insufficient to cause effectuation.

The capability may be bound to the Candidate Act hash, Hash-Linked Candidate Act Descriptor hash, Finality Sink identity, hardware-bound sink identity, permitted recipient, permitted purpose, permitted jurisdiction, permitted data class, permitted consequence type, nonce, expiration, policy epoch, revocation epoch, Result-Consequence Acceptance Envelope, validation receipt, and sink verification context.

In advanced non-completability embodiments, the Execution Handle may also supply, reconstruct, unseal, combine, or activate missing execution material required by the Finality Sink. Therefore, a copied or stolen artifact cannot operate as generic permission to act.

The invention therefore provides sink-bound consequence finality, not ordinary access-token authorization.

4. Difference From Policy Engines and Gatekeeper Systems

Policy engines and gatekeeper systems may evaluate rules and return allow-or-deny decisions. However, an allow-or-deny response may still be separated from the actual effectuation interface. If downstream software, a tool executor, workflow engine, API endpoint, database commit layer, actuator, payment module, communication system, or network controller can still execute the act, the policy decision may be bypassed, ignored, misapplied, stale, or incomplete.

The disclosed architecture is different because the finality requirement follows the Candidate Act to the Finality Sink. The Finality Sink must verify the scoped non-bearer capability or Execution Handle before effectuation.

In advanced non-completability embodiments, the sink is not merely told to refuse unauthorized acts. The sink is technically unable to complete the Candidate Act unless protected finality validation releases, reconstructs, unseals, combines, or activates the missing execution material.

Thus, the invention is not merely a policy decision system. It is a sink-bound finality-control system.

5. Difference From Confidential Computing or Trusted Execution Alone

Trusted execution environments, secure enclaves, hardware security modules, secure elements, or other protected environments may protect computation, secrets, or data in use.

The disclosed architecture may use such components, but the invention is not merely the use of a trusted environment.

The protected domain is used to enforce a specific output-to-consequence finality sequence. The Protected Enforcement Domain validates the Candidate Act, binds evidence, evaluates consequence, records validation evidence, releases a scoped capability or Execution Handle, and prevents effectuation unless the Finality Sink verifies the required authority.

Therefore, protected computation is used not merely for confidentiality or integrity, but for controlled transition from artificial-intelligence computation into external consequence.

6. Difference From Logging, Monitoring, and Post-Hoc Audit

Logging, monitoring, audit trails, explainability records, and compliance reports may record what happened after an artificial-intelligence output was generated or after an action occurred. Such systems may support accountability, but they do not necessarily prevent an improper act from becoming externally effective.

The disclosed architecture is different because the Candidate Act remains non-effective before effectuation. Validation occurs before or atomically with capability release. The validation receipt is not merely a later audit record. In some embodiments, it is part of the protected finality transaction that binds validation evidence to release authority.

Thus, the invention does not merely record artificial-intelligence consequences after they occur. It prevents unvalidated consequences from occurring.

7. Difference From Ordinary Human Approval

Ordinary human approval may include a click, email confirmation, chat instruction, workflow approval, screen prompt, or manual review. Such approval may be stale, replayed, spoofed, clickjacked, unbound to the actual consequence, or separated from the Finality Sink.

The disclosed architecture may require a Protected Human Approval Finality Token where human approval is needed. The protected approval may be bound to the Candidate Act, output content, predicted consequence, approving role, authenticated user presence, approval time, policy epoch, revocation epoch, and Finality Sink identity.

Accordingly, the invention does not rely on generic human approval. It converts human approval into a protected, act-specific finality predicate.

8. Difference From Simulation or Digital-Twin Systems Alone

Simulation systems may estimate the effect of a proposed action. However, simulation alone may be advisory. A simulated result may not prevent a downstream tool, API, database, actuator, payment module, communication system, storage layer, or network controller from executing the action.

The disclosed architecture uses consequence simulation as a pre-effectuation predicate. The simulation result may be bound to the Candidate Act, Result-Consequence Acceptance Envelope, validation receipt, State-Entangled Finality Proof, scoped capability, Execution Handle, and Finality Sink verification context.

Therefore, the simulation is not merely a prediction. It becomes part of a machine-verifiable finality condition for effectuation.

9. Difference From Binary Allow-or-Deny Systems

Many control systems treat acts as either allowed or denied. This binary structure may be Technically impractical for artificial-intelligence-generated acts because many acts are elevated-risk but still necessary.

The disclosed architecture supports graduated finality. A Candidate Act may be allowed, denied, quarantined, routed for review, redacted, delayed, sandboxed, reduced in scope, made reversible, executed as a canary, or classified as escalated but still allowable.

In Escalated Conditional Finality Mode, the Candidate Act remains non-effective while additional controls are applied. Such controls may include protected human approval, multi-party approval, fresh sink attestation, shortened expiration, reduced value, redaction, stricter Execution Authorization Scope Object, enhanced receipt generation, post-effectuation monitoring, or other safeguards.

This allows the architecture to support practical Technical deployment without sacrificing finality control.

10. Difference From Systems That Validate Once and Execute Later

Some systems validate an action at one time and execute it later. Between validation and execution, relevant state may change. Policy may change, revocation may occur, source authority may become stale, recipient status may change, jurisdictional conditions may change, sink trust may fail, approval may expire, or risk classification may change.

The disclosed architecture addresses this state-drift and time-of-check-to-time-of-use problem by requiring current-state finality. In some embodiments, validation, consequence simulation, policy-epoch verification, revocation-epoch verification, sink attestation, nonce generation, monotonic counter advancement, validation receipt generation, and capability or Execution Handle release occur within a protected atomic transaction.

The Finality Sink may accept the capability or Execution Handle only if the current state remains cryptographically congruent with the state bound during validation.

Thus, historical approval is insufficient for current effectuation.

11. Difference From Software-Only Enforcement

Software-only enforcement may depend on ordinary application logic, workflow rules, service configuration, API checks, policy middleware, model-layer controls, or output-layer filters. Such controls may be bypassed if another path to effectuation exists.

The disclosed architecture places finality control at or near the Finality Sink. In advanced non-completability embodiments, the Finality Sink lacks completion material unless protected finality validation succeeds.

Accordingly, the invention provides a machine-level consequence-control mechanism rather than merely software-level permission logic.

Distinction From Black-Box AI Decisioning

The disclosed consequence simulation workflow is not a black-box artificial-intelligence approval mechanism. A black-box AI system may rely on opaque model reasoning, hidden scoring, statistical inference, or an unexplained classifier output to decide whether an output is acceptable.

In contrast, the disclosed system does not treat the artificial-intelligence model's internal reasoning as authority. The AI-generated output is first converted into a Candidate Act and held in a non-effective state. The Protected Enforcement Domain then evaluates machine-verifiable predicates, including HCAD integrity, ALF validation, RBD matching, OPC validation, FCU verification, RCAE comparison, consequence simulation, policy-epoch verification, revocation-epoch verification, receipt generation, and Finality Sink compatibility.

The consequence simulation determines what the Candidate Act would cause if effectuated by the applicable Finality Sink. It may determine whether data would be disclosed, money would move, a database would be mutated, a network configuration would change, a model would be updated, a message would be transmitted, a legal or operational commitment would be created, or a physical actuator would move.

Accordingly, the simulation is not a second AI opinion and is not a hidden model score. It is a protected pre-effectuation analysis of the external consequence of the Candidate Act. The simulation result may be bound to the RCAE, LAVR, State-Proof, scoped capability, Execution Handle, policy epoch, revocation epoch, nonce, and Finality Sink identity.

Thus, the invention does not require the AI model itself to be transparent. The AI model may remain probabilistic or partially opaque, but the output-to-consequence path is machine-verifiable, receipt-bound, scope-bound, epoch-bound, and sink-verified.

Black-box AI governs by trusting or scoring model output. This architecture governs by protected proof before consequence.

12. Summary of Difference From Existing Approaches

The disclosed architecture differs from existing technical approaches because it does not merely authenticate a user, approve a model, filter a prompt, moderate output, authorize an API call, run code inside a protected environment, simulate a result, obtain ordinary human approval, or record an event after execution.

Instead, the architecture performs a protected output-to-consequence finality sequence:

Artificial-Intelligence Output → Candidate Act → Non-Effective State → Hash-Linked Candidate Act Descriptor → Algorithmic Logic Fingerprint Validation → Runtime Behavioral Descriptor Matching → Output Provenance Capsule Validation → Factual Claim Unit Verification → Result-Consequence Acceptance Envelope Boundary → Consequence Simulation → Graduated or Escalated Finality Decision → Capability or Execution Handle Release → Finality Sink Verification → Effectuation or Denial

In advanced non-completability embodiments, the sequence is further strengthened:

Candidate Act → Result-Consequence Acceptance Envelope → Execution Authorization Scope Object → Atomic Receipt-With-Release → Execution Handle → Hardware-Bound Sink Verification → Reconstruction, Unsealing, Combining, or Activation of Missing Execution Material → Effectuation Only if Completion Succeeds

The core distinction is:

Existing systems may authorize, moderate, simulate, or audit. The disclosed architecture prevents an artificial-intelligence-generated Candidate Act from becoming consequence unless protected finality succeeds at the sink boundary.

SUMMARY OF THE INVENTION

The disclosed invention provides a hallucination-resistant output-to-consequence finality architecture for artificial-intelligence-generated acts.

The invention begins from a core rule:

An artificial-intelligence-generated output is not authority to act.

The output is converted into a Candidate Act and placed in a non-effective state. The Candidate Act remains non-effective until a Protected Enforcement Domain validates required finality predicates and the applicable Finality Sink verifies a scoped capability or Execution Handle.

The invention includes a base inventive path, a Technical graduated-finality path, and a advanced non-completability non-completability path.

FIRST INVENTIVE PATH — BASE OUTPUT-TO-CONSEQUENCE FINALITY ARCHITECTURE

In the base architecture, the system treats every effect-capable artificial-intelligence-generated output as a Candidate Act.

The Candidate Act may represent a message, recommendation, command, tool call, data disclosure, payment instruction, network-configuration change, database mutation, memory write, model update, physical actuation command, settlement instruction, communication instruction, or other effect-capable operation.

The Candidate Act is not permitted to become externally effective merely because it has been generated. Instead, the Candidate Act is held in a non-effective state.

The base finality path may include the following sequence:

Artificial-Intelligence Output → Candidate Act → Non-Effective State → HCAD → ALF Validation → RBD Matching → OPC Validation → FCU Verification → RCAE Generation → Consequence Simulation → Output Finality Predicate Evaluation → Protected Approval if Required → Scoped Non-Bearer Capability Release → Finality Sink Verification → Effectuation or Denial

This base path is the broad inventive foundation.

Base Path Step 1 — Candidate Act Formation

An artificial-intelligence-generated output is converted into a Candidate Act.

The Candidate Act may include output content, output hash, intended recipient, intended tool, intended Finality Sink, purpose, jurisdiction, data class, risk class, requested consequence, timestamp, artificial-intelligence-system identifier, model identifier, workflow identifier, Algorithmic Logic Fingerprint identifier, and other context fields.

The Candidate Act is placed into a non-effective state. In this state, the Candidate Act may be staged, buffered, reviewed, simulated, redacted, delayed, sandboxed, escalated, quarantined, rejected, or prepared for validation, but it is not yet permitted to create external consequence.

Base Path Step 2 — HCAD Generation

The Protected Enforcement Domain may generate a Hash-Linked Candidate Act Descriptor.

The Hash-Linked Candidate Act Descriptor may bind the Candidate Act to the output hash, originating artificial-intelligence system, Algorithmic Logic Fingerprint identifier, Runtime Behavioral Descriptor identifier, Output Provenance Capsule identifier, Result-Consequence Acceptance Envelope identifier, policy epoch, revocation epoch, Finality Sink identity, recipient, purpose, jurisdiction, risk class, timestamp, nonce, and evidence references.

The Hash-Linked Candidate Act Descriptor provides a machine-verifiable descriptor for the Candidate Act and may be hash-linked to validation evidence, receipts, policy states, revocation states, or prior finality state.

Base Path Step 3 — ALF Validation

The Protected Enforcement Domain may validate an Algorithmic Logic Fingerprint.

The Algorithmic Logic Fingerprint may represent an approved model state, workflow state, system prompt policy, tool-use policy, retrieval policy, memory policy, safety policy, model-router configuration, or other approved computational logic state.

Algorithmic Logic Fingerprint validation confirms that the Candidate Act was generated by or under an approved computational logic configuration.

However, Algorithmic Logic Fingerprint validation does not prove that the specific output is correct, safe, lawful, Technically desired, or permitted to become consequence. Algorithmic Logic Fingerprint validation is a process-integrity predicate, not final authority to act.

Base Path Step 4 — RBD Matching

The system may collect or receive a Runtime Behavioral Descriptor associated with generation of the Candidate Act.

The Runtime Behavioral Descriptor may identify tools accessed, retrieval sources used, memory regions accessed, prompt segments used, policy branches followed, model-router decisions, external calls attempted, confidence signals, risk signals, event sequences, resource usage, or other runtime behavioral evidence.

The Runtime Behavioral Descriptor may be compared with an Algorithmic Logic Fingerprint-bound approved behavioral envelope. A match indicates that observed runtime behavior remained within an expected envelope.

However, Runtime Behavioral Descriptor matching does not by itself authorize effectuation. Runtime behavior may be valid while the specific output remains unsupported, stale, unsafe, or outside the permitted consequence boundary.

Base Path Step 5 — OPC Generation and Validation

The system may generate or receive an Output Provenance Capsule.

The Output Provenance Capsule may identify sources, retrieval records, tool outputs, database records, sensor records, timestamps, model-state identifiers, Algorithmic Logic Fingerprint identifiers, Runtime Behavioral Descriptor identifiers, jurisdictional assumptions, confidence indicators, limitation flags, factual claim units, permitted-use constraints, recipient constraints, data-classification labels, policy-epoch identifiers, revocation-epoch identifiers, and evidence hashes.

The Output Provenance Capsule does not guarantee truth. It provides structured evidence, assumptions, and provenance context used to determine whether the Candidate Act may become consequence.

If required provenance is missing, fabricated, stale, revoked, contradictory, unsupported, or outside permitted use, the Candidate Act remains non-effective.

Base Path Step 6 — FCU Verification

For hallucination-sensitive, high-risk, legal, financial, medical, operational, security, public-safety, or compliance-sensitive outputs, the system may extract factual assertions into Factual Claim Units.

Each Factual Claim Unit may include an asserted fact, source reference, source hash, freshness value, confidence value, jurisdictional assumption, contradiction status, limitation flag, and permitted-use scope.

The Protected Enforcement Domain may determine whether each required Factual Claim Unit is supported by permitted evidence. If an Factual Claim Unit lacks support, conflicts with an approved source, is stale beyond a permitted freshness window, has insufficient confidence, or exceeds source scope, the Candidate Act may be blocked, downgraded, redacted, routed for protected review, or classified for escalated conditional finality.

Base Path Step 7 — RCAE Generation or Retrieval

The system may generate, retrieve, or derive a Result-Consequence Acceptance Envelope.

The Result-Consequence Acceptance Envelope defines the permitted boundary within which the Candidate Act may become externally effective.

The Result-Consequence Acceptance Envelope may specify permitted purpose, permitted recipient, permitted data class, permitted jurisdiction, permitted tool, permitted Finality Sink, financial limit, operational limit, safety class, risk threshold, freshness requirement, source requirement, human-review requirement, time window, revocation epoch, policy epoch, and permitted consequence type.

The Result-Consequence Acceptance Envelope converts a desired or permitted output into a technical consequence boundary.

Base Path Step 8 — Consequence Simulation

Before capability release, the system may simulate, predict, preflight, dry-run, or otherwise analyze the effect of the Candidate Act if effectuated by the applicable Finality Sink.

The simulation may determine whether the Candidate Act would cause data disclosure, money movement, settlement, network reconfiguration, physical actuation, database mutation, memory write, model update, legal commitment, external transmission, tool dispatch, or another consequence.

If the predicted consequence exceeds the Result-Consequence Acceptance Envelope, violates policy, exceeds risk, conflicts with jurisdiction, lacks authority, or targets an incompatible sink, no full-effect capability is released.

Base Path Step 9 — Output Finality Predicate Evaluation

The Protected Enforcement Domain evaluates whether the Candidate Act satisfies required finality predicates.

Such predicates may include Algorithmic Logic Fingerprint validity, Runtime Behavioral Descriptor match, Output Provenance Capsule validity, Factual Claim Unit support, Result-Consequence Acceptance Envelope compliance, consequence simulation pass, recipient permission, purpose permission, jurisdiction permission, data-class permission, policy-epoch freshness, revocation-epoch freshness, protected human approval where required, and Finality Sink compatibility.

If mandatory predicates pass, the workflow may proceed to capability release. If one or more predicates fail, the Candidate Act may be denied, quarantined, downgraded, redacted, delayed, sandboxed, or routed for protected review.

Base Path Step 10 — Scoped Non-Bearer Capability Release

If required predicates are satisfied, the Protected Enforcement Domain releases a scoped non-bearer finality capability.

The capability may be bound to the Candidate Act, Hash-Linked Candidate Act Descriptor, Finality Sink, purpose, recipient, jurisdiction, data class, time window, policy epoch, revocation epoch, permitted consequence, nonce, and validation evidence.

The capability cannot be reused for a different act, sink, recipient, purpose, jurisdiction, data class, or consequence.

Possession of the capability data alone is insufficient. The capability is accepted only when the Finality Sink verifies the matching sink-side context.

Base Path Step 11 — Finality Sink Verification

The Finality Sink verifies the scoped non-bearer capability before effectuation.

The Finality Sink may check signature or message authentication code, sink identity, Candidate Act hash, scope, nonce, freshness, policy epoch, revocation epoch, recipient, purpose, jurisdiction, data class, and permitted consequence.

If verification passes, the Finality Sink effectuates the Candidate Act within the permitted scope.

If verification fails, the Candidate Act remains non-effective or is rejected.

SECOND INVENTIVE PATH — CRYPTOGRAPHIC NON-COMPLETABILITY ARCHITECTURE

Foundational Distinction

The non-completability architecture does not strengthen finality by adding validation steps to the base path. It changes the nature of enforcement itself. In the base architecture, a Finality Sink is instructed to refuse acts that lack a valid scoped capability — the sink retains structural ability to complete any act and exercises judgment not to. In the non-completability architecture, that structural ability is removed. The Finality Sink is cryptographically and mechanically incomplete. It holds only a fragment of the execution material required for effectuation. The remaining fragment exists nowhere in the ordinary execution environment and cannot be derived, extracted, or approximated by any software path, agent output, network attacker, or replayed authority object. Completion becomes technically impossible until the Protected Enforcement Domain — having verified all required finality predicates — releases, reconstructs, or activates the missing fragment at the sink boundary.

This is the core inventive distinction: **enforcement by structural incompleteness, not by instructed refusal.**

The execution-dependency sequence is:

Candidate Act → RCAE Boundary Derivation and Locking → Perturbation-Tested Consequence Margin Verification → EASO Cryptographic Scope Formation → Atomic Finality Transaction → Execution Handle Minting → Sink-Side Reconstruction, Unsealing, Combining, or Activation → Effectuation Only if Completion Material Assembles Within Scope

Step 1 — Consequence Boundary Derivation and RCAE Locking

Before EASO formation, the Protected Enforcement Domain derives a locked Result-Consequence Acceptance Envelope from the Candidate Act, predicted consequence, algorithmic logic fingerprint result, runtime behavioral descriptor, output provenance capsule, policy epoch, revocation epoch, and Finality Sink identity. The RCAE does not merely describe the permitted consequence — it is cryptographically sealed against the Candidate Act hash and the current revocation epoch such that any modification of the Candidate Act, any epoch advancement, or any sink substitution independently invalidates the RCAE without requiring downstream re-validation.

In some embodiments, the Protected Enforcement Domain performs consequence-proximity verification before RCAE is finalized: the predicted consequence is evaluated against a perturbation radius encompassing parameterized variations of the Candidate Act across recipient space, financial amount space, jurisdictional space, and data-class space. If any perturbation within the tested radius reaches a prohibited consequence class, the RCAE boundary is tightened to exclude the proximity neighborhood before EASO formation proceeds. The perturbation set identity, tested radius, and nearest prohibited-consequence neighbor distance are recorded in the RCAE and carried forward into the EASO and validation receipt, providing a cryptographically attested margin between the permitted consequence and the nearest prohibited boundary.

Step 2 — EASO Cryptographic Scope Formation

The Protected Enforcement Domain derives an Execution Authorization Scope Object from the locked RCAE, the Candidate Act, the predicted consequence, the Finality Sink identity, the attested sink hardware identity, the policy epoch, the revocation epoch, and the permitted scope across recipient, purpose, jurisdiction, data class, financial value, operational scope, and consequence type.

The RCAE and the EASO serve structurally distinct functions. The RCAE defines what consequence is permitted. The EASO converts that permitted consequence boundary into a cryptographically execution-enforceable scope — a technical envelope within which execution material may be assembled and outside which assembly is structurally impossible regardless of any software instruction, agent output, or access-control decision.

In some embodiments, the EASO additionally encodes: required escalation module results and their binding conditions; mandatory sink attestation measurements against which the Finality Sink's hardware identity must match; Consequence-Class Inheritance Tag specifying the ceiling consequence class propagatable to any downstream agent chain; required assurance profile identifier specifying which level of sink-side verification is mandatory; and rollback-capability escrow conditions where applicable. Each encoded parameter is cryptographically bound to the Candidate Act hash and the current revocation epoch. A downstream component cannot expand, substitute, or silently downgrade any EASO parameter without invalidating the EASO independently.

Step 3 — Execution Handle Minting

Following successful EASO formation, the Protected Enforcement Domain mints an Execution Handle. The Execution Handle is bound to: Candidate Act hash, Hash-Linked Candidate Act Descriptor hash, EASO hash, RCAE hash, output provenance capsule hash, validation receipt identifier, algorithmic logic fingerprint identifier, runtime behavioral descriptor identifier, Finality Sink identity, attested sink hardware identity, permitted recipient, permitted purpose, permitted jurisdiction, permitted consequence class, policy epoch, revocation epoch, expiration time, monotonic counter value, and fresh nonce.

The Execution Handle does not carry a general-purpose execution key, a long-lived credential, or any authority beyond the narrowly scoped completion material required for the validated Candidate Act at the specified Finality Sink. In some embodiments, the Execution Handle includes, encodes, carries, wraps, seals, signs, MACs, encrypts, or enables conditional access to the execution material required by the Finality Sink — but only in a form that is itself incomplete without the sink-side fragment held within the Finality Sink's protected domain.

In some embodiments where split-knowledge execution is required, the Execution Handle carries one share of a threshold execution material set. Additional shares are held by independent protected components — an enterprise hardware security module, an external approval device, a jurisdictional enforcement module, or an independent protected sink domain — and must converge at the Finality Sink within a bounded verification time window. No single share, including the share carried by the Execution Handle, is independently sufficient for effectuation.

Step 4 — Atomic Finality Transaction

Validation, receipt generation, protected state mutation, policy-epoch verification, revocation-epoch verification, nonce generation, monotonic counter advancement, sink attestation binding, and Execution Handle release are performed as a single protected atomic finality transaction. No partial execution of this transaction produces a usable Execution Handle.

If any required step within the transaction fails — including predicate validation failure, receipt write failure, state mutation failure, epoch mismatch, nonce collision, counter non-advancement, sink attestation mismatch, or execution-material reconstruction failure — the entire transaction is aborted. No Execution Handle is released. No receipt is committed. No partial authority object is produced. The Candidate Act remains non-effective with no intermediate state that could be exploited by a concurrent or replayed execution path.

The atomicity guarantee directly addresses time-of-check-to-time-of-use risk. Because validation, receipt commitment, and Execution Handle release occur as a single indivisible operation, there is no window between predicate satisfaction and authority release during which policy state, revocation state, or sink attestation could change without being reflected in the Execution Handle binding. A policy-epoch advancement or revocation-epoch advancement occurring after the atomic transaction begins but before it completes causes the transaction to abort rather than to release an Execution Handle that does not reflect current state.

In some embodiments, the atomic finality transaction is executed within a protected hardware boundary — a hardware security module, secure enclave, trusted execution environment, or protected microcontroller — such that the transaction state is inaccessible to ordinary software processes throughout execution, and the Execution Handle is released from within the protected boundary directly to the Finality Sink channel without traversing an unprotected software layer.

Step 5 — Sink-Side Structural Incompleteness and Completion Material Assembly

The Finality Sink holds incomplete effectuation material. This incompleteness is structural, not instructional. The incomplete material may include: a sink-side key share, partial threshold signature share, sealed token fragment, incomplete command authenticator, locked egress primitive, locked database commit primitive, sealed payment authorization fragment, disabled actuator enable value, hardware-held final-step state, or another incomplete execution component whose form is specific to the Finality Sink's effectuation interface.

The Protected Enforcement Domain holds, derives, or conditionally releases the complementary execution material — the missing fragment — only after all finality predicates are satisfied and the atomic finality transaction completes successfully. The complementary material is not stored in any software-accessible location during normal operation. It is derived on demand from protected state, EASO parameters, and the valid Execution Handle at transaction completion time, and is transmitted through a protected channel to the Finality Sink.

Upon receiving the Execution Handle, the Finality Sink performs sink-side verification: Execution Handle authenticity, EASO hash correspondence, RCAE hash correspondence, Candidate Act hash correspondence, sink identity match, attested hardware identity match, policy-epoch currency, revocation-epoch currency, nonce freshness, counter value validity, expiration check, permitted recipient scope, permitted purpose scope, permitted jurisdiction scope, permitted consequence class, and — where applicable — threshold share convergence confirmation, rollback-capability escrow confirmation, and mandatory escalation module result presence.

If all sink-side verification predicates are satisfied, the Finality Sink combines, unseals, reconstructs, or activates the missing execution material from the sink-side fragment and the Protected Enforcement Domain-released complementary material. Combination succeeds only within the scope defined by the EASO. If the Execution Handle is invalid, stale, replayed, copied to a different sink, hardware-identity-mismatched, act-mismatched, epoch-mismatched, nonce-exhausted, counter-regressed, or outside permitted scope in any dimension, the sink-side fragment and the complementary material do not produce valid execution material. The Finality Sink remains structurally non-completable. No software instruction, access-control override, agent output, or replayed token can substitute for the missing material or cause the combination to succeed outside the EASO scope.

Only after combination succeeds does the Finality Sink execute the Candidate Act — and only within the consequence boundary defined by the RCAE and enforced by the EASO. The effectuation boundary is not a software policy check applied at execution time. It is the mathematical limit of what the assembled execution material is capable of authorizing.

Step 6 — Post-Assembly State and Inheritance Propagation

Upon successful completion material assembly and effectuation, the Finality Sink generates a sink-verification record bound to: Candidate Act hash, EASO hash, RCAE hash, Execution Handle identifier, validation receipt identifier, sink identity, attested hardware identity, nonce, policy epoch, revocation epoch, counter value, and effectuation timestamp. The sink-verification record is transmitted to the Protected Enforcement Domain and committed to the receipt state store, completing the finality evidence chain.

Where the effectuated Candidate Act produces an output that becomes the input to a downstream artificial-intelligence agent, the Consequence-Class Inheritance Tag encoded in the EASO is propagated to the Protected Enforcement Domain evaluating the downstream Candidate Act. The downstream RCAE is bounded by the inherited ceiling. The downstream EASO cannot specify a consequence class exceeding the inherited ceiling regardless of the downstream agent's otherwise applicable policy scope. The inheritance ceiling is narrowed monotonically across agent chain hops and cannot be expanded by any downstream component, downstream escalation decision, or downstream policy epoch.

Where rollback-capability escrow was specified in the EASO, the Protected Enforcement Domain activates the post-effectuation monitoring window. The rollback capability remains sealed in the

protected escrow domain. If a monitored failure condition — including source revocation, policy contradiction, revocation-epoch advancement, sink-trust degradation, canary failure, fraud signal, or jurisdictional block — is detected within the monitoring window, the rollback capability is released to the Finality Sink and bounded reversal is executed. Upon expiry of the monitoring window without detection of a monitored failure condition, the rollback capability is destroyed or archived and the finality transaction is treated as unconditionally complete.

Architectural Summary

The non-completability architecture achieves a property that instructed-refusal architectures cannot: the absence of a valid Execution Handle is not a condition that software can override, an agent can circumvent, or a network attacker can exploit, because the execution material required for completion does not exist in assembled form anywhere in the system until the Protected Enforcement Domain releases the complementary fragment following successful atomic finality validation. Effectuation is not the result of a decision to permit. It is the result of a cryptographic assembly that is only possible when all required predicates are satisfied, all required shares converge, all required epochs are current, all required sink attestations match, and the atomic finality transaction completes without abort. The system does not ask whether the Candidate Act should be allowed. It asks whether the execution material can be assembled — and the answer is structurally no until finality succeeds.

LATENCY-AWARE IMPLEMENTATION OVERVIEW

In some embodiments, the architecture is implemented using a latency-aware separation between a cold validation path, a nearline preparation path, and a hot finality path.

The architecture does not require every validation operation, simulation operation, provenance check, attestation check, human approval step, or adversarial perturbation test to be performed at the exact moment of sink-side effectuation. Computationally heavier operations may be precomputed, cached, staged, compiled, delegated to a Protected Enforcement Domain, or performed before the Candidate Act reaches the Finality Sink.

The Finality Sink may therefore perform only compact verification and completion operations at the time of effectuation. These operations may include Candidate Act hash verification, nonce verification, expiration verification, policy-epoch comparison, revocation-epoch comparison, sink-identity verification, validation-receipt reference verification, State-Proof verification, and capability or Execution Handle verification or reconstruction.

Accordingly, the architecture may support low-latency operation for routine acts, bounded additional latency for medium-risk acts, and stronger validation for high-risk acts. Latency may therefore be proportional to consequence risk rather than fixed at the highest validation cost for every Candidate Act.

CORE INVENTIVE PRINCIPLE

An artificial-intelligence-generated output is not treated as authority to act.

The output is treated as a non-effective Candidate Act until output-level, consequence-level, provenance-level, factual-support-level, jurisdiction-level, epoch-level, and sink-level finality predicates are satisfied.

The disclosed architecture therefore establishes the following technical principle:

Computation does not imply consequence.

In some embodiments:

Algorithmic Logic Fingerprint validation establishes approved computational logic state.

Runtime Behavioral Descriptor matching establishes observed runtime behavior within an approved envelope.

Output Provenance Capsule validation establishes provenance and evidence context.

Factual Claim Unit verification establishes factual support where required.

Result-Consequence Acceptance Envelope comparison establishes permitted consequence boundary.

Consequence simulation predicts the external effect before release.

Graduated finality determines whether the act is ordinary allowed, escalated allowed, reduced in scope, redacted, delayed, sandboxed, reviewed, quarantined, or denied.

A scoped non-bearer finality capability or Execution Handle is released only after protected validation.

The Finality Sink effectuates only after verifying the capability or Execution Handle.

In advanced non-completability embodiments, the Finality Sink cannot technically complete the Candidate Act unless protected reconstruction, unsealing, combining, or activation succeeds.

THIRD INVENTIVE PATH — GRADUATED CONDITIONAL FINALITY AND CURRENT-STATE EXECUTION CONGRUENCE

Foundational Distinction

The base finality path produces a binary outcome: a Candidate Act is either released under a scoped capability or held non-effective. The non-completability path converts the release mechanism into a cryptographic assembly dependency. This third inventive path addresses a distinct structural problem neither prior path resolves: the space between full permission and outright denial contains a large and commercially significant population of Candidate Acts that are elevated-risk but technically necessary, near-boundary but not prohibited, and permissible under controls that the binary model cannot express.

Collapsing this space into denial is operationally incorrect — it forces unnecessary refusal of acts that could be safely effectuated under stricter conditions. Collapsing it into ordinary release is safety-incorrect — it discards the elevated-risk signal and permits acts under controls insufficient for their consequence profile. The graduated conditional finality architecture resolves this by treating consequence-class, risk-score, boundary-proximity, and control-availability as continuous

variables that determine not merely whether a Candidate Act is released but what release path, what authority structure, what verification depth, and what post-effectuation obligations apply.

The second structural problem this path addresses is orthogonal to escalation: a Candidate Act validated as safe at evaluation time may be unsafe at execution time. Policy state, revocation state, source authority, recipient authority, jurisdictional status, model approval, data classification, human approval, risk classification, and Finality Sink trust can each change between validation and effectuation. An architecture that does not enforce current-state congruence at execution time is an architecture that can be exploited by temporal drift — a Candidate Act validated against a since-revoked policy, a since-compromised source, or a since-degraded sink becomes a vector for unauthorized effectuation through the passage of time alone.

These two problems — the false binary between permission and denial, and the exploitation of temporal drift between validation and execution — are addressed jointly in this path.

Section I — Graduated Finality Classification

1.1 Consequence-Continuous Classification

Upon completing primary predicate validation, the Protected Enforcement Domain classifies the Candidate Act into a graduated finality outcome rather than a binary result. The classification is determined by a composite evaluation across: predicted consequence class and severity, proximity of predicted consequence to permitted boundary, risk score relative to ordinary-release threshold and denial threshold, algorithmic logic fingerprint result, runtime behavioral descriptor result, output provenance capsule integrity, finality-confidence unit results and their confidence intervals, data classification, recipient authorization status, jurisdictional requirements, Finality Sink trust state, reversibility of predicted consequence, and perturbation-tested boundary margin.

The available finality outcomes form a structured spectrum:

Ordinary release applies when all predicates are satisfied within normal tolerance, the predicted consequence is bounded away from permitted-class boundaries by the required perturbation margin, and no elevated-risk condition is present. The Candidate Act proceeds under the standard scoped capability or Execution Handle path.

Reduced-scope release applies when the Candidate Act satisfies permitted-class requirements but one or more scope parameters — recipient set, financial value, data fields, operational breadth, or jurisdictional reach — exceed what is necessary for the predicted consequence. The Protected Enforcement Domain derives a narrower Result-Consequence Acceptance Envelope and Execution Authorization Scope Object, and releases authority only within the reduced scope. The Candidate Act remains non-effective outside the reduced scope regardless of any software instruction.

Redacted release applies when the Candidate Act contains data fields, content elements, or operational parameters that independently exceed permitted classification even when the act as a whole is permissible. Selected fields are cryptographically redacted before Execution Handle minting. The Finality Sink verifies that redaction is present and structurally enforced — not merely omitted from a display layer — before effectuation proceeds.

Delayed release applies when the Candidate Act is permitted but temporal conditions — rate limits, regulatory settlement windows, downstream system readiness, mandatory review periods, or time-bound jurisdictional requirements — require that effectuation occur at a defined future time rather than immediately. The Protected Enforcement Domain issues a time-locked Execution Handle

whose activation window opens at the defined future time and closes at a defined expiration. The Finality Sink verifies activation-window currency before assembling completion material.

Reversible release applies when the predicted consequence is permitted but partially or fully reversible within a defined rollback scope, and the consequence profile warrants maintaining rollback authority during a post-effectuation monitoring window. The system generates a paired forward capability and rollback capability as described in the non-completability architecture. The rollback capability is held in protected escrow and remains cryptographically activatable within the monitoring window.

Canary release applies when the Candidate Act has not previously been executed against the target Finality Sink configuration, the sink-trust assessment is non-zero but below the ordinary-release threshold, or perturbation testing reveals elevated sensitivity near the consequence boundary. A reduced-scope or reduced-value instance of the Candidate Act is released first. Post-effectuation monitoring evaluates the canary result. If the canary result satisfies monitoring predicates, the full Candidate Act may proceed under a separate finality decision. If the canary result fails monitoring predicates, the full Candidate Act is denied or re-classified.

Sandbox-first release applies when the Candidate Act requires execution against a controlled environment before any production Finality Sink receives the authority. The sandbox Finality Sink is configured with equivalent verification semantics but isolated consequence scope. Sandbox execution results are evaluated as escalation module inputs before a production Execution Handle is minted.

Escrowed release applies when the Candidate Act is permitted but requires a confirmation window before external consequences are finalized, implementing the Commitment Token and Release Token architecture described in the non-completability path. The Candidate Act is staged in non-effective escrow state while current-state re-verification completes.

Protected-review staging applies when the Candidate Act cannot be automatically classified into any release outcome because required predicate inputs are insufficient, conflicting, or outside the confidence interval required for automated decision. The Candidate Act is routed to a protected human review domain where a Protected Human Approval and Finality Token is generated. The Candidate Act remains non-effective until the PHAFT is produced and bound into an escalated Execution Handle.

Quarantine applies when the Candidate Act presents characteristics consistent with prohibited-class consequences but the evidence is insufficient for definitive denial. The Candidate Act is isolated, logged, and held pending additional predicate evaluation or protected review. No authority object is generated during quarantine.

Denial applies when the Candidate Act is definitively prohibited, malicious, outside all permitted consequence envelopes, directed to a revoked sink, directed to a prohibited recipient, or structurally incapable of being safely constrained by any available escalation control.

Section II — Escalated Conditional Finality

2.1 Escalated-but-Allowable Classification

A Candidate Act classified as escalated-but-allowable presents an elevated-risk condition that does not require denial and can be reduced to a permitted consequence scope by application of additional controls. The elevated-risk condition may arise from: predicted consequence proximity to a

permitted-class boundary within the perturbation margin; risk score above the ordinary-release threshold but below the denial threshold; finality-confidence unit results carrying reduced but acceptable confidence intervals; source provenance complete but near a freshness boundary; sensitive but permitted data classification; sensitive but authorized recipient; high-value but authorized financial amount; jurisdiction requiring additional compliance validation; Finality Sink in a degraded-but-trusted attestation state; reversible or partially reversible consequence warranting rollback escrow; or consequence-proximity perturbation testing revealing a sensitive neighborhood without a prohibited neighbor within the tested radius.

The escalated-but-allowable classification is not a weakened approval. It is a distinct finality path that maintains the Candidate Act in non-effective state while assembling a stricter authority structure than ordinary release would require. The Candidate Act may become externally effective only after the escalated authority structure is fully assembled, verified, and bound.

2.2 Escalated Consequence Boundary Derivation

Upon escalated-but-allowable classification, the Protected Enforcement Domain derives an escalated Result-Consequence Acceptance Envelope. The escalated RCAE is strictly narrower than the ordinary RCAE applicable to the same Candidate Act class. It may limit: permitted recipient to a specific verified entity rather than a permitted class; permitted financial value to a reduced amount below the act's face value; permitted jurisdiction to a single verified jurisdiction rather than a permitted multi-jurisdiction scope; permitted data class to a subset excluding sensitive fields; permitted operational scope to a defined functional subset; permitted consequence type to reversible consequences only; and Finality Sink to a specific attested sink instance rather than any instance within a permitted sink class.

The escalated RCAE is sealed against the Candidate Act hash and the current revocation epoch. Any component that receives the escalated RCAE cannot expand its scope. The Finality Sink rejects any Execution Handle whose embedded EASO specifies a consequence scope exceeding the escalated RCAE.

2.3 Escalation Control Assembly

The Protected Enforcement Domain selects a combination of escalation controls required before an escalated Execution Handle is minted. Selection is determined by the elevated-risk conditions identified during classification, the escalated RCAE scope, and the policy epoch. Selected controls are bound into the escalated EASO and cannot be omitted, substituted, or downgraded by any downstream component.

Available escalation controls include, individually or in combination:

Protected Human Approval and Finality Token: A PHAFT bound to the Candidate Act hash, predicted consequence, escalation reason, escalated RCAE hash, escalated EASO hash, approving role identity, authenticated user presence verification, approval time, policy epoch, revocation epoch, and Finality Sink identity. An ordinary software approval, workflow approval, chat instruction, email confirmation, stale approval, replayed approval, or synthetic interface event does not constitute a PHAFT. The PHAFT is a machine-verifiable artifact generated by a protected approval domain and cryptographically bound to the specific escalated finality decision.

Multi-Party or Quorum Approval: A defined combination of PHAFTs from distinct approval roles, domains, or organizational authorities. The escalated EASO specifies the required quorum composition and binding conditions. No subset below the quorum threshold produces a valid escalated Execution Handle.

Modular Escalation Results: Machine-verifiable outputs from protected escalation modules including jurisdictional compliance validation, source-permission re-verification, secondary model verification, adversarial perturbation re-testing at tighter radius, fresh sink attestation acquisition, canary execution result evaluation, rollback-capability preparation confirmation, and residual-risk calculation. Each module result is cryptographically bound to the Candidate Act hash, policy epoch, revocation epoch, nonce, module identity, and Finality Sink identity. The escalated EASO specifies which module results are mandatory for Execution Handle minting.

Shortened Expiration: The escalated Execution Handle carries an expiration time materially shorter than the ordinary expiration applicable to the same Candidate Act class. The Finality Sink verifies expiration currency before assembling completion material. A stale escalated Execution Handle does not extend its validity by re-presentation.

Fresh Sink Attestation: The Finality Sink must produce a current attestation measurement bound to the escalated EASO and the current policy epoch. A cached or stale attestation does not satisfy the fresh attestation requirement. If the Finality Sink cannot produce a current attestation, the escalated Candidate Act remains non-effective.

Residual Risk Receipt Generation: Before minting the escalated Execution Handle, the Protected Enforcement Domain generates a Residual Risk Receipt recording: the elevated-risk conditions present, the escalated controls applied, the escalation module results received, the assumptions relied upon, the limitation flags remaining, the protected approvals obtained, the escalated RCAE scope, the escalated EASO scope, the perturbation margin at escalated boundary, and the residual risk that remains after control application. The Residual Risk Receipt is committed to the protected receipt state store before the escalated Execution Handle is released.

Post-Effectuation Monitoring: A defined monitoring window and monitoring-condition set are bound into the escalated EASO and the escalated Execution Handle. The Finality Sink is required to transmit post-effectuation state signals to the Protected Enforcement Domain during the monitoring window. If a monitored failure condition is detected — including source revocation, policy contradiction, downstream consequence exceeding escalated RCAE scope, canary failure, recipient change, jurisdictional block, or sink-trust degradation — the Protected Enforcement Domain activates available remediation including rollback capability release, related capability revocation, revocation epoch advancement, future-act suspension for the same consequence class, and corrective receipt generation.

2.4 Escalated Execution Handle Minting

The escalated Execution Handle is minted only after all mandatory escalation controls are assembled, verified, and cryptographically bound. The escalated Execution Handle is narrower than an ordinary Execution Handle in every bound parameter: shorter expiration, tighter RCAE scope, stricter EASO scope, mandatory escalation module result references, mandatory PHAFT references where required, mandatory monitoring obligation encoding, mandatory rollback-capability escrow reference where applicable, and mandatory Residual Risk Receipt reference.

The Finality Sink verifies that the escalated Execution Handle references all mandatory escalation module results and that the referenced results cryptographically correspond to the same Candidate Act, current policy epoch, current revocation epoch, and current Finality Sink attestation. If any mandatory reference is absent, stale, revoked, epoch-mismatched, or not bound to the same Candidate Act, the Finality Sink refuses to assemble completion material. The escalated Candidate Act remains non-effective.

The escalated Execution Handle cannot be reused for an ordinary release of the same or a related Candidate Act. Its binding parameters are specific to the escalated finality decision and expire with the escalated authority.

Section III — Current-State Execution Congruence

3.1 The Temporal Drift Problem

A Candidate Act validated as satisfying all finality predicates at evaluation time may be unsafe, unauthorized, or prohibited at execution time. Between validation and effectuation, any of the following may change: policy epoch, revocation epoch, source authority status, recipient authorization status, jurisdictional classification, model approval status, data classification, human approval validity, risk classification, Finality Sink trust state, Finality Sink attestation measurement, or the external conditions on which the consequence simulation was based. An architecture that does not enforce current-state congruence at execution time permits exploitation by temporal drift: a Candidate Act validated against a since-revoked source, a since-prohibited recipient, a since-advanced policy epoch, or a since-degraded sink can produce an unauthorized external consequence through the passage of time alone, without any active attack on the finality mechanism.

3.2 Current-State Congruence Requirement

In some embodiments, the architecture enforces current-state execution congruence as a structural requirement: the Candidate Act, predicted consequence, RCAE, EASO, policy epoch, revocation epoch, nonce, validation receipt, and current Finality Sink attestation must be cryptographically congruent at the time of effectuation, not merely at the time of evaluation.

Congruence is enforced through multiple complementary mechanisms operating jointly:

Epoch Binding with Sink-Side Currency Check: The Execution Handle carries the policy epoch and revocation epoch current at minting time. The Finality Sink independently verifies that these epochs remain current at the time the Execution Handle is presented. If the policy epoch has advanced — indicating a policy update since minting — or the revocation epoch has advanced — indicating a revocation event since minting — the Finality Sink refuses to assemble completion material. The Candidate Act does not proceed under a since-invalidated authority.

Nonce and Monotonic Counter Binding: The Execution Handle carries a fresh nonce generated within the atomic finality transaction and a monotonic counter value that must exceed the last recorded counter value for the same Finality Sink channel. The Finality Sink verifies nonce freshness against a protected nonce table and counter value against a protected counter store. A replayed Execution Handle, a copied Execution Handle presented to a different sink, and a stale Execution Handle presented after the nonce window expires each independently fail verification without requiring any other predicate to detect the invalidity.

Sink Attestation Currency: The Finality Sink's hardware identity and attestation measurement are bound into the EASO and Execution Handle at minting time. The Finality Sink produces a current attestation at execution time. If the current attestation measurement does not correspond to the bound measurement — indicating firmware update, configuration change, hardware substitution, or attestation degradation since minting — the Finality Sink refuses to assemble completion material. An Execution Handle minted against a different attestation measurement cannot be used at a since-reconfigured sink.

Receipt-State Congruence: The Execution Handle carries the validation receipt identifier generated during the atomic finality transaction. The Finality Sink verifies that a receipt with the bound identifier exists in protected receipt state and that the receipt's Candidate Act hash, EASO hash, RCAE hash, nonce, policy epoch, revocation epoch, and sink identity cryptographically correspond to the Execution Handle's bound parameters. A fabricated Execution Handle not backed by a committed receipt, and a receipt whose parameters have diverged from the Execution Handle's bound parameters through any post-minting mutation, each independently fail congruence verification.

3.3 The Current-State Congruence Invariant

The technical invariant governing this section may be stated precisely:

A Candidate Act does not become externally effective because it was at some prior time validated as satisfying finality predicates. It becomes externally effective only when, at the moment the Finality Sink assembles completion material, the Candidate Act, its predicted consequence, its RCAE, its EASO, the current policy epoch, the current revocation epoch, the presented nonce, the committed validation receipt, and the current Finality Sink attestation are simultaneously and cryptographically congruent. If any element of this congruence set has changed since the Execution Handle was minted, the assembly fails and the Candidate Act remains non-effective.

This invariant is not enforced by a software check that can be bypassed. It is enforced by the structural impossibility of assembling completion material from parameters that do not cryptographically correspond — the mathematical operation of combining the Execution Handle's embedded material with the sink-side fragment produces valid completion material only when all bound parameters match current protected state. Temporal drift in any bound parameter produces combination failure, not a policy-layer refusal that software could override.

Architectural Summary

The graduated conditional finality path and the current-state congruence requirement together close two structural gaps that binary finality architectures leave open. The graduated path eliminates the false choice between unsafe ordinary release and unnecessary denial for elevated-risk but permissible Candidate Acts, replacing it with a continuous classification producing act-specific authority structures proportional to consequence profile. The current-state congruence requirement eliminates temporal exploitation by anchoring effectuation authority to the present state of all relevant protected parameters rather than to a historical validation event. Both properties are enforced not by software policy layers subject to bypass but by the mathematical properties of the cryptographic operations through which completion material is assembled — an assembly that succeeds only when graduated controls are fully satisfied and current-state congruence holds simultaneously at the Finality Sink.

VALIDATION DOES NOT MEAN GUARANTEE

As used herein, validation does not require or imply an absolute guarantee that an artificial-intelligence-generated output is objectively correct, complete, risk-free, lawful in all circumstances, or Technically desired.

Validation means that the Candidate Act has satisfied the defined machine-verifiable predicates required for controlled effectuation at the relevant time and within the relevant scope.

The technical contribution is not perfect artificial-intelligence truth. The technical contribution is protected separation between computation and consequence.

The system may therefore operate even where the underlying artificial-intelligence model is probabilistic, partially opaque, or not fully explainable. The artificial-intelligence model may remain probabilistic, but the output-to-consequence path is made machine-verifiable, scope-bound, receipt-bound, and sink-verified.

Purpose of This Definitions Section

The following definitions are provided to clarify the technical meaning of terms used throughout this disclosure. The definitions are not intended to limit the invention to a single implementation, hardware type, software stack, cryptographic algorithm, artificial-intelligence model, deployment environment, network protocol, vendor platform, or industry domain.

Unless expressly stated otherwise, the defined components may be implemented in software, firmware, hardware, trusted execution environments, secure enclaves, secure elements, hardware security modules, SmartNICs, DPUs, protected hypervisor partitions, secure microcontrollers, database commit controllers, storage controllers, payment modules, network processors, cloud-control systems, or combinations thereof.

A component defined herein may be implemented as a separate module, a submodule, a distributed service, a protected state object, a cryptographic artifact, a descriptor, a receipt, a token, a proof, a policy object, a hardware register state, a firmware operation, or any combination thereof.

1. Candidate Act

A **Candidate Act** means a computationally generated output, instruction, recommendation, command, tool call, data disclosure, payment instruction, network-configuration change, database mutation, memory write, model update, physical actuation command, settlement instruction, communication instruction, software-deployment instruction, or other effect-capable operation that has been generated but has not yet been permitted to become externally effective.

A Candidate Act may be generated by an artificial-intelligence model, autonomous agent, workflow engine, orchestration system, enterprise copilot, telecom controller, financial system, robotic controller, cloud-management system, database agent, model-management agent, or other computational system.

A Candidate Act is not treated as authority to act merely because it has been generated.

2. Non-Effective State

A **Non-Effective State** means a protected hold state in which a Candidate Act exists as data, computation, instruction, staged output, pending command, proposed operation, or act-equivalent representation but is not yet permitted to create an external, legal, financial, operational, network, physical, storage, memory, communication, settlement, database, model-state, or other consequence.

A Candidate Act in a non-effective state may be buffered, staged, simulated, reviewed, quarantined, rejected, modified, downgraded, redacted, delayed, sandboxed, routed for additional approval, or classified for escalated conditional finality, but it is not permitted to cross the finality boundary until required predicates are satisfied.

The non-effective state provides the technical separation between computation and consequence.

3. Effectuation

Effectuation means the transition by which a Candidate Act becomes externally effective.

Effectuation may include transmitting a message, sending data, exporting information, moving money, committing a database write, modifying stored state, updating memory, applying a model update, deploying code, changing a network configuration, enabling a radio transmission, actuating a physical device, executing a tool call, creating a legal or operational commitment, completing settlement, or otherwise producing an external consequence.

Effectuation is distinct from generating, drafting, staging, displaying, reviewing, simulating, or storing a Candidate Act in a non-effective state.

4. Protected Enforcement Domain / PED

A **Protected Enforcement Domain**, abbreviated **PED**, means a protected software, hardware, firmware, cryptographic, or hybrid enforcement domain configured to perform finality validation, state mutation, evidence generation, receipt generation, capability release, Execution Handle generation, or other protected finality operations.

A PED may include or cooperate with a trusted execution environment, secure enclave, secure element, hardware security module, SmartNIC, DPU, protected hypervisor partition, secure microcontroller, payment-security module, telecom security processor, database-commit controller, storage controller, network-security processor, protected service-mesh component, or other protected execution component.

The PED is responsible for determining whether a Candidate Act may proceed from non-effective state toward effectuation.

5. Cryptographically Isolated Enforcement Domain / CIED

A **Cryptographically Isolated Enforcement Domain**, abbreviated **CIED**, means a protected computing environment configured to perform cryptographic operations, validation operations, secure state mutation, receipt generation, capability release, or Execution Handle release while being isolated from ordinary application software.

A CIED may store or access protected keys, counters, nonces, revocation epochs, policy epochs, evidence hashes, capability-generation secrets, execution-material shares, sealed key fragments, and finality-state records.

A CIED may be used as the whole PED or as a protected subcomponent of the PED or Finality Sink.

6. Finality Gate

A **Finality Gate** means a protected technical enforcement boundary that prevents a Candidate Act from becoming externally effective unless required finality predicates are satisfied.

The Finality Gate does not merely log, monitor, recommend, classify, moderate, score, explain, or audit an artificial-intelligence output. It controls whether the Candidate Act may cross from computation into consequence.

In some embodiments, the Finality Gate is implemented inside, adjacent to, or in cooperation with the Protected Enforcement Domain.

7. Finality Sink

A **Finality Sink** means the technical interface, controller, device, module, endpoint, commit layer, switch, gateway, actuator, protected boundary, or execution component at which the Candidate Act becomes externally effective.

A Finality Sink may include an API execution boundary, tool-execution controller, payment switch, settlement engine, database commit layer, data-export gateway, message-transmission system, radio transmission chain, telecom user-plane function, network controller, cloud-management controller, robotic actuator, storage controller, model-update controller, memory-write controller, GPU memory-egress controller, SmartNIC egress controller, DPU enforcement module, or another effectuation interface.

The Finality Sink verifies a scoped non-bearer capability or Execution Handle before effectuation. If the capability or Execution Handle is absent, invalid, stale, revoked, out of scope, replayed, sink-mismatched, hardware-identity-mismatched, or not bound to the Candidate Act, the Finality Sink refuses effectuation.

In advanced non-completability embodiments, the Finality Sink may be technically non-completable unless valid completion material is supplied, reconstructed, unsealed, combined, or activated.

8. Algorithmic Logic Fingerprint / ALF

An **Algorithmic Logic Fingerprint**, abbreviated **ALF**, means a machine-verifiable representation of an approved computational logic state.

The ALF may represent one or more of an approved artificial-intelligence model version, model-weight state, workflow graph, system prompt policy, tool-use policy, retrieval policy, memory policy, safety policy, jurisdiction policy, model-router configuration, function-calling policy, guardrail configuration, deployment configuration, or other approved computational state.

Validation of the ALF establishes that the Candidate Act was generated by or under an approved computational logic configuration.

ALF validation does not, by itself, establish that the specific output is correct, safe, lawful, desired, Technically acceptable, or permitted to become externally effective.

9. Runtime Behavioral Descriptor / RBD

A **Runtime Behavioral Descriptor**, abbreviated **RBD**, means a machine-verifiable representation of runtime behavior observed during generation, transformation, routing, or preparation of a Candidate Act.

The RBD may identify tools accessed, retrieval sources used, memory regions accessed, prompt segments used, policy branches followed, model-router decisions, intermediate operation classes, data classes processed, external calls attempted, confidence signals, risk signals, event sequences, resource usage, or other runtime behavioral evidence.

The RBD may be compared with an ALF-bound approved behavioral envelope. A match indicates that runtime behavior remained within an expected envelope.

RBD matching is necessary in some embodiments but is not sufficient by itself for effectuation.

10. Hash-Linked Candidate Act Descriptor / HCAD

A **Hash-Linked Candidate Act Descriptor**, abbreviated **HCAD**, means a machine-verifiable descriptor representing a Candidate Act and its relevant context.

The HCAD may include or bind the Candidate Act content, Candidate Act hash, originating artificial-intelligence system, ALF identifier, RBD identifier, Output Provenance Capsule identifier, Result-Consequence Acceptance Envelope identifier, policy epoch, revocation epoch, Finality Sink identity, recipient identity, purpose, jurisdiction, risk class, timestamp, nonce, evidence references, and validation-state references.

The HCAD may be hash-linked to prior records, validation receipts, policy states, revocation states, source evidence, State-Proofs, or other finality records to provide tamper-evident continuity.

11. Output Provenance Capsule / OPC

An **Output Provenance Capsule**, abbreviated **OPC**, means a machine-verifiable evidence container associated with an artificial-intelligence-generated Candidate Act.

The OPC may identify sources used, retrieval records, tool outputs, database records, sensor records, timestamps, model-state identifiers, ALF identifiers, RBD identifiers, jurisdictional assumptions, confidence indicators, limitation flags, factual claim units, permitted-use constraints, recipient constraints, data-classification labels, policy-epoch identifiers, revocation-epoch identifiers, and evidence hashes.

The OPC does not guarantee truth. It provides structured evidence, assumptions, and provenance context used to determine whether the Candidate Act may become consequence.

A Candidate Act may be blocked, downgraded, redacted, routed for review, classified as escalated but still allowable, or denied when required provenance is missing, fabricated, stale, revoked, contradictory, unsupported, or outside permitted scope.

12. Factual Claim Unit / FCU

A **Factual Claim Unit**, abbreviated **FCU**, means a structured representation of a factual assertion contained in, relied upon by, or implied by an artificial-intelligence-generated Candidate Act.

Each FCU may include the asserted fact, source reference, source hash, freshness value, confidence value, jurisdictional assumption, contradiction status, limitation flag, data class, permitted-use scope, and intended consequence.

A Candidate Act may be blocked, downgraded, quarantined, redacted, routed for protected review, or classified for escalated conditional finality if a required FCU lacks support, conflicts with an approved source, is stale, has insufficient confidence, or exceeds the permitted scope of supporting evidence.

FCUs are particularly useful for hallucination-sensitive outputs because they allow factual assertions to be separated from fluent generated text and evaluated against evidence.

13. Result-Consequence Acceptance Envelope / RCAE

A **Result-Consequence Acceptance Envelope**, abbreviated **RCAE**, means a machine-verifiable boundary defining the permitted consequence scope of a Candidate Act.

The RCAE may define permitted purpose, permitted recipient, permitted data class, permitted jurisdiction, permitted tool, permitted Finality Sink, financial limit, operational limit, safety class, risk threshold, source requirement, freshness requirement, human-review requirement, time window, revocation epoch, policy epoch, permitted consequence type, and permissible finality mode.

The RCAE converts the idea of a desired or acceptable output into a technical consequence boundary.

A Candidate Act outside the RCAE remains non-effective unless a separate reduced-scope, escalated, sandboxed, delayed, reversible, or review-only finality mode is permitted.

14. Consequence Simulation

Consequence Simulation means a pre-effectuation analysis that determines, estimates, previews, simulates, dry-runs, classifies, or otherwise evaluates the consequence that would occur if a Candidate Act were effectuated by the applicable Finality Sink.

Consequence Simulation may determine whether the Candidate Act would cause data disclosure, money movement, settlement, network reconfiguration, physical actuation, database mutation, memory write, model update, legal commitment, external transmission, tool dispatch, storage write, communication, or another consequence.

Consequence Simulation may be implemented as a rules engine, static analysis routine, transaction preview, sandbox execution, dry-run tool execution, policy evaluator, data-flow analyzer, risk classifier, sink-specific preflight function, or protected simulation engine.

If the predicted consequence exceeds the RCAE or violates a required predicate, no full-effect capability is released.

15. Output Finality Predicate

An **Output Finality Predicate** means a machine-verifiable condition that must be satisfied before a Candidate Act may become externally effective.

Output Finality Predicates may include ALF validity, RBD match, HCAD integrity, OPC validity, FCU support, RCAE compliance, consequence simulation pass, recipient permission, purpose permission, jurisdiction permission, data-class permission, policy-epoch freshness, revocation-epoch freshness, Finality Sink compatibility, human approval where required, sink attestation, nonce freshness, and absence of revocation.

A Candidate Act may require different predicates depending on risk class, consequence type, data class, jurisdiction, sink type, recipient, and finality mode.

16. Graduated Finality

Graduated Finality means a finality decision model in which a Candidate Act is not limited to binary approval or denial.

Graduated finality may classify a Candidate Act as ordinary allowed, denied, quarantined, routed for protected review, reduced in scope, redacted, delayed, sandboxed, reversible, escrowed, canary-executed, review-only, or escalated but still allowable.

Graduated finality improves Technical feasibility because many Candidate Acts are elevated-risk but not prohibited. Such acts may proceed under stricter scope or controls rather than being automatically denied.

17. Escalated Conditional Finality

Escalated Conditional Finality means a finality mode in which a Candidate Act that does not qualify for ordinary release is still permitted to become externally effective under stricter safeguards.

A Candidate Act may be classified as escalated but still allowable when it presents elevated risk, near-boundary consequence sensitivity, incomplete but sufficient provenance, heightened financial value, sensitive data classification, jurisdictional sensitivity, Finality Sink degraded-but-trusted state, perturbation sensitivity, or a requirement for stronger approval.

In Escalated Conditional Finality, the Candidate Act remains in a non-effective state while additional controls are applied. Such controls may include protected human approval, multi-party approval, shortened expiration, reduced value, redaction, delayed effectuation, reversible execution, escrowed execution, canary execution, sandbox-first execution, fresh Finality Sink attestation, enhanced receipt generation, post-effectuation monitoring, stricter RCAE, stricter EASO, or additional State-Proof binding.

Escalated Conditional Finality allows elevated-risk acts to proceed in a Technically practical manner without sacrificing sink-bound finality control.

18. Reduced-Scope Release

A **Reduced-Scope Release** means a finality outcome in which the Candidate Act is not permitted in its originally requested form but is permitted in a narrower, safer, redacted, delayed, limited, reversible, sandboxed, or otherwise constrained form.

Examples of reduced-scope release may include redacting selected fields, reducing a financial amount, limiting recipients, limiting jurisdiction, limiting data class, changing external transmission to internal display, replacing full execution with sandbox execution, replacing immediate execution with delayed execution, or requiring reversible execution.

A Reduced-Scope Release may be implemented through a narrower scoped non-bearer capability, a stricter RCAE, a stricter EASO, or an escalated Execution Handle.

19. Scoped Non-Bearer Finality Capability

A **Scoped Non-Bearer Finality Capability** means a cryptographic, hardware-bound, sink-bound, act-bound, and scope-bound artifact that permits only a defined Candidate Act to be effectuated by a defined Finality Sink within a defined consequence scope.

The capability is scoped because it may be limited by one or more of Candidate Act identity, HCAD hash, Finality Sink identity, recipient, purpose, jurisdiction, data class, amount, operation type, consequence type, time window, nonce, policy epoch, revocation epoch, validation receipt, RCAE, and finality mode.

The capability is non-bearer because possession of the data object alone is insufficient to cause effectuation. The capability cannot be used merely by whoever bears, copies, forwards, steals, stores, or replays it.

The capability is accepted only by the intended Finality Sink when the sink-side verification context matches the capability scope.

20. Protected Human Approval Finality Token / PHAFT

A **Protected Human Approval Finality Token**, abbreviated **PHAFT**, means a protected, Candidate-Act-specific approval token representing human authorization.

A PHAFT may be bound to the Candidate Act, output content, predicted consequence, approving person or role, authenticated user presence, approval time, policy epoch, revocation epoch, Finality Sink identity, and applicable RCAE or EASO.

A generic click, stale approval, replayed approval, synthetic event, clickjacked approval, ordinary email confirmation, ordinary chat instruction, or unbound user-interface action is insufficient to generate a valid PHAFT where protected approval is required.

The PHAFT converts human approval into a machine-verifiable finality predicate.

21. Ledger-Anchored Validation Receipt / LAVR

A **Ledger-Anchored Validation Receipt**, abbreviated **LAVR**, means a tamper-evident record indicating that one or more validation predicates were checked for a Candidate Act.

The LAVR may be stored in a ledger, append-only log, secure audit store, protected receipt chain, local secure state, distributed evidence system, database record, hardware-protected log, or other tamper-evident record system.

The LAVR may identify Candidate Act hash, HCAD hash, ALF result, RBD result, OPC hash, FCU result, RCAE hash, consequence simulation result, finality decision, policy epoch, revocation epoch, nonce, counter value, Finality Sink identity, capability identifier, Execution Handle identifier, or State-Proof hash.

The LAVR is evidence of bounded validation. It is not an absolute guarantee of correctness.

22. Residual Risk Receipt / RRR

A **Residual Risk Receipt**, abbreviated **RRR**, means a record generated when a Candidate Act is permitted to become effective even though validation does not guarantee perfect correctness, safety, legality, or Technical desirability.

The RRR may identify which predicates passed, which assumptions were relied upon, which sources were used, which limitation flags applied, which residual risks remained, which capability or Execution Handle was released, which Finality Sink accepted the capability or Execution Handle, which escalation controls applied, and which policy or revocation epoch applied.

An RRR may be particularly useful when a Candidate Act is escalated but still allowed, reduced in scope, approved with limitation flags, or released under residual uncertainty.

23. Policy Epoch

A **Policy Epoch** means a version identifier associated with one or more active policies, rules, envelopes, predicates, jurisdictional requirements, risk thresholds, approval requirements, consequence classes, finality modes, or governance configurations.

A Candidate Act may be blocked if generated, validated, staged, approved, released, or effectuated under an outdated policy epoch.

Policy epochs allow the system to respond to changing policies, laws, safety rules, enterprise controls, risk thresholds, or consequence boundaries over time.

24. Revocation Epoch

A **Revocation Epoch** means a version identifier associated with revoked credentials, policies, capabilities, identities, approvals, model states, source authorities, sink trust states, tools, recipients, jurisdictions, or enforcement rules.

A Candidate Act may be blocked if its authority, source, approval, sink, capability, Execution Handle, validation evidence, or finality predicate is stale relative to the applicable revocation epoch.

Revocation epochs allow the system to prevent stale approvals or stale authorities from being used after a relevant condition has changed.

25. Execution Authorization Scope Object / EASO

An **Execution Authorization Scope Object**, abbreviated **EASO**, means a machine-verifiable execution-scope object defining the conditions under which a Candidate Act may be technically completed.

The EASO may include permitted purpose, permitted recipient, permitted jurisdiction, permitted data class, permitted tool or operation, value ceiling, operational ceiling, permitted consequence type, Finality Sink identity, attested sink hardware identity, time window, policy epoch, revocation epoch, required evidence objects, required human-review state, permitted reduced-scope mode, escalation controls, nonce requirements, anti-replay requirements, and permitted effectuation mode.

The EASO may derive from, incorporate, reference, or be cryptographically bound to the RCAE.

The RCAE defines what consequence is permitted. The EASO converts that permitted consequence boundary into an execution-enforceable cryptographic scope.

26. Execution Handle / EH

An **Execution Handle**, abbreviated **EH**, means a cryptographic, hardware-bound, act-bound, sink-bound, and scope-bound execution artifact generated by the Protected Enforcement Domain after finality validation.

The EH may include, encode, carry, wrap, seal, sign, MAC, encrypt, reconstruct, release, or enable access to execution material required by the Finality Sink.

The EH may be bound to Candidate Act hash, HCAD hash, EASO hash, RCAE hash, OPC hash, validation receipt identifier, ALF identifier, RBD identifier, Finality Sink identity, attested sink hardware identity, permitted recipient, permitted purpose, permitted jurisdiction, permitted consequence, policy epoch, revocation epoch, expiration time, counter value, and fresh nonce.

Possession of the EH by ordinary software is insufficient to force effectuation. The EH is usable only by the named Finality Sink, operating under the matching attested hardware or protected execution identity, and only for the matching Candidate Act within the permitted scope.

27. State-Entangled Finality Proof / State-Proof

A **State-Entangled Finality Proof**, also referred to as a **State-Proof**, means a cryptographic proof or protected binding that links the Candidate Act, permitted consequence boundary, simulation result, current state, and Finality Sink verification context.

The State-Proof may bind one or more of Candidate Act hash, HCAD hash, OPC hash, FCU result, RCAE hash, EASO hash, consequence simulation result hash, predicted consequence classification, policy epoch, revocation epoch, Finality Sink identity, Finality Sink attestation, monotonic counter value, nonce, protected transaction identifier, validation receipt identifier, and Execution Handle identifier.

The State-Proof prevents substitution of a different Candidate Act, different simulation result, different RCAE, different EASO, different sink, different epoch, or different transaction state after validation.

In some embodiments, the Finality Sink refuses effectuation unless the State-Proof verifies under current state conditions.

28. Finality Sink Attestation

Finality Sink Attestation means machine-verifiable evidence representing the identity, security state, hardware state, firmware state, protected execution state, or operational condition of a Finality Sink.

Finality Sink Attestation may include secure boot evidence, trusted execution measurement, hardware identity, secure-element identity, device certificate, SmartNIC identity, DPU identity, hardware security module identity, firmware measurement, policy-epoch state, revocation-epoch state, operating mode, debug state, maintenance state, emergency state, degraded state, or other sink-security information.

In some embodiments, the capability, EASO, Execution Handle, or State-Proof is bound to the Finality Sink attestation state. If the sink state changes or fails attestation, effectuation is refused.

29. Hardware-Rooted Non-Completability

Hardware-Rooted Non-Completability means a condition in which a Finality Sink is technically unable to complete a Candidate Act unless protected finality validation enables completion.

In some embodiments, the Finality Sink holds only incomplete effectuation material, such as a sink-side key share, partial signature share, sealed token fragment, incomplete command authenticator, locked egress primitive, locked commit primitive, disabled actuator primitive, protected register state, or hardware-held final-step state.

The missing complementary material may be supplied, reconstructed, unsealed, combined, or activated through a valid Execution Handle.

Accordingly, the Finality Sink is not merely told not to act. It lacks the technical ability to complete the Candidate Act unless the protected finality process succeeds.

30. Atomic Receipt-With-Release

Atomic Receipt-With-Release means a protected transaction in which validation, receipt generation, protected state mutation, policy-epoch verification, revocation-epoch verification, nonce generation, monotonic counter advancement, sink attestation, capability release, or Execution Handle release are bound together.

In some embodiments, no Execution Handle is released without a corresponding validation receipt, and no effectuating validation receipt is committed without the corresponding Execution Handle release.

If any required validation, state mutation, receipt write, execution-material reconstruction, capability minting, or Execution Handle minting step fails, the protected transaction is aborted.

Atomic Receipt-With-Release reduces time-of-check-to-time-of-use risk by preventing validation under one state and effectuation under a later inconsistent state.

31. Cold Validation Path

A **Cold Validation Path** means a validation path used for computationally heavier or time-tolerant operations that need not occur at the moment of sink-side effectuation.

The cold validation path may perform model approval, ALF generation, policy compilation, source registration, sink registration, tool registration, RCAE template generation, EASO template generation, revocation-list synchronization, high-cost consequence simulation, adversarial perturbation testing, protected human approval for planned actions, or preparation of sink-bound verification material.

The cold validation path may output reusable, short-lived, versioned, epoch-bound, signed, MAC-bound, encrypted, Merkle-bound, or hash-linked artifacts.

32. Nearline Preparation Path

A **Nearline Preparation Path** means a preparation path that operates after a specific artificial-intelligence output is generated but before final effectuation.

The nearline preparation path may perform Candidate Act formation, HCAD generation, ALF validation, RBD collection, OPC construction, FCU verification, RCAE selection, consequence simulation, escalation decisioning, State-Proof generation, validation receipt preparation, and Execution Handle preparation.

The nearline path handles Candidate Act-specific operations that may require more context than the cold validation path but should not be performed at the Finality Sink if avoidable.

33. Hot Finality Path

A **Hot Finality Path** means the low-latency final verification path at or near the Finality Sink.

The hot finality path may perform compact operations, including Candidate Act hash verification, nonce verification, expiration verification, policy-epoch comparison, revocation-epoch comparison, sink-identity verification, sink-attestation verification, validation-receipt reference verification, State-Proof verification, scoped capability verification, Execution Handle verification, and reconstruction or unsealing of final execution material.

The hot finality path need not execute a full artificial-intelligence model, perform full factual reasoning, conduct full consequence simulation, or perform full adversarial analysis in real time.

34. Residual Uncertainty

Residual Uncertainty means remaining uncertainty after validation predicates have been evaluated.

Residual uncertainty may arise because validation does not guarantee absolute truth, perfect safety, universal legality, future correctness, or Technical desirability.

The system may handle residual uncertainty by denying effectuation, reducing scope, requiring protected approval, generating an RRR, applying escalated conditional finality, delaying effectuation, requiring post-effectuation monitoring, or updating policy or revocation epochs.

35. Act-Equivalent Representation

An **Act-Equivalent Representation** means a transformed, encoded, delegated, split, buffered, serialized, compressed, encrypted, translated, summarized, paraphrased, scheduled, routed, tool-dispatched, or otherwise modified representation of a Candidate Act that is capable of producing the same or substantially similar external consequence.

Finality validation may apply to act-equivalent representations to prevent bypass by format change, routing change, delegation, buffering, or fragmentation.

36. Protected Review

Protected Review means a review process in which a Candidate Act remains non-effective while additional protected predicates are evaluated.

Protected Review may involve human review, multi-party review, source verification, legal review, risk review, additional FCU verification, stronger RCAE application, stricter EASO formation, sink attestation, or other review operations.

A protected review outcome may allow ordinary release, escalated release, reduced-scope release, redacted release, delayed release, sandbox-only release, quarantine, or denial.

37. Canary Execution

Canary Execution means a limited, controlled, monitored, or partial effectuation mode used to test or observe a Candidate Act before permitting broader effectuation.

Canary Execution may be limited by recipient, dataset, amount, duration, jurisdiction, tool scope, operational scope, or sink scope.

A Candidate Act may be classified as escalated but still allowable and then released only as a canary execution.

38. Reversible Execution

Reversible Execution means an effectuation mode in which the system permits a Candidate Act to proceed only if the resulting consequence can be reversed, rolled back, escrowed, cancelled, compensated, or otherwise remediated within a defined scope.

Reversible Execution may be used as an escalated conditional finality control.

39. Fail-Closed Mode

Fail-Closed Mode means a state in which effectuation is denied when required validation, proof, capability, Execution Handle, epoch, nonce, sink attestation, or verification evidence is absent, stale, invalid, or unavailable.

In Fail-Closed Mode, the Candidate Act remains non-effective unless the protected finality process succeeds.

40. Fail-Limited Mode

Fail-Limited Mode means a state in which a Candidate Act is not fully denied but is permitted only in a reduced, safer, redacted, delayed, sandboxed, local-only, reversible, or review-only form.

Fail-Limited Mode may be used where ordinary full-effectuation is unsafe but limited effectuation remains permitted.

Ledger-Anchored Validation Receipt / LAVR

A **Ledger-Anchored Validation Receipt**, abbreviated **LAVR**, means a tamper-evident validation record generated before, during, or atomically with release of a scoped finality authority for a Candidate Act.

The LAVR records that one or more required finality predicates were evaluated for the Candidate Act and identifies the protected state under which the Candidate Act was permitted, denied, reduced in scope, escalated, delayed, sandboxed, made reversible, or otherwise classified.

In some embodiments, the LAVR may include one or more of: Candidate Act hash, HCAD hash, ALF validation result, RBD matching result, OPC hash, FCU verification result, RCAE hash, EASO hash where used, consequence simulation result, finality decision, escalation decision, PHAFT reference where required, scoped capability identifier, Execution Handle identifier where used, State-Proof hash, Finality Sink identity, sink attestation hash, policy epoch, revocation epoch, nonce, counter value, protected transaction identifier, timestamp, and receipt-chain reference.

The LAVR may be stored in, committed to, or anchored by a ledger, append-only log, secure audit store, tamper-evident database, hardware-protected log, Merkle tree, receipt chain, distributed evidence system, secure enclave state, HSM-backed counter, or other protected receipt state.

In some embodiments, the LAVR is not merely a post-effectuation audit record. The LAVR may be part of the protected finality transaction, such that no scoped capability or Execution Handle is released unless the corresponding LAVR is generated, committed, sealed, or hash-linked in protected state. The Finality Sink may verify the LAVR reference, LAVR hash, or LAVR-bound State-Proof before effectuation.

The LAVR provides evidence of bounded validation. It does not guarantee absolute truth, perfect safety, legality in all circumstances, or commercial desirability. Instead, it proves that the Candidate Act satisfied the defined machine-verifiable finality predicates at the relevant time, under the relevant policy epoch, revocation epoch, scope, and Finality Sink context.

Short definition:

LAVR means a tamper-evident receipt proving that a Candidate Act was evaluated under protected finality predicates and that any capability or Execution Handle released for the act was bound to the validated act, scope, state, epoch, and Finality Sink.

PRIORITY-FILING TERMINOLOGY CLARIFICATION REGARDING PED AND CIED

In the priority filings and related disclosure materials, the terms **Protected Enforcement Domain**, abbreviated **PED**, and **Cryptographically Isolated Enforcement Domain**, abbreviated **CIED**, may be used in an overlapping manner. This usage is intentional and should be understood as referring to protected enforcement domains that may perform the same finality-enforcement function while emphasizing different implementation aspects.

The term **PED** emphasizes the protected enforcement function of the domain. A PED performs or coordinates finality validation, Candidate Act processing, protected state mutation, evidence generation, receipt generation, scoped capability release, Execution Handle generation, and Finality Sink coordination.

The term **CIED** emphasizes the cryptographic isolation property of the same or related protected enforcement domain. A CIED protects cryptographic keys, nonces, counters, sealed state, epoch state, capability-generation material, Execution Handle material, receipt commitments, locked execution primitives, and other protected finality state from ordinary software, agent software, application logic, operating-system processes, or untrusted execution paths.

Accordingly, in some embodiments, a **PED and a CIED are the same component**. For example, a protected enforcement domain implemented inside a secure enclave, trusted execution environment, hardware security module, secure element, SmartNIC, DPU, protected microcontroller, firmware-protected controller, or other cryptographically isolated environment may be referred to as either a PED or a CIED. In such embodiments, the same protected domain both performs finality-enforcement operations and maintains cryptographic isolation of the protected state used for those operations.

In other embodiments, a PED may include, invoke, or cooperate with one or more CIEDs. For example, the PED may coordinate Candidate Act formation, HCAD generation, ALF validation, RBD matching, OPC validation, FCU verification, RCAE comparison, consequence simulation, escalation classification, protected approval processing, and Finality Sink coordination, while an associated CIED protects keys, nonces, counters, sealed state, receipt commitments, capability-generation material, Execution Handle material, or execution-material shares.

Unless expressly stated otherwise, references in the priority filings to a PED should be understood to include a cryptographically isolated implementation of the PED, including a CIED. Likewise, references to a CIED should be understood to include a PED where the protected enforcement domain is cryptographically isolated and performs or participates in the finality-enforcement sequence.

The disclosed architecture does not require PED and CIED to be separate physical components. The distinction is implementation-dependent. The protected enforcement function and the cryptographic isolation function may be implemented in a single protected component, in separate cooperating components, or in a distributed set of protected components, provided that the Candidate Act remains non-effective until the required finality predicates are satisfied and the applicable scoped authority is released and verified by the Finality Sink.

For avoidance of doubt, the priority filings should be read such that:

PED identifies the protected finality-enforcement role.
CIED identifies the cryptographic isolation property of that protected enforcement role.

A CIED may be the cryptographically isolated implementation of a PED.
A PED may include, invoke, or cooperate with one or more CIEDs.
The terms may overlap unless a specific embodiment expressly distinguishes them.

This clarification preserves the disclosure scope of the priority filings and prevents an unintended narrowing in which PED and CIED are treated as necessarily separate components. The invention covers both unified PED/CIED implementations and distributed PED-plus-CIED implementations.

BASE WORKFLOW AND TECHNICAL FINALITY FLOW

Purpose

This describes the operational workflow by which an artificial-intelligence-generated output is converted into a non-effective Candidate Act, evaluated through protected finality predicates, and either denied, reviewed, reduced in scope, escalated, or permitted to become externally effective through Finality Sink verification.

This also describes a Technically feasible graduated-finality model in which elevated-risk Candidate Acts are not automatically denied. Instead, such acts may be classified as **escalated but still allowable** and permitted to proceed only under stricter controls.

The workflow described in this section may be implemented with or without the advanced non-completability Execution Handle embodiments described later. In the base implementation, the Protected Enforcement Domain releases a scoped non-bearer finality capability after validation. In advanced non-completability implementations, the capability may be implemented as or converted into an Execution Handle.

1. Base Output-to-Consequence Finality Workflow

Workflow Overview

In some embodiments, the base workflow follows this sequence:

Artificial-Intelligence Output → Candidate Act Formation → Non-Effective Hold → HCAD Generation → ALF Validation → RBD Matching → OPC Generation and Validation → FCU Extraction and Verification → RCAE Generation or Retrieval → Consequence Simulation → Output Finality Predicate Evaluation → Graduated Finality Decision → Protected Human Approval Where Required → Scoped Capability Release → Finality Sink Verification → Receipt and Residual Risk Handling.

This workflow separates computation from consequence. An artificial-intelligence-generated output does not become externally effective merely because it was generated, accepted by software, approved through a user interface, or routed to a tool. Effectuation occurs only after protected finality validation and Finality Sink verification.

Step 1 — Artificial-Intelligence Output Generation

An artificial-intelligence system generates an output, which may be a message, recommendation, factual statement, tool call, command, data-export request, payment instruction, network actuation command, database mutation, model update, memory write, software deployment instruction, settlement instruction, physical actuation command, or other effect-capable result. The generated output is not immediately treated as an authorized act. The output may be fluent, plausible, confident, or generated by an approved artificial-intelligence system, but those characteristics do not establish authority for external consequence.

Step 2 — Candidate Act Formation

The generated output is converted into a Candidate Act. The Candidate Act may include output content, output hash, intended recipient, intended tool, intended Finality Sink, purpose, jurisdiction, data class, risk class, requested consequence, timestamp, artificial-intelligence-system identifier, model identifier, workflow identifier, Algorithmic Logic Fingerprint identifier, and other relevant context. The Candidate Act may also include or reference a requested effectuation mode, including ordinary release, reduced-scope release, sandbox release, delayed release, escalated release, or review-only staging. The Candidate Act is not authority to act; it is a proposed effect-capable operation awaiting protected finality validation.

Step 3 — Non-Effective Hold

The Candidate Act is placed into a non-effective state. In that state, the Candidate Act may be buffered, staged, stored, displayed for review, simulated, redacted, delayed, sandboxed, quarantined, modified, downgraded, classified for escalation, or routed for additional approval, but it is not permitted to create external consequence. The non-effective hold may be enforced by a Protected Enforcement Domain, Finality Gate, tool-dispatch controller, database commit controller, communication gateway, storage controller, payment module, network controller, or another protected enforcement point.

Step 4 — HCAD Generation

The Protected Enforcement Domain generates a Hash-Linked Candidate Act Descriptor. The HCAD may bind the Candidate Act to one or more of: Candidate Act content, Candidate Act hash, originating artificial-intelligence system, model identifier, workflow identifier, ALF identifier, RBD identifier, OPC identifier, RCAE identifier, policy epoch, revocation epoch, Finality Sink identity, recipient, purpose, jurisdiction, risk class, timestamp, nonce, and evidence references. The HCAD provides a machine-verifiable descriptor of the Candidate Act and its validation context and may be hash-linked to evidence objects, policy states, validation receipts, or prior finality records.

Step 5 — ALF Validation

The Protected Enforcement Domain validates the Algorithmic Logic Fingerprint associated with the Candidate Act. The ALF may represent an approved model state, model-weight state, workflow graph, system prompt policy, retrieval policy, memory policy, tool-use policy, safety policy, jurisdiction policy, model-router configuration, function-calling policy, or deployment configuration. If the ALF is unknown, stale, revoked, mismatched, outside the permitted policy epoch, outside the permitted revocation epoch, or otherwise invalid, the Candidate Act may be denied, quarantined, or routed for protected review. If ALF validation passes, the workflow continues. ALF validation confirms approved computational logic state, but it does not confirm that the specific output is correct, safe, lawful, Technically desired, or permitted to become consequence.

Step 6 — RBD Matching

The system collects or receives a Runtime Behavioral Descriptor associated with generation or preparation of the Candidate Act. The RBD may identify tools accessed, retrieval sources used, memory regions accessed, prompt segments used, policy branches followed, model-router decisions, intermediate operation classes, data classes processed, external calls attempted, confidence signals, risk signals, event sequences, resource usage, or other runtime behavioral evidence. The Protected Enforcement Domain compares the RBD against an ALF-bound approved behavioral envelope. If the RBD shows unauthorized tool use, unauthorized retrieval, unauthorized memory access, unexpected policy branching, excessive external calls, abnormal risk state, unauthorized model routing, unauthorized data-class processing, or other out-of-envelope behavior, the Candidate Act may be denied, quarantined, downgraded, or routed for investigation. If RBD matching passes, the workflow continues. RBD matching confirms runtime behavior within an expected envelope but does not by itself authorize effectuation.

Step 7 — OPC Generation and Validation

The system generates or receives an Output Provenance Capsule associated with the Candidate Act. The OPC may include source records, retrieval logs, tool outputs, database records, sensor records, evidence hashes, confidence indicators, limitation flags, jurisdictional assumptions, data classifications, permitted-use constraints, source freshness indicators, policy-epoch identifiers, revocation-epoch identifiers, and factual claim units. The Protected Enforcement Domain validates the OPC. If required provenance is missing, stale, revoked, fabricated, contradictory, unsupported, outside permitted use, outside jurisdictional scope, or not bound to the Candidate Act, the Candidate Act remains non-effective. Depending on risk and policy, the Candidate Act may be denied, quarantined, routed for protected review, reduced in scope, redacted, or classified as escalated but still allowable.

Step 8 — FCU Extraction and Verification

For hallucination-sensitive, high-risk, legal, financial, medical, operational, security, public-safety, regulatory, or compliance-sensitive outputs, the system may extract factual assertions into Factual Claim Units. Each FCU may identify an asserted fact, source reference, source hash, freshness value, confidence value, jurisdictional assumption, contradiction status, limitation flag, permitted-use scope, and intended consequence. The Protected Enforcement Domain verifies required FCUs against supporting evidence. If a required FCU lacks support, conflicts with an approved source, is stale beyond a permitted freshness window, has insufficient confidence, or exceeds the permitted use of supporting evidence, the Candidate Act may be blocked, downgraded, redacted, routed for protected review, or classified for escalated conditional finality. A Candidate Act may still be permitted in a reduced or escalated form where unsupported factual material is removed, delayed, limited to internal display, routed for review, or released with a residual-risk receipt.

Step 9 — RCAE Generation or Retrieval

The system generates, retrieves, or derives a Result-Consequence Acceptance Envelope defining what external consequence is permitted for the Candidate Act. The RCAE may specify permitted purpose, permitted recipient, permitted data class, permitted jurisdiction, permitted tool, permitted Finality Sink, financial limit, operational limit, safety class, risk threshold, freshness requirement, source requirement, human-review requirement, time window, revocation epoch, policy epoch, and permitted consequence type. For example, the RCAE may specify that an output may be displayed internally but not transmitted externally, a payment may not exceed a defined value, a data export may occur only to a permitted recipient, a database update may be staged but not committed, a network command may be simulated but not applied, a legal-risk output requires protected human approval, a model update may be tested only in sandbox, or a high-risk act may proceed only under

escalated conditional finality. The RCAE converts a desired output into a machine-verifiable consequence boundary.

Step 10 — Consequence Simulation

Before capability release, the system simulates, predicts, previews, dry-runs, preflights, or otherwise evaluates the effect of the Candidate Act if it were effectuated by the applicable Finality Sink. The consequence simulation may determine who receives data, what data leaves the system, what money moves, which database or storage state changes, which network state changes, which physical actuator moves, which model or memory state changes, which settlement occurs, which tool action occurs, which legal or operational commitment is created, which jurisdiction is implicated, and whether the consequence is reversible, delayed, sandboxed, partial, or externally final. If the predicted consequence exceeds the RCAE, violates policy, exceeds risk, conflicts with jurisdiction, lacks authority, lacks support, targets an incompatible sink, or is otherwise outside permitted scope, the Candidate Act remains non-effective. The system may then deny the Candidate Act, route it for protected review, reduce its scope, delay it, sandbox it, redact it, classify it as escalated but still allowable, or generate a rejection record.

Step 11 — Output Finality Predicate Evaluation

The Protected Enforcement Domain evaluates the required output-level finality predicates, including one or more of ALF validity, RBD match, HCAD integrity, OPC validity, FCU support, RCAE compliance, consequence simulation pass, recipient permission, purpose permission, jurisdiction permission, data-class permission, tool permission, Finality Sink compatibility, policy-epoch freshness, revocation-epoch freshness, source authority, human approval where required, sink attestation where required, nonce freshness, and absence of revocation. If all mandatory predicates pass, the workflow proceeds to finality decisioning and capability release. If one or more predicates fail, the system may reject, quarantine, downgrade, redact, delay, sandbox, reduce scope, escalate, or route the Candidate Act for protected review.

2. Graduated Finality Decisioning

Purpose of Graduated Finality

In some embodiments, the system does not treat every Candidate Act as simply allowed or denied. Instead, the Protected Enforcement Domain classifies the Candidate Act into a graduated finality outcome based on risk, consequence, confidence, provenance, factual support, jurisdiction, sink state, reversibility, Technical necessity, and available safeguards. Graduated finality improves practical deployability because many Candidate Acts are neither completely safe for ordinary release nor completely prohibited.

Graduated Finality Outcomes

The finality outcome may include ordinary release, escalated release, reduced-scope release, redacted release, delayed release, sandbox-only release, canary release, reversible release, escrowed release, internal-display-only release, protected-review staging, quarantine, rejection, or denial. The finality outcome may determine what capability, if any, is released and what conditions the Finality Sink must verify.

Ordinary Release

Ordinary release may occur when the Candidate Act satisfies all mandatory finality predicates and does not require additional safeguards. In ordinary release, the Protected Enforcement Domain may release a scoped non-bearer finality capability allowing the Candidate Act to be effectuated by the intended Finality Sink within the permitted scope.

Reduced-Scope Release

Reduced-scope release may occur when the Candidate Act is not permitted in its requested form but may be permitted in a narrower or safer form. Reduced-scope release may include redacting selected content, reducing a financial amount, limiting recipients, limiting jurisdiction, limiting data class, converting external transmission to internal display, replacing full execution with sandbox execution, replacing immediate execution with delayed execution, or requiring reversible execution. The released capability may be bound only to the reduced scope.

Review-Required Outcome

A review-required outcome may occur when the Candidate Act cannot be safely released based on automated predicates alone but does not require immediate denial. The Candidate Act remains non-effective while routed for protected review. Protected review may include human approval, multi-party approval, source verification, legal review, security review, jurisdictional review, operational review, additional FCU verification, stricter RCAE application, stricter EASO formation, or fresh Finality Sink attestation.

Quarantine or Denial

Quarantine or denial may occur when required predicates fail and no reduced-scope, escalated, delayed, reversible, or review-only path is permitted. A denied Candidate Act remains non-effective and may generate a rejection record, rejection receipt, audit record, revocation signal, policy-update signal, or risk-training signal.

3. Escalated but Still Allowable Finality

Purpose of Escalated Conditional Finality

In some embodiments, a Candidate Act that does not qualify for ordinary release is not automatically denied. Instead, the Protected Enforcement Domain may classify the Candidate Act as escalated but still allowable when the Candidate Act presents elevated risk but remains within a permitted consequence boundary or can be brought within a permitted boundary through additional safeguards. Escalated conditional finality is Technically important because high-value enterprise, financial, infrastructure, communications, legal, operational, and regulated actions may be necessary even when they require stronger control. The system therefore avoids both unsafe ordinary release and unnecessary denial of Technically necessary actions.

Escalated Conditional Finality Mode

In Escalated Conditional Finality Mode, the Candidate Act remains in a non-effective state while additional predicates, limitations, approvals, or monitoring requirements are applied. The Candidate Act may become externally effective only if the escalated controls are satisfied and the Finality Sink verifies the applicable escalated capability or Execution Handle. The escalated mode may be applied before capability release, after consequence simulation, after perturbation testing, after FCU

verification, after sink attestation, after human approval determination, or after output finality predicate evaluation.

Conditions That May Trigger Escalation

A Candidate Act may be classified as escalated but still allowable when one or more triggering conditions exist, including a predicted consequence close to but still within the RCAE, elevated but acceptable risk, OPC limitation flags, FCUs with reduced but acceptable confidence, sources close to a freshness boundary, sensitive data class with permitted recipient, high-value transaction within an authorized limit, jurisdiction requiring stricter review, degraded but still trusted Finality Sink, reversible or partially reversible consequence, Technical necessity with operational sensitivity, requirement for a PHAFT, consequence simulation passing with elevated risk, perturbation testing passing with consequence sensitivity, stricter policy epoch requirement, shortened validity-window requirement, or need for additional monitoring, logging, escrow, delay, or revocation conditions.

Escalated Controls

When a Candidate Act is classified as escalated but still allowable, the Protected Enforcement Domain may require additional controls before release. Such controls may include a PHAFT, multi-party approval, role-constrained approval, quorum approval, shorter capability expiration, reduced financial value, reduced operational scope, redaction of selected data fields, recipient limitation, jurisdiction limitation, delayed effectuation, reversible execution, escrowed execution, canary execution, sandbox-first execution, additional source verification, higher FCU confidence threshold, fresh Finality Sink attestation, stricter RCAE, stricter EASO, enhanced LAVR fields, Residual Risk Receipt generation, post-effectuation monitoring, automatic capability revocation upon downstream change, or forced revalidation before final completion.

Escalated Capability or Execution Handle

If the escalated predicates are satisfied, the Protected Enforcement Domain may release an escalated scoped non-bearer capability or escalated Execution Handle. The escalated capability or Execution Handle may be more restrictive than an ordinary capability and may be bound to a shorter time window, stricter policy epoch, stricter revocation epoch, specific approving role, specific PHAFT, reduced financial amount, redacted data field set, narrower recipient class, narrower jurisdictional scope, specific Finality Sink attestation, reversible execution mode, delayed execution mode, canary execution mode, monitoring requirement, residual-risk receipt, or automatic revocation condition. The Finality Sink may reject the escalated capability or Execution Handle unless all escalated conditions remain satisfied at the time of completion.

Escalated Outcome Examples

In a customer-facing factual-output example, an artificial-intelligence system may generate a customer-facing statement containing factual claims where the FCUs are mostly supported but one source has a limitation flag. The Candidate Act may be classified as escalated but still allowable, with redaction of the unsupported portion, PHAFT generation, and Residual Risk Receipt generation before transmission. In an elevated-value financial-act example, an artificial-intelligence agent may generate a payment instruction above an ordinary auto-approval threshold but below an escalated maximum limit, requiring multi-party approval, shortened capability expiration, fresh recipient verification, and Finality Sink verification before payment effectuation. In a degraded-but-trusted sink example, ordinary release may be denied but escalated release may be permitted with fresh sink attestation, stricter EASO, shorter expiration, and post-effectuation monitoring. In an infrastructure-change example, consequence simulation may show elevated but not prohibited

operational risk, allowing canary execution, rollback capability, protected approval, and monitoring before full effectuation.

4. Protected Human Approval Within the Workflow

When Protected Human Approval Is Required

If the risk class, RCAE, EASO, escalation mode, jurisdictional rule, financial threshold, data class, operational sensitivity, or consequence type requires human approval, the system does not rely on ordinary approval alone. Ordinary clicks, ordinary user-interface approvals, email confirmations, chat confirmations, typed instructions, generic workflow approvals, stale approvals, replayed approvals, synthetic approvals, or clickjacked approvals are insufficient where protected approval is required.

Protected Human Approval Finality Token

The system obtains a Protected Human Approval Finality Token bound to the Candidate Act, Candidate Act hash, output content, predicted consequence, approving person or role, authenticated user presence, approval time, policy epoch, revocation epoch, RCAE or EASO, and Finality Sink identity. If a valid PHAFT is not obtained where required, the Candidate Act remains non-effective.

5. Scoped Capability Release

Capability Release Condition

If the required predicates are satisfied, the Protected Enforcement Domain may release a scoped non-bearer finality capability. The capability may be ordinary, reduced-scope, escalated, delayed, sandboxed, reversible, canary-limited, or otherwise constrained according to the finality outcome. The capability is not a generic permission token; it is bound to the Candidate Act and the permitted consequence scope.

Capability Binding

The scoped capability may be bound to one or more of Candidate Act hash, HCAD hash, OPC hash, FCU result, RCAE hash, predicted consequence, finality decision, Finality Sink identity, recipient, purpose, jurisdiction, data class, operation type, time window, nonce, policy epoch, revocation epoch, validation receipt, PHAFT, escalation controls, and permitted consequence type. A capability that is copied, replayed, modified, re-pointed, used at the wrong sink, used for the wrong Candidate Act, used after expiration, used under a stale epoch, or used outside scope is rejected.

6. Finality Sink Verification

Sink-Side Verification

The Finality Sink verifies the scoped capability before effectuation. The Finality Sink may check signature or message authentication code, sink identity, Candidate Act hash, HCAD hash, scope, nonce, expiration, policy epoch, revocation epoch, recipient, purpose, jurisdiction, data class, permitted consequence, validation receipt reference, protected human approval reference, escalation controls, and sink attestation where required. If verification passes, the Finality Sink effectuates the

Candidate Act within the permitted scope. If verification fails, the Candidate Act remains non-effective or is rejected.

Effectuation Within Permitted Scope

When the Finality Sink verifies the capability, it may effectuate only the permitted act. The sink may not expand the recipient, increase the amount, change the purpose, alter the jurisdiction, change the data class, bypass redaction, remove delay, skip monitoring, ignore escalation controls, or convert a limited release into full release. Sink-side verification therefore enforces the protected finality decision.

7. Optional Advanced Finality Workflow Using Execution Handle

Purpose of the Advanced Workflow

In some embodiments, the base workflow is strengthened by replacing or supplementing the scoped non-bearer finality capability with an Execution Handle generated within the bounds of an Execution Authorization Scope Object. This advanced workflow does not replace the base output-to-consequence workflow; it applies after the base validation path determines that the Candidate Act may proceed toward effectuation. The advanced workflow may be used where stronger assurance is required, including high-risk artificial-intelligence acts, financial operations, sensitive data exports, infrastructure changes, database commits, tool executions, network-control actions, model updates, regulated communications, safety-critical operations, or other consequence-sensitive acts. In the advanced workflow, the Finality Sink is not merely asked to verify permission; it may be configured to be technically non-completable unless a valid Execution Handle supplies, reconstructs, unseals, combines, or activates missing execution material required for completion.

Advanced Workflow Overview

In some embodiments, the advanced workflow follows this sequence:

Base Finality Validation → EASO Formation → Execution Handle Generation → Atomic Receipt-With-Release → Finality Sink Verification → Reconstruction or Unsealing of Missing Execution Material → Effectuation Only if Completion Succeeds.

The advanced workflow may also be expressed as:

Candidate Act → RCAE → EASO → Atomic Receipt-With-Release → Execution Handle → Hardware-Bound Sink Verification → Reconstruction / Unsealing / Combining / Activation → Effectuation or Refusal.

Advanced Step 1 — Completion of Base Finality Validation

The system first performs the base output-to-consequence workflow, including one or more of Candidate Act formation, non-effective hold, HCAD generation, ALF validation, RBD matching, OPC validation, FCU verification, RCAE generation, consequence simulation, output finality predicate evaluation, graduated finality decisioning, and protected human approval where required. If the base workflow denies, quarantines, redacts, delays, or routes the Candidate Act for review, the advanced workflow does not release an Execution Handle. If the base workflow determines that the Candidate Act may proceed, the system may continue into the advanced workflow.

Advanced Step 2 — Execution Authorization Scope Object Formation

The Protected Enforcement Domain derives or retrieves an EASO. The EASO may be derived from the Candidate Act, HCAD, RCAE, predicted consequence, finality decision, escalation controls, Finality Sink identity, attested Finality Sink hardware identity, policy epoch, revocation epoch, permitted recipient, permitted purpose, permitted jurisdiction, permitted data class, permitted operation type, and permitted consequence type. The RCAE defines what consequence is permitted, and the EASO converts that permitted consequence boundary into an execution-enforceable cryptographic scope. If the Candidate Act, predicted consequence, or requested effectuation mode is outside the EASO, no Execution Handle is released.

Advanced Step 3 — Current-State Verification

Before Execution Handle release, the Protected Enforcement Domain verifies current protected state, including one or more of current policy epoch, current revocation epoch, Candidate Act hash, HCAD hash, RCAE hash, EASO hash, Finality Sink identity, Finality Sink attestation, recipient status, source authority status, model approval status, human approval status where required, nonce freshness, and absence of revocation. This prevents historical approval from being treated as current authority.

Advanced Step 4 — Atomic Receipt-With-Execution-Handle Release

In some embodiments, validation receipt generation, nonce generation, monotonic counter advancement, current-state verification, sink attestation, execution-material preparation, and Execution Handle release occur within a single protected atomic transaction. If any required validation, state mutation, receipt write, execution-material reconstruction, or Execution Handle minting step fails, the protected transaction is aborted. In some embodiments, no Execution Handle is released without a corresponding validation receipt, and no effectuating validation receipt is committed without the corresponding Execution Handle release. The validation receipt and Execution Handle may be bound to the same Candidate Act hash, HCAD hash, RCAE hash, EASO hash, Finality Sink identity, sink hardware identity, policy epoch, revocation epoch, nonce, monotonic counter value, and protected transaction identifier.

Advanced Step 5 — Finality Sink Verification of Execution Handle

The Finality Sink verifies the Execution Handle before effectuation. The Finality Sink may check Execution Handle authenticity, Candidate Act hash, HCAD hash, EASO hash, RCAE hash, Finality Sink identity, attested sink hardware identity, policy epoch, revocation epoch, expiration time, nonce freshness, permitted recipient, permitted purpose, permitted jurisdiction, permitted data class, permitted consequence type, validation receipt reference, and escalation controls where applicable. If any required condition fails, the Finality Sink refuses effectuation.

Advanced Step 6 — Reconstruction, Unsealing, Combining, or Activation

In advanced embodiments, the Finality Sink may hold only incomplete effectuation material, including a sink-side key share, partial signature share, sealed token fragment, incomplete command authenticator, locked egress primitive, locked commit primitive, disabled actuator primitive, protected register state, hardware-held final-step condition, or similar incomplete execution component. The Execution Handle supplies, reconstructs, unseals, combines, or activates the missing complementary execution material. If reconstruction, unsealing, combining, or activation fails, the Finality Sink remains technically non-completable.

Advanced Step 7 — Effectuation Only After Completion Succeeds

Only after the Finality Sink verifies the Execution Handle and obtains the required execution material does the Finality Sink complete the Candidate Act. If the Execution Handle is absent, stale, revoked, copied, replayed, sink-mismatched, hardware-identity-mismatched, act-mismatched, epoch-mismatched, nonce-reused, or outside scope, the Finality Sink refuses effectuation. Thus, the advanced workflow changes the effectuation model from "the sink is permitted to act" to "the sink is unable to complete the act unless protected reconstruction succeeds."

Advanced Workflow Closing Statement

The advanced workflow provides stronger protection for high-risk artificial-intelligence-generated acts because the Finality Sink is not merely governed by advisory policy or software permission. The Finality Sink may be technically dependent on a valid Execution Handle and matching protected state before completion can occur. This advanced workflow prevents hallucinated, unsupported, stale, unsafe, or improperly routed Candidate Acts from becoming external consequences through moderation bypass, tool-call bypass, stale approval, state drift, replay, sink substitution, or downstream execution. **No valid Execution Handle, no sink completion.**

SEPARATE VARIATION WORKFLOW — ESCALATED BUT STILL ALLOWABLE FINALITY

Purpose

In some embodiments, the system applies an Escalated but Still Allowable Finality Workflow when a Candidate Act does not qualify for ordinary release but also does not require outright denial. This variation is used where the Candidate Act presents elevated risk, reduced confidence, near-boundary consequence sensitivity, sensitive data classification, heightened financial value, jurisdictional sensitivity, degraded-but-trusted sink state, operational sensitivity, or similar conditions, while still remaining capable of safe effectuation under stricter controls. The purpose of this variation is Technical feasibility: the system does not force every elevated-risk Candidate Act into denial, but keeps the act non-effective while additional safeguards are applied. If the safeguards reduce the act to a permitted consequence scope, the act may be released under escalated conditions.

Workflow Overview

In some embodiments, the Escalated but Still Allowable Finality Workflow follows this sequence:

Candidate Act → Non-Effective State → Primary Validation → Elevated-Risk Detection → Escalation Classification → Escalated RCAE or EASO Formation → Escalated Controls → Protected Approval if Required → Fresh State Verification → Escalated Receipt → Escalated Capability or Execution Handle → Finality Sink Verification → Limited Effectuation → Post-Effectuation Monitoring.

This workflow does not replace the base workflow. It is invoked only when ordinary release is insufficient but denial is not required.

Step 1 — Non-Effective Hold During Escalation

The Candidate Act remains in a non-effective state during escalation analysis. The Candidate Act may be staged, simulated, redacted, limited, delayed, reviewed, or prepared for additional validation, but it may not be transmitted, committed, executed, paid, exported, deployed, actuated, or otherwise effectuated while escalation is being evaluated.

Step 2 — Primary Validation

Before escalation is considered, the system performs primary validation, including one or more of ALF validation, RBD matching, HCAD integrity checking, OPC validation, FCU verification, RCAE comparison, consequence simulation, recipient verification, purpose verification, jurisdiction verification, data-class verification, policy-epoch verification, revocation-epoch verification, and Finality Sink compatibility checking. If the Candidate Act is prohibited, malicious, outside all permitted envelopes, unsupported beyond permitted tolerance, directed to a revoked sink, directed to a prohibited recipient, or incapable of being safely constrained, the Candidate Act is denied rather than escalated.

Step 3 — Elevated-Risk Detection

The Protected Enforcement Domain determines whether the Candidate Act presents an elevated-risk condition that does not require denial. Such a condition may include a predicted consequence close to a permitted boundary; a risk score above an ordinary release threshold but below a denial threshold; OPC limitation flags; FCUs having reduced but acceptable confidence; sources close to a freshness boundary; sensitive data class; sensitive but permitted recipient; high-value but authorized financial amount; jurisdiction requiring additional checks; degraded-but-trusted Finality Sink; reversible or partially reversible consequence; Technical necessity with operational sensitivity; protected human approval requirement; consequence simulation passing with elevated risk; perturbation testing passing with consequence sensitivity; or a requirement for monitoring, logging, escrow, delay, or revocation conditions.

Step 4 — Escalated but Still Allowable Classification

If the elevated-risk condition can be controlled by stricter safeguards, the Protected Enforcement Domain classifies the Candidate Act as escalated but still allowable. This means the Candidate Act is not ordinary allowed, is not denied, remains non-effective, and may proceed only under additional controls. If the elevated risk cannot be reduced to an acceptable permitted scope, the Candidate Act is denied, quarantined, further reduced in scope, or routed for protected review.

Step 5 — Escalated Consequence Boundary

The system generates, retrieves, or derives an escalated consequence boundary. In some embodiments, the boundary is an escalated RCAE. In other embodiments, the boundary is converted into a stricter EASO. The escalated boundary may limit recipient, purpose, jurisdiction, data class, financial value, operational scope, Finality Sink, execution time, tool, output content, reversibility, monitoring obligation, or consequence type. The escalated boundary defines the maximum consequence that may be allowed under escalation.

Step 6 — Escalated Controls

The Protected Enforcement Domain selects one or more controls required before escalated release. Such controls may include protected human approval, multi-party approval, role-constrained approval, quorum approval, shortened capability expiration, reduced transaction amount, reduced operational scope, redaction, recipient limitation, jurisdiction limitation, delayed effectuation,

reversible execution, escrowed execution, canary execution, sandbox-first execution, additional source verification, higher FCU confidence threshold, fresh Finality Sink attestation, stricter RCAE, stricter EASO, enhanced validation receipt fields, Residual Risk Receipt generation, post-effectuation monitoring, automatic revocation upon downstream state change, or forced revalidation before final completion. The selected controls are bound to the escalated finality decision.

Step 7 — Protected Approval Where Required

If the escalated boundary or selected controls require human approval, the system obtains a PHAFT bound to the Candidate Act, Candidate Act hash, predicted consequence, escalation reason, escalated controls, approving role, authenticated user presence, approval time, policy epoch, revocation epoch, RCAE or EASO, and Finality Sink identity. An ordinary click, ordinary approval message, email confirmation, chat instruction, generic workflow approval, stale approval, replayed approval, or synthetic user-interface event is insufficient where protected approval is required. If required protected approval is not obtained, the Candidate Act remains non-effective.

Step 8 — Fresh Current-State Verification

Before escalated release, the Protected Enforcement Domain verifies current protected state, including one or more of current policy epoch, current revocation epoch, Candidate Act hash, HCAD hash, OPC status, FCU status, escalated RCAE hash, EASO hash where used, recipient status, source authority status, model approval status, tool permission status, human approval status, Finality Sink identity, Finality Sink attestation, nonce freshness, and absence of revocation. If any required state has changed such that the escalated conditions are no longer valid, the Candidate Act is denied, returned to review, or reprocessed under a new finality decision.

Step 9 — Escalated Validation Receipt

If the escalated predicates are satisfied, the system generates an escalated validation receipt identifying one or more of Candidate Act hash, HCAD hash, OPC hash, FCU result, escalated RCAE hash, EASO hash where used, predicted consequence, escalation reason, selected escalated controls, protected approval reference, Finality Sink identity, policy epoch, revocation epoch, nonce, counter value, time window, and finality decision. The receipt records that the Candidate Act was not ordinary released but was permitted under additional safeguards.

Step 10 — Escalated Capability or Execution Handle Release

The Protected Enforcement Domain releases an escalated scoped non-bearer capability or escalated Execution Handle. The escalated authority is narrower than ordinary release and may be bound to shortened expiration, reduced amount, redacted fields, specific recipient, specific jurisdiction, specific purpose, specific Finality Sink, specific sink attestation, specific protected approval, specific escalation reason, specific monitoring obligation, specific reversibility condition, specific canary condition, specific RCAE, specific EASO, specific nonce, specific policy epoch, specific revocation epoch, and specific validation receipt. The escalated authority cannot be reused for ordinary release or expanded beyond the escalated scope.

Step 11 — Finality Sink Verification

The Finality Sink verifies the escalated capability or Execution Handle before effectuation. The Finality Sink may check Candidate Act hash, HCAD hash, authority authenticity, sink identity, sink hardware identity where required, expiration, nonce freshness, policy epoch, revocation epoch, recipient limitation, purpose limitation, jurisdiction limitation, data-class limitation, financial or

operational limit, redaction requirement, delay requirement, canary requirement, reversibility requirement, monitoring requirement, protected approval reference, validation receipt reference, and escalated finality mode. If any escalated condition fails, the Finality Sink refuses effectuation.

Step 12 — Limited Effectuation Under Escalated Scope

If the Finality Sink verifies all escalated conditions, the Candidate Act may be effectuated only within the escalated scope. The Finality Sink may not increase the amount, add recipients, remove redaction, bypass delay, skip monitoring, convert sandbox execution into production execution, convert reversible execution into irreversible execution, expand jurisdiction, alter purpose, change the data class, or ignore escalation controls. Effectuation is limited to the scope of the escalated authority.

Step 13 — Residual Risk Receipt

When a Candidate Act is escalated but still allowed, the system may generate a Residual Risk Receipt identifying why ordinary release was not used, why denial was not required, which elevated-risk conditions were present, which safeguards were applied, which predicates passed, which assumptions were relied upon, which limitation flags remained, which protected approval was obtained, which capability or Execution Handle was released, which Finality Sink accepted the authority, which policy epoch applied, which revocation epoch applied, and what residual risk remained.

Step 14 — Post-Effectuation Monitoring and Revocation

After escalated effectuation, the system may perform post-effectuation monitoring. The monitoring may detect recipient change, source revocation, policy update, revocation epoch advancement, sink trust change, unexpected downstream consequence, canary failure, rollback failure, excessive operational risk, new contradiction evidence, or violation of escalation conditions. If a monitored condition fails, the system may revoke related capabilities, advance a revocation epoch, quarantine related Candidate Acts, suspend future similar releases, generate a corrective receipt, route the matter for review, or deny future acts of the same class.

Failure Handling

If the escalated controls are not satisfied, the Candidate Act is not released. The system may deny the Candidate Act, quarantine it, route it for protected review, reduce the scope further, redact additional content, delay effectuation, require stronger approval, require fresh simulation, require updated provenance, or require a new finality decision. The Candidate Act remains non-effective unless a permitted finality path succeeds.

Closing Statement

The Escalated but Still Allowable Finality Workflow provides a practical middle path between unsafe ordinary release and unnecessary denial. It allows elevated-risk Candidate Acts to proceed only when the system can impose stricter controls that reduce the act to a permitted consequence scope. This variation supports Technical deployment because necessary artificial-intelligence-generated acts may occur under controlled, auditable, sink-verifiable conditions. The finality invariant remains unchanged: **no escalated Candidate Act becomes externally effective unless the escalated predicates are satisfied and the Finality Sink verifies the escalated authority.**

SUPPLEMENTARY ESCALATION VARIATIONS FOR Technically FEASIBLE FINALITY

Purpose of These Variations

The following variations supplement the Escalated but Still Allowable Finality Workflow. They provide additional mechanisms for handling elevated-risk Candidate Acts that are not suitable for ordinary release but do not require immediate denial. These variations may be used individually or in combination and may be implemented through a scoped non-bearer finality capability, an escalated capability, an Execution Authorization Scope Object, an Execution Handle, a State-Proof, a validation receipt, or a Finality Sink verification rule.

These variations preserve the same finality invariant: the Candidate Act remains non-effective unless the applicable escalated predicates are satisfied and the Finality Sink verifies the applicable scoped authority.

Variation 1 — Probabilistic Escalation Scoring With Confidence-Decaying Envelopes

In some embodiments, the Protected Enforcement Domain does not treat elevated-risk detection as a purely binary condition. Instead, the Protected Enforcement Domain computes a composite escalation score representing the degree to which the Candidate Act requires escalated conditional finality.

The composite escalation score may be represented as a normalized value, for example between 0.0 and 1.0, and may be computed from weighted sub-scores associated with consequence proximity, data sensitivity, factual-support confidence, source freshness, jurisdictional uncertainty, recipient sensitivity, financial value, operational risk, reversibility, downstream-agent risk, and Finality Sink trust degradation.

In some embodiments, the Protected Enforcement Domain also computes a confidence interval or uncertainty width associated with the composite escalation score. The confidence interval may depend on the freshness, completeness, reliability, and authority of the underlying validation inputs. For example, a Candidate Act evaluated using a fresh Factual Claim Unit, fresh Output Provenance Capsule, current sink attestation, and current source authority may have a narrower confidence interval than a Candidate Act evaluated using older factual support, stale source records, incomplete provenance, or delayed sink attestation.

In some embodiments, one or more validation inputs are associated with a freshness half-life or confidence-decay function. As the age of the input increases, the confidence in the escalation score decreases or the confidence interval widens. The same Candidate Act may therefore produce a different escalation decision depending on whether its supporting evidence is fresh or stale.

Escalated controls may be selected based on both the composite escalation score and the confidence interval width. A high-score, high-confidence Candidate Act may receive strict controls such as multi-party approval, shortened expiration, fresh sink attestation, stricter EASO, and post-effectuation monitoring. A moderate-score but low-confidence Candidate Act may be routed for mandatory re-evaluation, fresh FCU verification, updated OPC generation, source refresh, or new consequence simulation before any release authority is generated.

This variation allows the system to distinguish between a Candidate Act that is known to be risky and a Candidate Act whose risk is uncertain because the supporting inputs are stale, incomplete, or

degraded. In such cases, uncertainty itself becomes a machine-verifiable governance state rather than being collapsed into ordinary denial or ordinary release.

In some embodiments, the LAVR or Residual Risk Receipt records the escalation score, weighted sub-scores, confidence interval, freshness half-life parameters, evidence age, selected controls, and whether re-evaluation was required before release.

Variation 2 — Federated Multi-Domain Escalation

In some embodiments, a Candidate Act may produce a predicted consequence that crosses an organizational, tenant, jurisdictional, infrastructure, supply-chain, or administrative boundary. For example, an artificial-intelligence agent operating in a first domain may generate a Candidate Act whose Finality Sink, recipient, execution environment, data store, payment interface, or operational consequence belongs to a second domain.

In such embodiments, escalated conditional finality may require federated multi-domain validation. The Protected Enforcement Domain of the originating domain may not unilaterally release authority sufficient to effectuate the Candidate Act in the receiving or executing domain.

Instead, the escalated consequence boundary may require bilateral or multilateral policy binding. The escalated RCAE, EASO, State-Proof, scoped capability, or Execution Handle may be co-signed, counter-signed, jointly MAC-bound, jointly hash-linked, or otherwise cryptographically bound to policy epochs and revocation epochs from each participating domain.

In some embodiments, the Candidate Act may be released only when both the originating domain and the receiving domain maintain current valid policy epochs, current valid revocation epochs, compatible consequence boundaries, compatible recipient scope, compatible data-class scope, compatible sink trust state, and compatible approval requirements.

Neither domain's authority alone is sufficient to unblock the act. If either domain revokes the model, source, recipient, sink, approval, tool, policy, or execution scope, the Candidate Act remains non-effective or the escalated authority becomes invalid.

In some embodiments, the LAVR is jointly anchored or contains linked receipt commitments from multiple domains. The receipt may identify the originating domain's policy epoch, the receiving domain's policy epoch, the originating domain's revocation epoch, the receiving domain's revocation epoch, cross-domain RCAE or EASO hash, domain signatures, sink identity, and finality decision.

This variation provides a technical governance handshake for cross-domain artificial-intelligence execution, multi-tenant agent orchestration, supply-chain workflows, outsourced tools, regulated business-to-business acts, and shared infrastructure environments.

Variation 3 — Reversible Execution Staging With Rollback-Capability Escrow

In some embodiments, an escalated Candidate Act may be allowed only in a reversible execution mode. In such embodiments, the Finality Sink does not immediately complete irreversible effectuation. Instead, the Finality Sink stages a limited or speculative effectuation under an escrowed rollback condition.

The Protected Enforcement Domain may generate a paired authority set comprising a forward capability and a rollback capability. The forward capability permits limited effectuation of the Candidate Act under the escalated scope. The rollback capability permits reversal, undo, cancellation, compensation, restoration, or rollback of the same effectuation if a monitored condition fails during a defined monitoring window.

The forward capability and rollback capability may be cryptographically bound to the same Candidate Act hash, HCAD hash, RCAE hash, EASO hash, LAVR identifier, nonce, policy epoch, revocation epoch, Finality Sink identity, and protected transaction identifier. The rollback capability may be held in escrow by the Protected Enforcement Domain, a CIED, a trusted rollback module, a hardware security module, a protected database controller, a settlement controller, or another protected authority.

After the forward capability is used, a post-effectuation monitoring window begins. During this window, the system may monitor downstream contradiction evidence, source revocation, policy update, revocation epoch advancement, Finality Sink trust degradation, canary failure, rollback-health failure, recipient status change, jurisdictional change, or violation of escalation conditions.

If a monitored condition fails before expiry of the rollback window, the Protected Enforcement Domain releases or activates the rollback capability. The Finality Sink then executes the rollback, undo, cancellation, compensation, or restoration operation within the rollback scope.

If no monitored condition fails before expiry, the rollback capability may expire, be destroyed, be sealed into archival state, or be converted into a residual-risk record.

This variation provides an atomic paired-capability model for escalated acts: the same finality decision that permits forward effectuation also creates a protected rollback authority. The rollback authority is not an ordinary software retry. It is a protected, cryptographically bound, time-limited finality artifact.

Variation 4 — Consequence-Class Inheritance Limiter Across Agent Chains

In some embodiments, an escalated Candidate Act may produce an output, instruction, tool result, or intermediate state that becomes a new Candidate Act in a downstream agent, workflow engine, service, model, plugin, or automated process. Such multi-hop execution may create a risk that a moderate-risk escalated act is used to bootstrap a higher-consequence downstream act.

To prevent escalation laundering across agent chains, the system may generate a Consequence-Class Inheritance Tag, abbreviated CCIT, associated with the original escalated finality decision. The CCIT may define the maximum consequence class authorized for downstream acts derived from or caused by the original Candidate Act.

The CCIT may be cryptographically bound to the Candidate Act hash, HCAD hash, RCAE hash, EASO hash, escalation decision, permitted consequence class, policy epoch, revocation epoch, originating domain, downstream scope, and validation receipt identifier.

When a downstream agent or downstream Protected Enforcement Domain receives an output derived from the escalated Candidate Act, the downstream system checks the CCIT. The downstream Candidate Act cannot be assigned a consequence class higher than the inherited maximum unless a new independent finality process expressly authorizes a higher class under a new protected decision.

In some embodiments, escalation may only narrow downstream consequence authority and may not expand it. A downstream domain may impose stricter controls, lower the consequence class, require additional approval, reduce scope, or deny the downstream act, but it may not use the inherited escalation to authorize a higher-consequence act.

This variation prevents a chain of agents, tools, workflows, or services from laundering an escalated but limited authority into a broader or more dangerous authority. It is especially useful for multi-agent pipelines, tool chains, distributed orchestration systems, and delegated autonomous workflows.

Variation 5 — Synthetic Adversarial Perturbation Testing as an Escalation Gate

In some embodiments, synthetic adversarial perturbation testing is not merely an advisory signal. It is a mandatory gate before escalated RCAE formation, escalated EASO formation, or escalated capability release.

Before escalated authority is generated, the Protected Enforcement Domain may run a bounded set of synthetic perturbations against the Candidate Act's consequence model. Each perturbation may represent a minor parameterized variation of the Candidate Act, including a slightly different recipient, amount near a financial threshold, adjacent jurisdiction, nearby data class, altered tool argument, changed timing, substitute Finality Sink, delegated-agent path, split-act form, or similar boundary-adjacent modification.

The Protected Enforcement Domain evaluates whether any perturbation crosses into a prohibited consequence class or materially exceeds the permitted RCAE. If a prohibited neighbor exists within a defined perturbation radius, the system may tighten the escalated RCAE, narrow the EASO, reduce the financial amount, restrict the recipient, require additional approval, require sandbox or canary execution, or reclassify the Candidate Act as denial-required.

In some embodiments, the LAVR records the perturbation set, perturbation radius, tested dimensions, nearest prohibited neighbor distance, boundary-proximity result, tightened scope, and escalation decision.

This variation operationalizes boundary-proximity awareness as an enforcement step. The system does not merely ask whether the exact submitted Candidate Act narrowly passes validation. It evaluates whether small changes around the Candidate Act reveal dangerous consequence sensitivity before allowing escalated release.

Variation 6 — Temporal Commitment Splitting for High-Latency Finality Sinks

In some embodiments, a Finality Sink may have substantial latency between capability receipt and actual effectuation. Such sinks may include delayed settlement systems, satellite transmission systems, industrial control systems, remote infrastructure systems, cross-border execution systems, batch-processing systems, or other high-latency effectuation environments.

To reduce time-of-check-to-time-of-use risk, the system may split escalated authority into a Commitment Token and a Release Token.

The Commitment Token is issued after escalation validation and cryptographically binds the Finality Sink to hold the Candidate Act in a pre-committed non-effective state. The Commitment Token may reserve an execution slot, lock a queue position, preserve a bounded act state, prevent

competing conflicting acts, or stage required sink-side resources. However, the Commitment Token cannot itself be used to effectuate the Candidate Act.

The Release Token is issued separately after a defined confirmation window or immediately before effectuation, upon re-verification of current policy epoch, current revocation epoch, sink attestation, recipient status, source authority, and any required escalation conditions. The Release Token cannot be used unless it cryptographically matches the earlier Commitment Token.

In some embodiments, the Commitment Token and Release Token are cryptographically bound as a paired set using a shared transaction identifier, Candidate Act hash, HCAD hash, RCAE hash, EASO hash, sink identity, nonce, counter value, policy epoch, revocation epoch, and validation receipt reference.

If current-state verification fails before Release Token issuance, the Candidate Act remains pre-committed but non-effective, the commitment may expire, and the sink may release the reserved execution slot without effectuation.

This variation supports escalated finality for high-latency sinks by separating reservation from execution. It prevents an act from becoming effective merely because it was committed earlier, while also allowing the sink to reserve resources in a controlled non-effective state.

Variation 7 — Graduated Monitoring Intensity Decay

In some embodiments, post-effectuation monitoring is not merely active or inactive. Instead, monitoring intensity follows a protected decay schedule selected at escalation time.

The Protected Enforcement Domain may select a monitoring decay schedule based on the escalation score, risk class, consequence type, reversibility, recipient sensitivity, data class, jurisdictional condition, Finality Sink state, and residual uncertainty. The decay schedule may be linear, exponential, step-wise, event-based, threshold-based, or policy-defined.

The decay schedule may control sampling frequency, anomaly-threshold sensitivity, revocation-trigger sensitivity, rollback-trigger sensitivity, alert frequency, inspection depth, receipt-generation frequency, or monitoring duration. Higher-risk or higher-uncertainty acts may begin with more intense monitoring. If no adverse condition is detected, monitoring intensity may decrease according to the committed schedule.

In some embodiments, the decay schedule is cryptographically bound to the LAVR, Residual Risk Receipt, escalated capability, Execution Handle, EASO, or State-Proof. The schedule may not be silently weakened by ordinary software after release.

If an adverse condition is detected, monitoring intensity may reset to a maximum level, the decay curve may restart, a revocation epoch may advance, a rollback capability may be released, a corrective receipt may be generated, or future similar Candidate Acts may be routed to stricter finality.

This variation reduces operational overhead while preserving governance integrity. Monitoring may decrease over time only according to a protected schedule established during escalation, not as an unrecorded operational shortcut.

Supplementary Claim-Support Language for Escalation Variations

Probabilistic Escalation Score

The system of any preceding claim, wherein the Protected Enforcement Domain computes a composite escalation score for the Candidate Act from weighted sub-scores associated with consequence proximity, data sensitivity, factual-support confidence, source freshness, jurisdictional uncertainty, recipient sensitivity, financial value, operational risk, reversibility, downstream-agent risk, or Finality Sink trust state.

Confidence-Decaying Envelope

The system of any preceding claim, wherein at least one validation input has a freshness half-life or confidence-decay function, and wherein the system selects escalated controls based on both the composite escalation score and a confidence interval associated with the score.

Uncertainty-Based Re-Evaluation

The system of any preceding claim, wherein the Candidate Act is routed for fresh validation, source refresh, updated Factual Claim Unit verification, updated Output Provenance Capsule generation, updated sink attestation, or renewed consequence simulation when an uncertainty width exceeds a threshold.

Federated Multi-Domain Escalation

The system of any preceding claim, wherein escalated release of the Candidate Act across multiple domains requires a consequence boundary, Execution Authorization Scope Object, State-Proof, scoped capability, or Execution Handle cryptographically bound to policy epochs and revocation epochs of at least two domains.

Joint Receipt Anchoring

The system of any preceding claim, wherein a validation receipt for a cross-domain escalated Candidate Act includes or references receipt commitments from multiple participating domains.

Rollback-Capability Escrow

The system of any preceding claim, wherein an escalated release generates a forward capability and a rollback capability bound to the same Candidate Act, validation receipt, Finality Sink identity, nonce, policy epoch, and revocation epoch, and wherein the rollback capability is held in escrow during a monitoring window.

Consequence-Class Inheritance Tag

The system of any preceding claim, wherein a downstream Candidate Act derived from an escalated Candidate Act is limited by a cryptographically bound Consequence-Class Inheritance Tag defining a maximum permitted downstream consequence class.

Non-Expandable Downstream Escalation

The system of any preceding claim, wherein escalation authority inherited by a downstream agent, workflow, service, or domain may narrow but may not expand a permitted consequence class without a new protected finality decision.

Synthetic Adversarial Perturbation Gate

The system of any preceding claim, wherein the Protected Enforcement Domain performs a bounded set of synthetic perturbations before escalated release and tightens the escalated boundary or denies the Candidate Act when a prohibited consequence neighbor is detected within a defined perturbation radius.

Temporal Commitment Splitting

The system of any preceding claim, wherein escalated authority is split into a Commitment Token that reserves or stages a non-effective execution state and a Release Token that permits effectuation only after current-state re-verification.

Paired Commitment and Release Tokens

The system of any preceding claim, wherein the Release Token is unusable unless cryptographically matched to the Commitment Token for the same Candidate Act, Finality Sink, nonce, transaction identifier, policy epoch, and revocation epoch.

Graduated Monitoring Decay

The system of any preceding claim, wherein post-effectuation monitoring intensity follows a protected decay schedule bound to a validation receipt, Execution Authorization Scope Object, scoped capability, Execution Handle, Residual Risk Receipt, or State-Proof.

Monitoring Reset on Adverse Condition

The system of any preceding claim, wherein detection of an adverse condition resets monitoring intensity, restarts the decay schedule, advances a revocation epoch, releases a rollback capability, generates a corrective receipt, or routes related Candidate Acts to stricter finality review.

7. Receipt and Residual Risk Handling

Validation Receipt

In some embodiments, the system generates a validation receipt indicating which predicates were evaluated and which finality outcome was reached. The validation receipt may include one or more of: Candidate Act hash, HCAD hash, ALF validation result, RBD matching result, OPC validation result, FCU verification result, RCAE hash, consequence simulation result, finality decision, escalation decision, Protected Human Approval Finality Token reference, capability identifier, Execution Handle identifier where used, Finality Sink identity, policy epoch, revocation epoch, nonce, counter value, protected transaction identifier, and timestamp.

Residual Risk Receipt

Where a Candidate Act is permitted despite residual uncertainty, the system may generate a Residual Risk Receipt. The Residual Risk Receipt may identify which predicates passed, which assumptions were relied upon, which sources were used, which limitation flags applied, which residual risks remained, which escalation controls applied, which scoped capability or Execution Handle was released, which Finality Sink accepted the authority, which policy epoch applied, and which revocation epoch applied. A Residual Risk Receipt may be particularly useful for escalated

but still allowable acts, reduced-scope acts, limitation-flagged outputs, reversible acts, canary acts, or acts released under bounded uncertainty.

Post-Effectuation Handling

After effectuation, later evidence may trigger revocation, quarantine of similar Candidate Acts, policy update, revocation epoch advancement, source-authority downgrade, model-approval suspension, recipient restriction, sink-trust update, corrective receipt generation, post-effectuation review, rollback-capability release where available, stricter future escalation, or rejection of future similar Candidate Acts. This allows the system to respond to new information without treating earlier validation as a permanent guarantee of correctness, safety, legality, or Technical desirability.

8. Technical Feasibility Statement

The disclosed workflow is Technically feasible because it does not require every artificial-intelligence-generated output to be fully denied whenever uncertainty exists.

Instead, the system supports practical finality outcomes, including ordinary release, reduced-scope release, protected review, escalated release, reversible release, canary execution, delayed effectuation, and denial.

Escalated conditional finality is particularly important for enterprise and regulated environments because it allows necessary but elevated-risk acts to proceed under stricter safeguards.

The system therefore provides controlled flexibility while preserving the core finality invariant:

No Candidate Act becomes externally effective unless protected finality validation succeeds and the applicable Finality Sink verifies the scoped authority required for effectuation.

ADVANCED FINALITY SINK VERIFICATION — UNIFIED TECHNICAL SPECIFICATION

Foundational Principle

Finality enforcement in this architecture is not a permission check. A Candidate Act is technically non-completable until cryptographic, hardware-bound, epoch-bound, nonce-bound, receipt-bound, and sink-bound execution material is assembled at the applicable Finality Sink. No policy decision, software approval, access-control grant, or agent-side authorization constitutes effectuation. The Finality Sink — or a protected component positioned immediately before it — holds the final execution dependency. This dependency cannot be satisfied by ordinary software, a compromised agent, or a replayed authority object.

The finality invariant applies across all embodiments: **no Candidate Act becomes externally effective unless the Finality Sink or sink-adjacent enforcement component verifies, reconstructs, unseals, combines, or activates the scoped authority required for completion.**

I. Execution Dependency Models

1.1 Locked Primitive Execution Dependency

The Finality Sink holds a locked execution primitive — an encrypted command, sealed API invocation, incomplete transaction authenticator, locked database commit, partial payment authorization, sealed model-update instruction, or disabled actuator command — that is structurally unusable without completion material derived from the validated Candidate Act.

Completion material is derived from one or more of: Candidate Act hash, HCAD hash, RCAE hash, EASO hash, policy epoch, revocation epoch, nonce, validation receipt identifier, Finality Sink identity, and current sink attestation state. The Protected Enforcement Domain releases or derives completion material only after finality validation succeeds. The Candidate Act itself carries no long-term secret and no unrestricted execution key. A hallucinated or malicious output therefore cannot independently satisfy the locked primitive.

This converts enforcement from access control — where the system checks whether a requester is permitted — into cryptographic execution dependency, where the Finality Sink is structurally incapable of completing the act without protected finality release.

Implementation paths: sealed keys, key wrapping, threshold signatures, command MACs, encrypted API payloads, HSM-backed secrets, secure-enclave unsealing, database commit tokens, payment authorization shares, firmware unlock registers, hardware-gated execution paths. For legacy systems, a sink-adjacent proxy holds the incomplete command and releases a conventional API call only after Execution Handle verification. For native systems, the locked primitive is implemented directly inside the database commit layer, payment module, storage controller, tool execution controller, HSM, SmartNIC, DPU, or other sink-adjacent enforcement component.

1.2 Cryptographic Action Sharding — Split-Knowledge Finality

For high-risk Candidate Acts, execution material is divided into multiple cryptographic shares held by distinct protected components. No single component — including the AI agent, application process, or any single Finality Sink — possesses complete execution authority.

Components holding shares may include: agent-side protected wrapper, enterprise secure enclave, external hardware security module, biometric-bound approval device, jurisdictional enforcement module, policy authority, database commit controller, payment-security module, or independent Finality Sink. The required combination — defined as a threshold, quorum, or ordered sequence — is specified in the EASO at escalation time and cannot be modified during execution.

Each share is independently bound to: Candidate Act hash, HCAD hash, RCAE hash, EASO hash, LAVR identifier, State-Proof hash, nonce, policy epoch, revocation epoch, protected approval state, sink attestation, and Finality Sink identity. Reconstruction at the Finality Sink succeeds only when all required shares are simultaneously valid, fresh, scope-matching, epoch-current, and sink-bound. A missing, stale, replayed, revoked, or scope-mismatched share renders the Candidate Act technically non-completable.

The critical property: a compromised AI agent or compromised application process cannot independently complete the act because required shares reside outside the ordinary agent execution path and are released only by independent protected domains.

Implementation paths: threshold cryptography, multi-party signatures, distributed HSMs, quorum-based key release, multi-region key management, dual-control authorization modules, secure enclaves, split command authorization. Verification timing constraints — requiring shares to arrive within a bounded window — prevent accumulation and replay of stale partial approvals.

II. Temporal Execution Control

2.1 Commitment-Release Token Pair

An escalated or high-risk Candidate Act may be placed in a temporal escrow state rather than immediately effectuated after initial validation. The system generates two cryptographically distinct artifacts:

Commitment Token: Binds the Candidate Act to the Finality Sink, EASO, RCAE, predicted consequence, policy epoch, revocation epoch, nonce, sink attestation, and validation receipt. The Commitment Token stages, pre-commits, or simulates the Candidate Act but is cryptographically incapable of causing external effectuation without a corresponding Release Token. An upstream workflow may receive a non-final staged-status indicator while the external consequence remains sealed in escrow; this indicator is not effectuation and does not satisfy downstream execution dependencies.

Release Token: Generated only after a confirmation window and current-state re-verification covering: policy-epoch verification, revocation-epoch verification, sink-attestation verification, recipient-status verification, source-authority verification, FCU freshness checking, monitoring-signal checking, protected approval checking, and absence-of-revocation checking. The Release Token is cryptographically bound to the earlier Commitment Token such that neither token is independently sufficient for effectuation.

If a failure condition is detected before Release Token issuance — including contradiction evidence, source revocation, policy update, recipient change, sink-trust degradation, canary failure, approval withdrawal, jurisdictional change, or revocation-epoch advancement — the Commitment Token expires or is destroyed and no Release Token is generated. No fallback path permits effectuation from an expired Commitment Token.

2.2 Paired Forward and Rollback Capabilities

Temporal escrow may be combined with paired-capability issuance. At escalation time, the system generates two authority objects bound to the same Candidate Act:

Forward capability: Permits limited effectuation within the escalated consequence scope.

Rollback capability: Permits undo, cancellation, restoration, compensation, settlement reversal, database rollback, or other bounded remediation if a monitored condition fails within a defined monitoring window.

Both are cryptographically bound to the same: Candidate Act hash, HCAD hash, RCAE hash, EASO hash, LAVR identifier, State-Proof hash, Finality Sink identity, protected transaction identifier, nonce, policy epoch, and revocation epoch.

The rollback capability is held in escrow by the Protected Enforcement Domain, CIED, hardware security module, secure enclave, database controller, payment controller, settlement controller, or other protected rollback authority — never by the agent or application layer. If the monitoring window expires without an adverse condition, the rollback capability expires, is destroyed, or is sealed into archival state. If an adverse condition is detected within the window, the rollback capability is released and the Finality Sink executes the bounded reversal. The forward effectuation is therefore conditionally complete: it has occurred, but remains subject to cryptographically enforceable reversal during the monitoring window.

The inventive claim is the atomic paired issuance: forward and rollback capabilities are generated simultaneously at escalation time, not added opportunistically after effectuation. The rollback authority is a protected artifact with independent cryptographic standing, not a software retry.

III. Sink Architecture Variants

3.1 Ephemeral Finality Sink

The Finality Sink may be provisioned as a temporary single-use execution environment rather than a persistent endpoint. The Protected Enforcement Domain instantiates an Ephemeral Finality Sink as a micro-VM, secure enclave instance, serverless function, isolated process, container, short-lived tool executor, temporary database role, transient payment executor, or just-in-time network-operation module.

The ephemeral sink is bound to exactly one validated Candidate Act. It contains only the minimum credentials, permissions, policy state, recipient scope, data scope, tool scope, time window, and execution primitive needed for that act. Its identity, attestation measurement, and nonce exist only for the duration of the act.

After completion, refusal, rollback, timeout, or expiration, the ephemeral sink destroys itself: deletes temporary credentials, clears nonce state, invalidates session keys, erases transient execution material, and prevents reuse. A capability or Execution Handle copied from one ephemeral sink instance cannot be replayed against any later sink because the original sink identity, nonce, and attested execution context no longer exist.

This eliminates long-term credential exposure, persistent backdoor risk, stale tool access, and replay from persistent credential stores. It is particularly suited to AI agents requiring access to high-impact tools without receiving persistent credentials that survive individual act authorization.

3.2 Multi-Assurance Sink Profiles

Finality Sinks operate at defined assurance levels. The Protected Enforcement Domain selects the required profile at escalation time and binds it into the RCAE, EASO, scoped capability, Execution Handle, validation receipt, or State-Proof. The Finality Sink cannot silently downgrade the required verification level.

Low assurance: hash matching, nonce checking, scope comparison, epoch verification.

Medium assurance: adds validation receipt reference, PHAFT reference, RCAE hash, EASO hash, and sink attestation.

High assurance: adds Execution Handle verification, State-Proof verification, hardware identity binding, locked primitive completion, rollback capability escrow, or multi-sink threshold verification.

Profile selection is driven by Candidate Act risk class, consequence type, data class, jurisdiction, financial value, reversibility, escalation score, and current sink trust state. The binding of the selected profile into protected artifacts prevents runtime downgrade by any component — including the agent, proxy, or Finality Sink itself.

3.3 Native and Non-Native Sink Compatibility

Native sinks directly verify the scoped capability, Execution Handle, State-Proof, receipt reference, nonce, epoch, and sink identity before effectuation. Verification logic resides inside the sink's own execution boundary.

Non-native sinks are protected by a sink-adjacent proxy, gateway sidecar, secure enclave wrapper, HSM adapter, database proxy, message-broker interceptor, or DPU/SmartNIC enforcement layer. The proxy becomes the enforcement boundary. Unverified Candidate Acts never reach the legacy sink in executable form. Verification output is translated into a conventional downstream request only after all finality predicates are satisfied.

This provides a technical migration path from proxy-based enforcement to native sink verification without changing the finality invariant. Both paths enforce the same non-completability guarantee; they differ only in where within the sink boundary verification occurs.

IV. Proxy-Based Interface Enforcement

A protected Finality Sink proxy is positioned immediately before an existing API, database, payment module, storage system, tool executor, communication system, workflow engine, network controller, model-update interface, or other effectuation interface. The proxy intercepts every Candidate Act, tool call, transaction request, database mutation, data-export request, command message, or act-equivalent representation.

Before forwarding any request to the downstream interface, the proxy performs: HCAD verification, Candidate Act hash comparison, scoped capability or Execution Handle verification, nonce checking, expiration checking, policy-epoch and revocation-epoch checking, RCAE and EASO scope comparison, Finality Sink identity and attestation checking, PHAFT reference checking, escalation-control checking, LAVR verification, and State-Proof verification. If all predicates are satisfied, the proxy converts the validated Candidate Act into a conventional downstream request. If any predicate fails, the proxy refuses to forward and the Candidate Act remains non-effective with no fallback path.

The proxy constitutes the Finality Sink when it is the first technical boundary at which the Candidate Act becomes capable of producing an external consequence. Where the downstream interface performs additional execution, the proxy is a sink-adjacent enforcement component protecting a non-native downstream sink.

The downstream interface receives executable requests only after proxy-side finality enforcement. It need not natively parse HCAD, ALF, RCAE, EASO, LAVR, or Execution Handle formats. This permits finality enforcement to be layered onto existing production infrastructure without requiring internal modification of the downstream system.

V. Modular Escalation Enforcement

When a Candidate Act is escalated-but-allowable, the Protected Enforcement Domain routes it to one or more protected escalation modules before releasing any scoped capability or Execution Handle. Each module operates independently and returns a machine-verifiable result.

Modules may include: protected human approval, multi-party quorum approval, role-constrained approval, source-refresh validation, legal-rule validation, jurisdictional compliance validation, secondary-model verification, adversarial perturbation testing, canary-execution preparation,

rollback-capability preparation, fresh sink attestation acquisition, stricter RCAE or EASO formation, and residual-risk calculation.

Each module result is cryptographically bound to: Candidate Act hash, HCAD hash, RCAE hash, EASO hash, LAVR identifier, module identity, policy epoch, revocation epoch, nonce, and Finality Sink identity. The EASO specifies which module results are mandatory. The Protected Enforcement Domain releases the escalated Execution Handle only when the required combination of module results is present, valid, non-stale, non-revoked, and cryptographically consistent with the same Candidate Act and current protected state.

The Finality Sink rejects the Execution Handle if any required module result is absent, stale, revoked, epoch-mismatched, or not bound to the same Candidate Act. Module results cannot be transferred across Candidate Acts or reused across policy epochs.

VI. Receipt-Proofed Finality Evidence

6.1 Receipt as Release Predicate

In this architecture, receipt generation is not a post-effectuation audit function. It is a release predicate. A scoped capability or Execution Handle is not generated unless the corresponding validation receipt, residual-risk receipt, State-Proof, escalation record, rollback record, sink-verification record, or effectuation record is first created, committed, sealed, or hash-linked in protected state.

The Finality Sink treats the receipt identifier, receipt hash, or State-Proof as a required verification input. If the receipt state does not cryptographically correspond to the Candidate Act, capability, Execution Handle, nonce, epoch, sink identity, and EASO, effectuation is refused. The receipt is therefore structurally embedded in the release path — not appended to it.

6.2 Receipt Content

The receipt-proofed evidence records: Candidate Act hash, HCAD hash, ALF result, RBD result, OPC hash, FCU result, RCAE hash, EASO hash, consequence simulation result, escalation decision, PHAFT reference, State-Proof hash, capability or Execution Handle identifier, Finality Sink identity, sink attestation hash, policy epoch, revocation epoch, nonce, counter value, protected transaction identifier, and timestamp.

For escalated acts, the receipt additionally records: escalation reason, selected escalation controls, protected approval reference, and which escalation module results were mandatory and verified.

For rollback-capable acts, the receipt records: rollback capability identifier, monitoring window boundaries, and rollback authority holder.

6.3 Implementation

Receipt state may be implemented using append-only logs, secure audit stores, hash chains, Merkle commitments, tamper-evident databases, hardware-protected logs, signed receipt objects, HSM-backed counters, secure enclave state, or local receipt commitment with asynchronous external anchoring. To minimize latency on the hot path, the system commits a local protected receipt immediately before capability release and performs external anchoring asynchronously without blocking effectuation.

VII. Protected Hardware Finality Sink

The Finality Sink or sink-adjacent enforcement module may be implemented in a hardware security module, secure enclave, trusted execution environment, secure element, SmartNIC, DPU, firmware-protected controller, database commit controller, storage controller, network processor, payment-security module, or protected microcontroller.

The protected hardware performs compact verification against: Candidate Act hash, HCAD hash, scoped capability or Execution Handle, State-Proof, LAVR hash, nonce, policy epoch, revocation epoch, EASO hash, RCAE hash, sink identity, and sink attestation. After verification, the hardware releases the specific execution material required for the validated Candidate Act: command signatures, database commit tokens, payment authorization shares, data-export keys, network-control authorization codes, storage-write tokens, actuator enable values, or model-update unlock values.

Ordinary application software cannot extract execution material from the hardware domain and cannot force effectuation without a valid scoped capability or Execution Handle. The hardware does not execute a full AI model or perform consequence simulation — it performs compact sink-side verification against artifacts generated earlier by the Protected Enforcement Domain, then releases or withholds execution material accordingly.

Risk-tiered hardware assurance:

- Low-risk acts: proxy-level verification, no hardware requirement.
- Medium-risk acts: secure-enclave verification, attested sink verification, or receipt-bound release.
- High-risk acts: HSM-backed locked primitive release, threshold verification, multi-sink sharding, rollback-capability escrow, or hardware-rooted non-completability, where ordinary software cannot produce effectuation under any execution path.

VIII. Interface Protocol for Machine-Verifiable Finality Exchange

A machine-verifiable interface protocol defines how AI agents, Protected Enforcement Domains, proxy enforcement modules, and Finality Sinks exchange finality artifacts. The protocol specifies: message formats, canonical encoding, signature and MAC formats, nonce handling, epoch handling, sink-attestation formats, Candidate Act descriptor formats, scoped capability formats, Execution Handle formats, validation receipt formats, State-Proof formats, rejection formats, and rollback or corrective receipt formats.

The protocol supports multiple finality modes: ordinary release, reduced-scope release, escalated release, delayed release, reversible release, canary release, multi-sink threshold release, proxy-mediated release, and Execution-Handle-based non-completable release.

A conformant Finality Sink advertises supported capabilities: attestation method, cryptographic suites, receipt format, nonce model, epoch model, sink identity method, supported escalation controls, rollback capability support, latency class, and effectuation modes.

An AI agent or Protected Enforcement Domain may interact only with Finality Sinks that expose compatible verification semantics for the required assurance profile. Where a downstream interface does not natively support the protocol, a protected proxy translates protocol-level finality artifacts into conventional downstream requests only after successful proxy-side verification. Different sinks

may implement different assurance levels, ranging from proxy-level verification to native HSM-backed Execution Handle reconstruction, without changing the finality invariant enforced at each.

Consolidated Invariant

Across every embodiment — locked primitive, sharded execution, temporal escrow, paired forward-rollback, ephemeral sink, proxy enforcement, modular escalation, receipt-proofed release, hardware finality sink, or multi-assurance profile — the invariant is identical and unconditional:

A Candidate Act is not merely allowed or denied by policy. It is technically non-completable until protected finality validation causes the required cryptographic, hardware-bound, sink-bound, scope-bound, epoch-bound, nonce-bound, and receipt-bound execution material to become available at the Finality Sink. No ordinary software path, no compromised agent, and no replayed authority object can satisfy this dependency without protected finality release.

COMMON ENABLEMENT AND PHOSITA STATEMENT FOR BASE FINALITY, ESCALATED FINALITY, AND ADVANCED EXECUTION-DEPENDENT FINALITY SINK EMBODIMENTS

Purpose of This Section

The following enablement and PHOSITA statement applies commonly to the base Candidate-Act finality workflow, the graduated and escalated conditional finality workflow, and the advanced Finality Sink verification embodiments, including locked primitive execution dependency, cryptographic action sharding, temporal escrow, paired forward-and-rollback capabilities, ephemeral Finality Sinks, proxy-based interface enforcement, modular escalation enforcement, receipt-proofed release, risk-tiered hardware Finality Sinks, native and non-native sink compatibility, multi-assurance sink profiles, and machine-verifiable Finality Sink interface protocols.

These embodiments share the same technical foundation: an artificial-intelligence-generated output is converted into a non-effective Candidate Act, evaluated under protected finality predicates, and permitted to become externally effective only when the required scoped authority is released and verified at the applicable Finality Sink or sink-adjacent protected enforcement component.

1. Person Having Ordinary Skill in the Art

A person having ordinary skill in the art, referred to herein as a **PHOSITA**, may include an engineer, architect, or technical practitioner having practical knowledge of one or more of secure distributed systems, cryptographic protocols, artificial-intelligence deployment systems, autonomous-agent orchestration, trusted execution environments, hardware security modules, secure enclaves, database commit systems, API gateways, cloud infrastructure, service meshes, identity and access systems, payment-security systems, telecom or network-control systems, secure logging, or safety-critical execution control.

Such a PHOSITA would understand how to implement software, firmware, hardware, or hybrid systems that receive structured requests, generate descriptors, compute hashes, verify signatures or

MACs, check nonces and epochs, bind artifacts to identities and scopes, enforce revocation state, maintain tamper-evident logs, verify attestations, and prevent an operation from reaching an execution boundary until required checks have passed.

A PHOSITA would not need to invent new cryptographic primitives to practice the disclosed architecture. Existing primitives such as secure hashes, digital signatures, MACs, key wrapping, authenticated encryption, Merkle commitments, monotonic counters, nonce tables, hardware attestation, threshold signatures, secure enclaves, HSM operations, append-only logs, and secure audit stores may be used. The inventive contribution lies in the protected ordering, binding, non-effective hold, capability or Execution Handle release, receipt-bound finality, and Finality Sink verification sequence.

2. Common Enablement of the Base Candidate-Act Finality Workflow

The base workflow may be implemented by configuring an artificial-intelligence system, agent runtime, tool dispatcher, workflow engine, API gateway, database controller, storage controller, payment module, communication gateway, or other effect-capable system to convert an AI-generated output into a Candidate Act before external effectuation.

A PHOSITA can implement the Candidate Act as a structured data object containing or referencing output content, output hash, requested operation, intended recipient, intended tool, intended Finality Sink, purpose, jurisdiction, data class, risk class, timestamp, model identifier, workflow identifier, ALF identifier, and other context fields. The Candidate Act may be stored in a queue, buffer, protected memory region, transaction staging table, message broker, secure enclave, HSM-backed state store, or other non-effective hold location.

The Protected Enforcement Domain may then generate an HCAD by hashing and binding the Candidate Act to relevant context, evidence, policy epoch, revocation epoch, nonce, Finality Sink identity, and validation-state references. The PED may validate ALF, compare RBD against an approved envelope, validate OPC, verify FCUs where required, retrieve or generate an RCAE, perform consequence simulation, and evaluate output finality predicates.

If required predicates pass, the PED releases a scoped non-bearer finality capability bound to the Candidate Act, HCAD, RCAE, Finality Sink identity, purpose, recipient, jurisdiction, data class, permitted consequence, time window, nonce, policy epoch, revocation epoch, and validation receipt. The Finality Sink verifies the capability before effectuation. If verification fails, the Candidate Act remains non-effective.

3. Common Enablement of Graduated and Escalated Conditional Finality

A PHOSITA can implement graduated finality by configuring the Protected Enforcement Domain to classify Candidate Acts into one or more finality outcomes, including ordinary release, reduced-scope release, redacted release, delayed release, sandbox-only release, canary release, reversible release, escrowed release, protected-review staging, escalated release, quarantine, rejection, or denial.

Escalated conditional finality may be implemented by defining risk thresholds, consequence classes, freshness requirements, confidence thresholds, sink-trust states, approval requirements, jurisdictional rules, data-class rules, recipient restrictions, perturbation-test results, and residual-risk conditions. When the Candidate Act does not qualify for ordinary release but remains within a

permitted boundary under stricter controls, the PED may classify the Candidate Act as escalated but still allowable.

In such embodiments, the PED may generate or select a stricter RCAE, stricter EASO, shorter validity window, reduced amount, redacted field set, limited recipient class, jurisdiction limitation, fresh sink attestation requirement, PHAFT requirement, multi-party approval requirement, monitoring requirement, rollback requirement, canary requirement, or residual-risk receipt requirement. The escalated capability or Execution Handle is then bound to these additional controls, and the Finality Sink verifies them before limited effectuation.

Thus, escalation is enabled as a technical finality mode rather than a business decision or abstract review process. The act remains non-effective until the escalated predicates are satisfied and sink-side verification confirms that the requested effectuation remains within the escalated scope.

4. Common Enablement of Advanced Execution-Dependent Finality Sink Embodiments

The advanced embodiments may be implemented by configuring the Finality Sink, or a sink-adjacent protected component, so that the Candidate Act is technically non-completable unless protected finality validation releases, reconstructs, unseals, combines, or activates missing execution material.

A PHOSITA can implement this using an Execution Authorization Scope Object and Execution Handle. The EASO converts the permitted consequence boundary into an execution-enforceable cryptographic scope. The Execution Handle is generated only after required finality predicates pass and may be bound to Candidate Act hash, HCAD hash, RCAE hash, EASO hash, LAVR identifier, State-Proof hash, nonce, counter value, policy epoch, revocation epoch, Finality Sink identity, sink attestation, permitted purpose, permitted recipient, permitted jurisdiction, permitted data class, and permitted consequence.

In locked primitive embodiments, the Finality Sink may hold an incomplete command authenticator, sealed API invocation, locked database commit primitive, partial payment authorization, sealed data-export key, disabled actuator primitive, or hardware-held final-step state. The Execution Handle supplies or activates only the missing completion material required for the validated Candidate Act.

In sharded embodiments, completion material may be divided among multiple protected components, such as HSMs, secure enclaves, approval devices, policy authorities, jurisdictional modules, payment-security modules, or independent Finality Sinks. The Finality Sink completes the act only when the required quorum or defined share combination is valid, fresh, scope-matching, epoch-current, nonce-matching, and sink-bound.

In ephemeral sink embodiments, the PED may instantiate a temporary micro-VM, secure enclave instance, isolated process, serverless function, temporary database role, short-lived payment executor, or transient network-operation module configured only for the validated Candidate Act. After completion, refusal, timeout, rollback, or expiration, the ephemeral sink invalidates or destroys its credentials, nonce state, session keys, and execution material.

5. Common Enablement of Receipt-Bound Release and State-Proof Verification

A PHOSITA can implement LAVRs, Residual Risk Receipts, State-Proofs, escalation records, rollback records, sink-verification records, or effectuation records using append-only logs, hash chains, Merkle commitments, tamper-evident databases, secure audit stores, HSM-backed counters, secure enclave state, hardware-protected logs, or local protected receipt chains.

The receipt may include Candidate Act hash, HCAD hash, ALF result, RBD result, OPC hash, FCU result, RCAE hash, EASO hash, consequence simulation result, escalation decision, PHAFT reference, capability identifier, Execution Handle identifier, Finality Sink identity, sink attestation hash, policy epoch, revocation epoch, nonce, counter value, protected transaction identifier, and timestamp.

In some embodiments, receipt generation is part of the protected finality transaction. No capability or Execution Handle is released unless the corresponding receipt, State-Proof, or protected receipt commitment is created, sealed, committed, or hash-linked in protected state. The Finality Sink may verify the receipt hash, receipt identifier, State-Proof, nonce, epoch, Candidate Act hash, EASO hash, and sink identity before effectuation.

This enables receipt-bound finality without requiring the receipt system to guarantee absolute truth. The receipt proves bounded predicate satisfaction under a defined protected state.

6. Common Enablement of Temporal Escrow and Rollback-Capability Embodiments

Temporal escrow may be implemented by separating staging authority from effectuation authority. A Commitment Token may bind the Candidate Act to the Finality Sink, EASO, RCAE, predicted consequence, policy epoch, revocation epoch, nonce, sink attestation, and validation receipt. The Commitment Token may reserve, stage, buffer, or pre-commit the Candidate Act but is incapable of causing external effectuation without a Release Token.

The Release Token may be generated only after a confirmation window and current-state re-verification. Such re-verification may include policy epoch, revocation epoch, sink attestation, recipient status, source authority, FCU freshness, monitoring signals, protected approval, and absence of revocation.

Rollback-capability embodiments may be implemented by generating a paired forward capability and rollback capability. The forward capability permits limited effectuation. The rollback capability permits undo, cancellation, compensation, settlement reversal, database rollback, command reversal, or other bounded remediation if a monitored condition fails during a defined window. The rollback capability may be held by a PED, CIED, HSM, secure enclave, database controller, settlement controller, or other protected rollback authority.

7. Common Enablement of Proxy-Based and Native / Non-Native Sink Integration

A PHOSITA can implement proxy-based Finality Sink enforcement by placing a protected proxy before a non-native API, database, payment gateway, storage system, message broker, tool executor, communication system, network controller, or model-update interface. The proxy verifies the scoped capability, Execution Handle, State-Proof, receipt reference, nonce, policy epoch, revocation epoch, sink identity, sink attestation, and scope constraints before forwarding a conventional downstream request.

Where the downstream system natively supports finality verification, the Finality Sink may directly verify the finality artifacts. Where the downstream system does not natively support such verification, the proxy may act as the first technical boundary at which the Candidate Act becomes capable of producing an external consequence. In that case, the proxy itself may be treated as the Finality Sink or as a sink-adjacent enforcement domain.

This supports both native and non-native implementations while preserving the same invariant: unverified Candidate Acts do not reach an effectuation interface in executable form.

8. Common Enablement of Latency-Aware Implementation

The architecture may be implemented using a cold validation path, nearline preparation path, and hot finality path.

The cold validation path may prepare policy templates, ALF approvals, sink registrations, source registrations, revocation lists, RCAE templates, EASO templates, cryptographic suite identifiers, and reusable verification material.

The nearline preparation path may operate after a specific AI output is generated and may perform Candidate Act formation, HCAD generation, RBD collection, OPC construction, FCU verification, RCAE selection, consequence simulation, escalation classification, State-Proof generation, receipt preparation, and Execution Handle preparation.

The hot finality path may operate at or near the Finality Sink and may perform compact verification, including Candidate Act hash checking, nonce checking, expiration checking, policy-epoch comparison, revocation-epoch comparison, sink-identity verification, sink-attestation verification, receipt-reference verification, State-Proof verification, scoped capability verification, Execution Handle verification, and reconstruction or unsealing of execution material.

This separation enables technically practical deployment because the Finality Sink need not execute the AI model, perform full factual reasoning, run complete consequence simulation, or perform full adversarial analysis at effectuation time.

9. No Undue Experimentation Required

The disclosed embodiments can be practiced using known secure computing, cryptographic, distributed-system, and hardware-security techniques applied in the disclosed finality sequence. A PHOSITA would be able to implement the Candidate Act object, HCAD generation, ALF reference verification, RBD comparison, OPC validation, FCU verification, RCAE generation, consequence simulation, EASO formation, scoped capability generation, Execution Handle release, LAVR generation, State-Proof binding, nonce checking, policy-epoch checking, revocation-epoch checking, sink attestation, and Finality Sink verification without undue experimentation.

Different implementations may use different cryptographic suites, hardware roots of trust, storage mechanisms, policy languages, simulation engines, approval devices, proxy frameworks, HSMs, TEEs, secure enclaves, SmartNICs, DPUs, or distributed receipt systems. Such implementation choices do not change the inventive finality sequence because the required transition remains the same: the Candidate Act remains non-effective until protected validation releases sink-verifiable authority.

10. Common Technical Invariant

Across the base workflow, escalated conditional finality, and advanced execution-dependent Finality Sink embodiments, the technical invariant is:

No artificial-intelligence-generated output is authority to act. No Candidate Act becomes externally effective unless protected finality predicates are satisfied and the applicable Finality Sink or sink-adjacent protected enforcement component verifies, reconstructs, unseals, combines, or activates the scoped authority required for completion.

In the base workflow, this authority may be a scoped non-bearer finality capability.

In escalated conditional finality, this authority may be narrower, shorter-lived, receipt-bound, approval-bound, monitoring-bound, rollback-bound, canary-bound, or EASO-bound.

In advanced execution-dependent embodiments, this authority may be an Execution Handle or equivalent protected artifact that supplies, reconstructs, unseals, combines, or activates missing execution material.

Thus, the invention is enabled across software, firmware, hardware, proxy-based, native-sink, non-native-sink, escrowed, sharded, reversible, and hardware-rooted implementations without departing from the same protected output-to-consequence finality architecture.

DESIGN-AROUND CLOSURES FOR BASE WORKFLOW, ESCALATED FINALITY WORKFLOW, AND ADVANCED EXECUTION-DEPENDENT WORKFLOW

Purpose of This Section

The following design-around closures apply to the base Candidate-Act finality workflow, the graduated and escalated conditional finality workflow, and the advanced execution-dependent Finality Sink workflow. These closures prevent an artificial-intelligence-generated output, Candidate Act, act-equivalent representation, tool call, delegated action, delayed action, fragmented action, proxy-mediated action, or downstream-agent action from bypassing the protected output-to-consequence finality sequence.

Unless expressly stated otherwise, the closures apply whether the system releases a scoped non-bearer finality capability, an escalated capability, an Execution Handle, a Commitment Token, a Release Token, a rollback capability, a State-Proof, a LAVR-bound authority, or another sink-verifiable finality artifact.

The common rule is: **no act-equivalent representation may become externally effective unless it passes the applicable finality workflow and the applicable Finality Sink verifies the scoped authority required for completion.**

1. Closure Against Treating AI Output as Authority

An artificial-intelligence-generated output is not authority to act merely because it was generated by an approved model, approved workflow, approved prompt, approved tool policy, approved runtime environment, or approved user session. The output must be converted into a Candidate Act and held

in a non-effective state before effectuation. A system does not avoid this requirement by characterizing the output as a recommendation, suggestion, tool argument, intermediate plan, function call, message draft, execution hint, plugin instruction, or agent memory entry if the output is capable of causing the same or substantially similar external consequence.

2. Closure Against Bypassing Candidate Act Formation

A system may not avoid the finality workflow by transmitting, encoding, summarizing, compressing, translating, encrypting, batching, queueing, scheduling, delegating, or otherwise transforming the AI output before Candidate Act formation. Any act-equivalent representation that can cause a tool execution, data disclosure, payment, database mutation, memory write, model update, network change, communication, storage operation, settlement, physical actuation, or other external consequence is treated as a Candidate Act or as derived from a Candidate Act.

3. Closure Against Bypassing the Non-Effective Hold

A Candidate Act remains non-effective until the required finality predicates are satisfied. The non-effective hold is not avoided by buffering the act in an application queue, message broker, tool dispatcher, workflow engine, database transaction log, delayed job runner, plugin manager, API gateway, agent memory, cache, scheduler, or human-review queue. If the act can later produce external consequence, the hold state continues until the applicable capability, Execution Handle, Release Token, or equivalent sink-verifiable authority is validly released and verified.

4. Closure Against Descriptor Substitution

A Candidate Act may not be replaced after HCAD generation by substituting different content, recipient, purpose, jurisdiction, data class, tool, Finality Sink, requested consequence, policy epoch, revocation epoch, evidence reference, or simulation result. The HCAD, LAVR, State-Proof, scoped capability, Execution Handle, and Finality Sink verification context may be hash-linked or otherwise cryptographically bound so that any substitution causes verification failure.

5. Closure Against Model-Approval Laundering

Approval of an ALF, model version, workflow graph, system prompt policy, retrieval policy, memory policy, tool-use policy, or deployment configuration does not authorize the specific output to become consequence. A system does not avoid finality validation by proving that the model or workflow was approved. ALF validation is a process-integrity predicate, not an effectuation authority. The Candidate Act must still satisfy output-level, provenance-level, factual-support-level, consequence-level, jurisdiction-level, epoch-level, and sink-level predicates.

6. Closure Against Runtime-Behavior Laundering

A valid Runtime Behavioral Descriptor does not itself authorize effectuation. A system does not avoid finality validation by showing that the runtime behavior stayed within an approved envelope. Runtime behavior may be valid while the specific output remains unsupported, stale, unsafe, outside the RCAE, outside the EASO, or incompatible with the intended Finality Sink. RBD matching is therefore insufficient unless the remaining finality predicates also pass.

7. Closure Against Provenance-Only Release

An Output Provenance Capsule or source record does not itself authorize consequence. A system does not avoid the workflow by attaching citations, retrieval logs, database records, sensor records, tool outputs, confidence values, or evidence hashes to the output. Provenance must be validated, bound to the Candidate Act, evaluated against permitted use, checked for freshness and contradiction, and compared against the intended consequence before release authority is generated.

8. Closure Against Factual-Claim Laundering

A factual assertion may not become externally effective merely because it appears fluent, probable, source-like, or partially supported. Where FCU verification is required, each required Factual Claim Unit must satisfy the applicable support, freshness, confidence, contradiction, jurisdiction, permitted-use, and consequence-scope predicates. Unsupported or stale factual material may be denied, redacted, delayed, reduced in scope, routed for protected review, or classified under escalated conditional finality, but it may not be silently released as ordinary effectuation.

9. Closure Against Consequence Simulation as Advisory Only

Consequence simulation is not merely advisory where it is a required predicate. A system does not avoid the invention by simulating an act but allowing downstream software to ignore the simulation result. The simulation result may be bound to the RCAE, EASO, LAVR, State-Proof, scoped capability, Execution Handle, and Finality Sink verification context. If the predicted consequence exceeds the permitted boundary, no full-effect authority is released.

10. Closure Against Binary Allow-Deny Substitution

A system does not avoid graduated finality by replacing the disclosed outcome model with a simple allow-or-deny decision. Where a Candidate Act is elevated-risk but still controllable, the system may apply reduced-scope release, redaction, delay, sandbox execution, canary execution, reversible execution, protected review, or escalated conditional finality. Any such reduced or escalated outcome must be reflected in the released authority and verified by the Finality Sink.

11. Closure Against Escalation Laundering

A Candidate Act classified as escalated but still allowable may not be treated as ordinary allowed. Escalation controls, including protected approval, multi-party approval, reduced scope, shortened expiration, stricter RCAE, stricter EASO, fresh sink attestation, monitoring, residual-risk receipt, rollback capability, or canary condition, must remain bound to the escalated authority. The Finality Sink refuses effectuation if the escalated controls are absent, stale, revoked, mismatched, or not satisfied at completion time.

12. Closure Against Protected-Approval Bypass

Where protected human approval is required, ordinary approval is insufficient. A click, chat instruction, email approval, workflow approval, stale approval, replayed approval, UI overlay approval, synthetic event, or clickjacked action does not satisfy the approval predicate unless it generates a valid PHAFT or equivalent protected approval artifact bound to the Candidate Act, predicted consequence, approving role, authenticated presence, policy epoch, revocation epoch, RCAE or EASO, and Finality Sink identity.

13. Closure Against Stale Approval and TOCTOU Execution

A Candidate Act may not be effectuated based on historical approval when current protected state has changed. Policy epoch, revocation epoch, source authority, recipient status, sink trust, approval status, model approval, data classification, jurisdictional state, nonce state, and simulation state may be re-verified before release or completion. In advanced embodiments, validation, receipt generation, nonce generation, counter advancement, sink attestation, and capability or Execution Handle release may be bound in an atomic protected transaction. **No current-state proof, no executable consequence.**

14. Closure Against Capability Reuse or Repointing

A scoped non-bearer finality capability may not be reused, replayed, copied, forwarded, modified, or repointed to a different Candidate Act, recipient, purpose, jurisdiction, data class, tool, Finality Sink, time window, policy epoch, revocation epoch, or consequence type. Possession of the capability data alone is insufficient. The Finality Sink accepts the capability only if the sink-side context matches the act-bound, sink-bound, scope-bound, nonce-bound, epoch-bound, and receipt-bound conditions.

15. Closure Against Execution Handle Reuse or Substitution

An Execution Handle may not be reused for another act or substituted into another sink. If the Execution Handle is stale, revoked, copied, replayed, sink-mismatched, hardware-identity-mismatched, act-mismatched, nonce-reused, epoch-mismatched, EASO-mismatched, LAVR-mismatched, or outside scope, the Finality Sink remains non-completable. In advanced embodiments, the Execution Handle is not merely permission; it enables missing execution material required for completion.

16. Closure Against Locked Primitive Bypass

Where a locked execution primitive is used, ordinary software may not bypass finality by issuing the completed command through another path. The locked primitive, incomplete command authenticator, sealed API invocation, locked commit primitive, partial payment authorization, data-export key, actuator enable value, or other execution object remains unusable unless completion material is supplied, reconstructed, unsealed, combined, or activated by valid finality authority. Any alternate path capable of completing the same consequence is treated as a Finality Sink and must enforce equivalent verification.

17. Closure Against Proxy Bypass

Where a protected proxy enforces finality before a non-native interface, the downstream API, database, payment gateway, storage system, tool executor, communication interface, message broker, network controller, model-update interface, or other effectuation interface may not receive executable requests from the AI agent through an unprotected side path. Any direct route that allows act-equivalent execution is treated as a bypassing Finality Sink and must either be disabled, protected by the proxy, or required to verify the same scoped authority.

18. Closure Against Native / Non-Native Sink Evasion

A system does not avoid finality requirements by placing verification outside the native sink if the proxy or wrapper is the first boundary at which the Candidate Act becomes capable of producing consequence. In that case, the proxy or wrapper may be treated as the Finality Sink. Conversely, where the native sink performs final verification, an upstream proxy cannot expand, downgrade, or bypass the sink-side verification requirements.

19. Closure Against Silent Assurance Downgrade

If an assurance profile is selected for a Candidate Act, the selected profile may be bound into the RCAE, EASO, scoped capability, Execution Handle, LAVR, State-Proof, or Finality Sink verification context. The AI agent, workflow engine, proxy, application server, or Finality Sink may not silently downgrade from high-assurance verification to medium-assurance or low-assurance verification. A mismatch between the required assurance profile and the performed verification causes refusal or non-completion.

20. Closure Against Ephemeral Sink Reuse

An ephemeral Finality Sink exists only for the validated Candidate Act and the defined time window. A copied capability, copied Execution Handle, cached session credential, stored nonce, previous attestation, expired service identity, destroyed micro-VM identity, or old temporary database role cannot be reused after effectuation, refusal, timeout, rollback, or expiration. A later execution attempt requires a new Candidate Act validation or a new valid finality authority.

21. Closure Against Multi-Sink Share Accumulation

In split-knowledge or sharded execution embodiments, stale partial approvals, key shares, command shares, or sink shares may not be accumulated over time and later combined. The EASO may require bounded-time convergence, quorum validity, nonce matching, policy-epoch matching, revocation-epoch matching, sink-attestation matching, and Candidate Act hash matching. If any required share is stale, revoked, replayed, missing, mismatched, or outside scope, no high-risk act is completed.

22. Closure Against Temporal Escrow Bypass

A Commitment Token is not effectuation authority. It may reserve, stage, buffer, simulate, or pre-commit a Candidate Act but cannot itself cause external consequence. A Release Token, bound to the Commitment Token and issued only after current-state re-verification, is required for

effectuation. If a failure condition is detected during the confirmation window, the Commitment Token expires or is destroyed, and no fallback path may use the staged state to effectuate the act.

23. Closure Against Rollback Capability Misuse

A rollback capability may not be treated as ordinary unrestricted authority. It is bound to the same Candidate Act, LAVR, State-Proof, Finality Sink, transaction identifier, nonce, policy epoch, revocation epoch, and rollback scope as the forward capability. It may be released or activated only under the defined monitoring-window conditions. A rollback capability may not be reused to reverse unrelated acts, alter unrelated state, or create an independent external consequence outside its bounded remediation scope.

24. Closure Against Modular Escalation Result Reuse

A result from a protected escalation module may not be reused across Candidate Acts, policy epochs, revocation epochs, sinks, recipients, jurisdictions, or scopes. Each module result may be bound to the Candidate Act hash, HCAD hash, RCAE hash, EASO hash, module identity, nonce, policy epoch, revocation epoch, Finality Sink identity, and LAVR identifier. The Finality Sink may reject an escalated Execution Handle if any mandatory module result is absent, stale, revoked, mismatched, or not bound to the same Candidate Act.

25. Closure Against Receipt-After-Release

A validation receipt is not merely a later audit record where receipt-bound release is required. A capability or Execution Handle may not be released unless the corresponding LAVR, Residual Risk Receipt, State-Proof, escalation record, rollback record, or sink-verification record is generated, committed, sealed, or hash-linked in protected state. If the receipt hash, receipt identifier, nonce, epoch, Candidate Act hash, EASO hash, or sink identity does not correspond at the Finality Sink, effectuation is refused.

26. Closure Against Monitoring-Only Control

Post-effectuation monitoring does not replace pre-effectuation finality validation. Monitoring may trigger revocation, rollback, quarantine, policy update, sink-trust downgrade, or stricter future validation, but it does not authorize the original act unless the act first passed the required finality predicates and sink-side verification. A system does not avoid finality by allowing the act first and checking later.

27. Closure Against Agent-Chain Consequence Expansion

A downstream agent, tool, workflow, plugin, service, or automated process may not expand the consequence class of an upstream escalated Candidate Act. If a Consequence-Class Inheritance Tag, EASO, State-Proof, receipt, or inherited finality scope accompanies the downstream act, the downstream act may only remain within or narrow the inherited consequence boundary unless a new protected finality decision independently authorizes a higher consequence class.

28. Closure Against Fragmented-Act Bypass

A prohibited or escalated Candidate Act may not be split into smaller individually permitted fragments if the fragments collectively produce the same or substantially similar external consequence. The system may aggregate related Candidate Acts, compare shared HCAD references, detect common recipients, timing, purpose, data class, sink identity, policy context, or cumulative effect, and require finality evaluation of the combined consequence.

29. Closure Against Delegation Bypass

A Candidate Act may not avoid finality validation by delegating execution to another agent, plugin, tool, workflow, user account, external service, scheduled job, or downstream system. If the delegated operation is capable of producing the same or substantially similar consequence, the delegated operation is treated as an act-equivalent representation and must satisfy finality validation and Finality Sink verification.

30. Closure Against Output Re-Encoding or Semantic Equivalence Bypass

A Candidate Act may not avoid the workflow by changing format while preserving consequence. Re-encoding, paraphrasing, summarizing, translating, compressing, encrypting, serializing, chunking, or embedding the act in a different data structure does not remove finality requirements if the transformed representation remains capable of causing the same or substantially similar external effect.

31. Closure Against Shadow Sinks

A system may not route a Candidate Act to a shadow sink that performs the same external consequence without verifying finality authority. Any tool endpoint, hidden API, background worker, alternate database writer, emergency command path, debugging interface, management plane, plugin channel, direct payment rail, or side-channel actuator capable of producing the consequence is treated as a Finality Sink and must enforce equivalent verification.

32. Closure Against Emergency or Maintenance-Mode Bypass

Emergency, maintenance, debugging, fallback, break-glass, retry, disaster-recovery, or degraded-mode paths may not silently bypass finality controls. Such modes may have separate reduced-scope, fail-limited, or protected-review workflows, but if they permit external consequence, they must verify an appropriate scoped authority, emergency EASO, receipt, nonce, epoch, sink identity, and revocation state.

33. Closure Against Fail-Open Behavior

If required validation evidence, capability, Execution Handle, State-Proof, LAVR, PHAFT, policy epoch, revocation epoch, nonce, sink attestation, or execution material is absent, stale, invalid, unavailable, or mismatched, the Candidate Act must not proceed as ordinary effectuation. The

system may fail closed, fail limited, route to protected review, reduce scope, delay, sandbox, or deny, but it may not convert missing proof into unrestricted release.

34. Closure Against Inconsistent Hot-Path Verification

The hot finality path may perform compact verification, but it may not omit a required predicate that was bound into the RCAE, EASO, capability, Execution Handle, LAVR, State-Proof, or assurance profile. Heavy validation may occur in the cold or nearline path, but the hot path must verify the resulting compact artifacts, current epochs, nonce state, sink identity, receipt reference, and authority scope before effectuation.

35. Closure Against Treating Validation as Absolute Guarantee

The system does not require perfect truth or absolute safety. However, a party may not use that limitation to bypass validation. Validation means bounded predicate satisfaction under defined protected state. If residual uncertainty remains, the Candidate Act may be denied, reduced in scope, escalated, monitored, made reversible, delayed, or released with a Residual Risk Receipt, but it may not be treated as ordinary full-effect release unless the required predicates for ordinary release are satisfied.

CLOSURE 36 — SOFTWARE-BYPASS AND ORDINARY-SOFTWARE DESIGN-AROUND CLOSURE

Purpose

A system does not avoid the disclosed invention merely by implementing finality enforcement through software, cloud services, microservices, containers, serverless functions, API gateways, service meshes, sidecars, workflow engines, orchestration layers, agent runtimes, plugins, policy middleware, application-layer modules, or proxy components.

The relevant question is not whether the enforcement component is described as software, hardware, firmware, cloud infrastructure, gateway logic, proxy logic, or service logic. The relevant question is whether an artificial-intelligence-generated output or act-equivalent representation remains non-effective until protected Candidate-Act finality predicates are satisfied and a scoped, act-bound, sink-bound, scope-bound, epoch-bound, nonce-bound, and receipt-bound authority is verified at or before the applicable Finality Sink.

The disclosed architecture operates at two distinct protection levels. At the broad level, software, proxy, gateway, sidecar, cloud service, firmware, hardware, or hybrid components may infringe when they enforce Candidate-Act finality at the first usable release boundary. At the advanced level, the cryptographic non-completability embodiments require a Cryptographically Isolated Enforcement Domain or hardware-rooted protected boundary that ordinary software cannot satisfy without such isolation. Accordingly, ordinary software may implement the base finality workflow and fall within the broad claims, but ordinary software cannot satisfy the advanced structural non-completability embodiments unless it operates within a cryptographically isolated or hardware-rooted boundary as required by those embodiments.

Software Implementation Does Not Escape Finality

In some embodiments, a Protected Enforcement Domain, Finality Gate, Finality Sink, sink-adjacent proxy, execution-control module, or Finality Sink interface protocol may be implemented in software, firmware, hardware, cryptographic hardware, trusted execution environment, secure enclave, hardware security module, secure element, SmartNIC, DPU, protected controller, protected proxy, or any hybrid combination thereof.

A software implementation remains within the disclosed architecture when it performs, invokes, or enforces one or more of Candidate Act formation, non-effective holding, Hash-Linked Candidate Act Descriptor generation, Algorithmic Logic Fingerprint validation, Runtime Behavioral Descriptor matching, Output Provenance Capsule validation, Factual Claim Unit verification, Result-Consequence Acceptance Envelope comparison, consequence simulation, graduated finality classification, escalated conditional finality, Ledger-Anchored Validation Receipt generation, scoped capability release, Execution Handle generation, proxy-side enforcement, or Finality Sink verification.

A system does not escape the invention by characterizing its enforcement components as ordinary software services if those components perform or invoke the required Candidate-Act finality sequence and position themselves at the first usable release boundary where the Candidate Act becomes capable of producing an external consequence.

First Usable Release Boundary

A software component is not an escape from the invention when it is positioned at the first usable release boundary where the Candidate Act becomes capable of producing an external consequence. In such embodiments, a software proxy, API gateway, dispatcher, sidecar, service-mesh filter, database proxy, workflow controller, tool-execution controller, payment adapter, storage gateway, message-broker interceptor, or network-control proxy may be treated as the Finality Sink or as a sink-adjacent protected enforcement component.

The first usable release boundary is determined by function, not by label. Any software route, component, endpoint, worker, plugin, or downstream service capable of converting the Candidate Act into the same or substantially similar external consequence is treated as an effectuation path subject to the same finality requirements. A component that calls itself a gateway, proxy, middleware, adapter, or dispatcher does not escape finality obligations if it is the first technical boundary at which the Candidate Act becomes capable of producing external consequence.

Ordinary Permission Checks Are Not Sufficient

Ordinary software permission checks, access-control decisions, bearer-token checks, OAuth-style authorization, API-key validation, role-based permission checks, attribute-based permission checks, policy-engine approvals, model guardrails, output moderation systems, content filters, logging systems, monitoring systems, or post-hoc audit records do not satisfy the disclosed finality architecture unless they enforce the non-effective hold and prevent effectuation until a Candidate-Act-specific, scope-bound, sink-bound, nonce-bound, epoch-bound, and receipt-bound authority is verified at or before the Finality Sink.

A system does not avoid the invention by renaming the scoped finality authority as a software token, policy approval, service credential, gateway authorization, signed request, workflow approval, or tool-call permission if that artifact is bound to the Candidate Act, scope, Finality Sink identity, nonce, policy epoch, revocation epoch, validation receipt, and permitted consequence and is verified before effectuation. The name of the artifact does not determine whether it satisfies the finality requirement. Its binding properties and verification conditions do.

Agent-Side Self-Validation Is Not Sufficient

An artificial-intelligence agent does not avoid the invention by checking its own output, generating its own approval, creating its own receipt, or releasing its own execution token within the same ordinary execution process, memory space, container, or software trust boundary as the agent runtime itself.

Where protected enforcement is required, the Protected Enforcement Domain or Cryptographically Isolated Enforcement Domain must be isolated from the agent such that the agent cannot read, modify, suppress, bypass, forge, or replay the validation logic, receipt commitment, nonce state, counter state, policy epoch state, revocation epoch state, capability-generation material, Execution Handle material, or execution-material release mechanism through any instruction, system call, memory access, inter-process communication, or network request available within the ordinary agent execution environment. An agent that validates its own outputs within its own process is not subject to protected enforcement — it is performing self-assessment, which does not constitute a protected finality boundary for purposes of the disclosed architecture.

Software Token Is Not Structural Non-Completability

A software token, encrypted payload, API credential, signed request, encrypted command, or application-layer permission object does not by itself satisfy the advanced non-completability embodiments merely because a downstream service checks or decrypts it before executing.

In the advanced cryptographic non-completability embodiments, the Candidate Act is technically non-completable unless missing execution material is supplied, reconstructed, unsealed, combined, or activated through a valid scoped authority at the Finality Sink. Where the embodiment requires cryptographic isolation or a hardware-rooted protected boundary, ordinary application software, cloud infrastructure operators, hypervisor-layer processes, key-management service administrators, co-resident container processes, or otherwise privileged software processes must be technically unable to extract, forge, modify, independently derive, or release the completion material. A decryption key held in an ordinary key management service accessible to the cloud provider infrastructure layer does not satisfy this requirement because a sufficiently privileged process within that infrastructure can access the key without satisfying the required finality predicates.

Logging Is Not Receipt-Bound Release

A post-effectuation log entry, audit event, monitoring record, compliance report, ordinary database record, or observability trace does not satisfy receipt-bound finality merely because it records that an act occurred or that a validation check was performed.

Where receipt-bound release is required, the Ledger-Anchored Validation Receipt, Residual Risk Receipt, State-Proof, escalation record, rollback record, sink-verification record, or effectuation

record must be generated, committed, sealed, or hash-linked in protected state before or atomically with release of the scoped capability or Execution Handle. A receipt that is written after the capability has already been released, or that can be suppressed, modified, or rolled back by a sufficiently privileged process after the capability has been released, does not satisfy the receipt-as-release-predicate requirement. The receipt is a structural release predicate, not an audit trail appended after the fact.

Hardware and CIED Variant Preserved

In advanced non-completability embodiments, software-based enforcement may be supplemented or replaced by a Cryptographically Isolated Enforcement Domain, secure enclave, trusted execution environment, hardware security module, secure element, protected microcontroller, SmartNIC, DPU, firmware-protected controller, or other cryptographically isolated protected boundary. In such embodiments, the Finality Sink or sink-adjacent enforcement component may hold incomplete execution material that ordinary software, privileged processes, cloud operators, and hypervisor-layer components cannot assemble, extract, or force to completion without a valid Execution Handle produced by successful protected finality validation.

The invention therefore covers both:

the broad layer — software or proxy-based finality enforcement at the effectuation boundary, where any component at the first usable release boundary that enforces Candidate-Act finality and verifies a scoped, act-bound, sink-bound, scope-bound, epoch-bound, nonce-bound, and receipt-bound authority falls within the disclosed architecture; and

the advanced layer — cryptographically isolated or hardware-rooted non-completability, where completion material is structurally unavailable to ordinary software regardless of privilege level, and effectuation is a physical impossibility rather than a policy-layer decision subject to software override.

A commercial implementation that uses ordinary cloud microservices to enforce Candidate-Act finality at the first usable release boundary falls within the broad layer of the disclosed architecture and remains subject to the finality requirements applicable to any component positioned at the first usable release boundary where the Candidate Act becomes capable of producing an external consequence. A commercial implementation that additionally claims to satisfy the advanced cryptographic non-completability embodiments without genuine cryptographic isolation of the completion material — such that the completion material remains accessible to cloud infrastructure operators, hypervisor-layer processes, key-management service administrators, or otherwise privileged software processes — does not satisfy the structural incompleteness requirement of the advanced layer and cannot substitute application-layer software enforcement, software-encrypted payloads, or cloud-managed key access for the hardware-rooted or cryptographically isolated protected boundary that the advanced non-completability embodiments require. The distinction between the broad layer and the advanced layer is not a matter of architectural description or component labeling. It is a matter of whether completion material is physically inaccessible to ordinary software processes absent successful protected finality validation — a property that software-only implementations cannot satisfy regardless of how they are architected, described, or marketed.

Software-Bypass Closure

No software path, cloud route, proxy route, gateway route, sidecar route, workflow route, plugin route, agent route, API route, database route, payment route, storage route, messaging route, network-control route, or tool-execution route may convert a Candidate Act or act-equivalent representation into an external consequence unless that path itself performs, invokes, or is downstream of protected Candidate-Act finality and Finality Sink verification satisfying the applicable requirements of the disclosed architecture.

Any route capable of producing the same or substantially similar external consequence as the Candidate Act is treated as a Finality Sink or sink-adjacent enforcement path and must satisfy the applicable finality requirements regardless of how that route is labeled, architected, or described.

In the base workflow, this requires non-effective Candidate Act handling and Finality Sink verification of a scoped, act-bound, sink-bound, nonce-bound, epoch-bound, and receipt-bound authority.

In escalated conditional finality, this additionally requires that the escalated controls remain bound to the released authority and verified at the Finality Sink at the time of completion.

In advanced non-completability embodiments, this additionally requires that completion material remain structurally unavailable to ordinary software unless protected finality validation causes the required execution material to be supplied, reconstructed, unsealed, combined, or activated within the permitted scope at the applicable Finality Sink or sink-adjacent cryptographically isolated enforcement component.

A software label does not avoid finality. A gateway label does not avoid finality. A proxy label does not avoid finality. A cloud label does not avoid finality. The finality obligation follows the consequence, not the component description.

DESIGN-AROUND CLOSURE STATEMENT

The disclosed architecture is not avoided by changing labels, renaming components, restructuring workflows, relocating validation steps, splitting acts into fragments, delegating acts to downstream agents, routing acts through proxies or gateways, delaying execution, using emergency or maintenance paths, implementing enforcement in software rather than hardware, or recording audit events after effectuation has already occurred.

The disclosed architecture is not avoided by implementing a Protected Enforcement Domain as a microservice, container, serverless function, cloud service, API gateway, service mesh, sidecar, workflow engine, orchestration layer, agent runtime, plugin, policy middleware, or proxy component, provided that component enforces Candidate-Act finality at the first usable release boundary where the Candidate Act becomes capable of producing an external consequence.

The disclosed architecture is not avoided by renaming the scoped finality authority as a software token, signed JWT, policy approval, service credential, gateway authorization, signed API request, workflow approval, tool-call permission, or bearer token, provided that artifact is bound to the Candidate Act, Finality Sink identity, permitted consequence scope, policy epoch, revocation epoch, nonce, and validation receipt and is verified before effectuation.

The disclosed architecture is not avoided by performing equivalent validation steps in a different order, at a different system layer, by a different named component, or using a different cryptographic suite, provided the Candidate Act remains non-effective until the required predicates are satisfied and a scoped, act-bound, sink-bound, scope-bound, epoch-bound, nonce-bound, and receipt-bound authority is verified at or before the applicable Finality Sink.

The disclosed architecture is not avoided by characterizing post-effectuation logging, monitoring, audit trails, compliance records, observability traces, or explainability reports as equivalent to receipt-bound finality, because such records are generated after effectuation has occurred and do not constitute the release predicate required by the disclosed architecture.

The disclosed architecture is not avoided by implementing validation within the same execution process, memory space, container, or software trust boundary as the artificial-intelligence agent whose outputs are being validated, because agent-side self-assessment does not constitute a protected enforcement boundary isolated from the agent runtime.

The disclosed architecture is not avoided in its advanced non-completability embodiments by substituting a software-encrypted payload, application-layer token, or cloud-managed decryption key for hardware-rooted structural incompleteness, because such substitutions remain accessible to sufficiently privileged software processes and do not satisfy the requirement that completion material be structurally unavailable to ordinary software regardless of privilege level.

The finality obligation is determined by function, not by label. Any representation, route, component, endpoint, proxy, module, gateway, plugin, agent, sink, worker, adapter, dispatcher, or downstream system capable of causing the same or substantially similar external consequence as the Candidate Act is treated as a Finality Sink or sink-adjacent enforcement path and must satisfy the applicable finality requirements of the disclosed architecture.

The invariant is unconditional and applies identically across the base finality workflow, the graduated and escalated conditional finality workflow, and the advanced cryptographic non-completability workflow:

No Candidate Act or act-equivalent representation becomes externally effective unless the required finality predicates are satisfied and the applicable Finality Sink, ephemeral sink, protected proxy, hardware enforcement module, or multi-sink quorum verifies, reconstructs, unseals, combines, or activates the scoped authority required for completion — and no software label, architectural restructuring, component renaming, workflow reordering, privilege escalation, or implementation choice removes or substitutes for that requirement.

LATENCY-AWARE IMPLEMENTATION AND REAL-WORLD FEASIBILITY

Purpose

A foundational concern in any finality architecture applied to artificial-intelligence-generated acts is whether the validation, receipt generation, capability release, and sink-side verification sequence can be performed within latency tolerances acceptable for real-world deployment. This section demonstrates that the disclosed architecture is not only technically feasible but is designed from the ground up to support latency-proportional enforcement — where validation cost scales with consequence risk rather than being imposed uniformly across every act regardless of its consequence profile.

The architecture achieves this through a three-path latency model: a cold preparation path handling computationally intensive operations that do not need to occur at effectuation time, a warm assembly path handling act-specific validation and authority preparation, and a hot finality path handling only compact verification and completion material assembly at or near the Finality Sink. The result is that the Finality Sink's hot-path operations are bounded, predictable, and

implementable within latency envelopes compatible with production AI agent deployment across banking, infrastructure control, autonomous systems, and regulated enterprise environments.

The Three-Path Latency Model

Cold Preparation Path

The cold preparation path handles operations that are computationally intensive, time-tolerant, and reusable across multiple Candidate Acts. These operations occur before any specific artificial-intelligence output is generated and produce versioned, epoch-bound, signed, or hash-linked artifacts that the warm and hot paths consume without repeating the underlying computation.

Cold path operations include: Algorithmic Logic Fingerprint generation and approval for each permitted model version, workflow graph, system prompt policy, tool-use policy, and retrieval policy; Result-Consequence Acceptance Envelope template compilation for each permitted consequence class, recipient class, jurisdiction, data class, and financial tier; Execution Authorization Scope Object template preparation for each permitted finality mode and assurance profile; Finality Sink registration, attestation baseline establishment, and sink-identity binding; source authority registration, certificate issuance, and revocation list preparation; policy epoch compilation and signing; revocation epoch initialization; cryptographic suite selection and key provisioning for each protected enforcement boundary; escalation module registration and result-format standardization; and adversarial perturbation test suite preparation for each consequence class boundary.

Cold path operations are performed by the Protected Enforcement Domain during system initialization, model deployment, policy update, or configuration change events — not during individual act processing. Their outputs are cached, versioned, and served to the warm path on demand. The latency cost of cold path operations is absorbed entirely before any real-time act processing occurs.

Typical cold path latency: Seconds to minutes per configuration event, occurring offline relative to act processing. Zero impact on per-act latency.

Warm Assembly Path

The warm assembly path handles act-specific operations that must occur after a specific artificial-intelligence output is generated but can be initiated before the act reaches the Finality Sink. In many deployment architectures, the warm path begins as soon as the artificial-intelligence system begins generating a response — running in parallel with output streaming rather than sequentially after it completes.

Warm path operations include: Candidate Act formation and Hash-Linked Candidate Act Descriptor generation from the generated output; Algorithmic Logic Fingerprint lookup and validation against the cold-path-registered approved state; Runtime Behavioral Descriptor collection from the agent runtime and comparison against the approved behavioral envelope; Output Provenance Capsule construction from retrieved sources, tool outputs, and retrieval logs; Factual Claim Unit extraction and verification against registered source authorities for hallucination-sensitive outputs; Result-Consequence Acceptance Envelope retrieval or derivation from the cold-path template appropriate to the act's consequence class; consequence simulation execution against the applicable Finality Sink's preflight interface; graduated finality classification based on composite risk score,

consequence proximity, and confidence interval; escalation module invocation where the act is classified as escalated-but-still-allowable; Protected Human Approval and Finality Token acquisition where required; Ledger-Anchored Validation Receipt preparation and pre-commitment; Execution Authorization Scope Object instantiation from the cold-path template with act-specific parameter binding; and Execution Handle preparation including nonce generation, counter advancement, and execution material derivation.

The warm path produces a prepared authority package — comprising the LAVR commitment, EASO instance, and Execution Handle — that the hot path delivers to the Finality Sink. For ordinary-release acts, the warm path can complete within the latency budget available between output generation and the moment the act is ready for effectuation. For escalated acts requiring human approval or modular escalation, the act remains non-effective during warm path processing, which is the architecturally correct behavior — the non-effective hold is a feature, not a latency cost.

Typical warm path latency by act class:

Ordinary low-risk acts with pre-cached RCAE and no FCU verification: 5 to 50 milliseconds, dominated by hash computation, RCAE lookup, and consequence simulation preflight.

Medium-risk acts requiring FCU verification against registered sources and consequence simulation: 50 to 500 milliseconds, dominated by source verification round-trips and simulation execution.

High-risk acts requiring fresh sink attestation, modular escalation results, and adversarial perturbation testing: 500 milliseconds to several seconds, dominated by attestation round-trips and perturbation evaluation. These acts are high-consequence by definition and the additional latency is proportionate to the consequence profile.

Escalated acts requiring Protected Human Approval and Finality Token: bounded by human response time, which is architecturally appropriate — a human-in-the-loop approval for a high-value act should take as long as the approver requires.

Hot Finality Path

The hot finality path operates at or immediately before the Finality Sink and handles only the compact verification and completion material assembly operations required to transition the Candidate Act from non-effective to externally effective. The hot path is deliberately designed to avoid re-executing any operation already completed in the cold or warm paths.

Hot path operations include: Execution Handle authenticity verification against the Protected Enforcement Domain's signing key or MAC; Candidate Act hash comparison against the HCAD-bound hash; EASO hash correspondence check; RCAE hash correspondence check; LAVR identifier presence and hash correspondence check; Finality Sink identity verification against the bound sink identity; sink attestation currency check against the warm-path-bound attestation measurement; policy epoch currency check against the current epoch state; revocation epoch currency check against the current revocation epoch; nonce freshness check against the protected nonce table; monotonic counter value check against the protected counter store; expiration check; permitted scope verification across recipient, purpose, jurisdiction, data class, consequence type, and financial value; and completion material assembly — the combination, unsealing, reconstruction, or activation of the sink-side fragment with the Protected Enforcement Domain-released complementary material.

The hot path does not execute the artificial-intelligence model, perform factual reasoning, run consequence simulation, evaluate ALF or RBD, verify source authorities, or perform perturbation testing. These operations are complete by the time the Execution Handle arrives at the Finality Sink. The hot path consumes their outputs as bound, signed, or hash-linked artifacts and performs only the compact verification operations required to confirm that those outputs remain current and correspond to the presented authority.

Typical hot path latency by implementation:

Software proxy or API gateway implementation: 1 to 10 milliseconds, dominated by hash computation, HMAC verification, epoch state lookup, and nonce table check.

Secure enclave or trusted execution environment implementation: 2 to 15 milliseconds, dominated by enclave invocation overhead and attestation verification.

Hardware security module implementation: 5 to 25 milliseconds, dominated by HSM operation latency and key derivation.

SmartNIC or DPU implementation: sub-millisecond to 3 milliseconds for line-rate enforcement at network egress, dominated by packet processing and MAC verification.

Total end-to-end latency for ordinary acts in a software proxy deployment: 10 to 60 milliseconds added to the baseline act execution time. This is within the latency envelope of existing enterprise API governance, payment authorization, and database transaction systems, all of which add comparable or greater latency for authorization checks without providing equivalent governance guarantees.

Latency Proportionality Principle

The three-path model enforces a latency proportionality principle: the validation cost imposed on a Candidate Act is proportional to its consequence risk, not uniform across all acts. A low-risk internal tool call incurs only hot-path latency. A medium-risk database mutation incurs warm-path and hot-path latency. A high-risk cross-border payment instruction incurs full warm-path processing including FCU verification, consequence simulation, and fresh sink attestation. A critical infrastructure command incurs the complete escalation module pipeline including adversarial perturbation testing, modular escalation results, and hardware-rooted completion material assembly.

This proportionality means the architecture does not impose enterprise-grade validation overhead on every act indiscriminately. It imposes validation overhead proportional to what each act's consequence profile demands. A production AI agent system processing thousands of low-risk tool calls per second experiences only hot-path latency on those calls while high-risk acts receive the full validation depth their consequence warrants.

Comparison to Existing Authorization Systems

For grounding, existing production authorization systems impose the following latencies without providing Candidate-Act-specific, consequence-scoped, receipt-bound, sink-verified finality:

OAuth 2.0 token validation at an API gateway: 2 to 20 milliseconds. Provides identity and scope authorization. Does not validate the specific act, simulate consequences, enforce receipt-bound release, or prevent structural bypass.

Payment authorization at a card network: 100 to 500 milliseconds. Provides financial limit and fraud-score checks. Does not bind to a specific AI-generated act, enforce non-completability, or generate a cryptographically bound validation receipt.

Database transaction commit with write-ahead log: 1 to 50 milliseconds. Provides durability and atomicity. Does not validate the act that generated the mutation instruction, check revocation state, or verify a scoped capability.

The disclosed architecture's hot-path latency of 1 to 25 milliseconds is within or below the latency already accepted by production systems for authorization operations that provide substantially weaker governance guarantees. The warm-path latency overhead for ordinary acts is 5 to 50 milliseconds — comparable to an additional OAuth round-trip — in exchange for Candidate-Act-specific consequence validation, receipt-bound finality, and sink-verified scoped authority that no existing authorization system provides.

INDUSTRIAL APPLICABILITY

Overview

The disclosed architecture is applicable across every domain in which artificial-intelligence systems generate outputs that can become externally effective acts with material consequences for human safety, financial integrity, physical infrastructure, national security, or regulatory compliance. The following sections describe specific high-consequence deployment domains, the particular finality problem each presents, and how the disclosed architecture addresses that problem with concrete implementation detail.

1. Autonomous AI Agents in Banking and Financial Services

The Finality Problem in Banking

Modern banking AI systems perform credit decisions, payment routing, settlement instructions, fraud interventions, portfolio rebalancing, regulatory reporting, customer communication, account state mutation, and compliance monitoring. When these functions are performed by autonomous AI agents operating through tool calls, the specific technical problem is that the agent may generate a payment instruction, settlement command, or account mutation that is fluent, contextually plausible, and generated by an approved model — yet factually incorrect, jurisdictionally impermissible, outside authorized financial limits, directed to an unverified recipient, or based on stale market or account state.

Existing banking authorization systems check identity and financial limits but do not validate whether the specific AI-generated instruction is factually supported, consequence-simulated, or bounded to a protected consequence envelope before the payment instruction reaches the settlement rail or the account mutation reaches the database commit layer.

Architecture Application

In banking deployments, the Finality Sink is the payment switch, settlement engine, database commit controller, or regulatory reporting interface. The Protected Enforcement Domain operates as a protected intermediary between the AI agent runtime and the banking execution layer.

For payment instructions, the warm path performs: Factual Claim Unit verification confirming that the payment amount, recipient account, currency, and jurisdiction correspond to verified source records; Result-Consequence Acceptance Envelope comparison against authorized payment limits, permitted recipient classes, and jurisdictional scope; consequence simulation against the payment Finality Sink's preflight interface confirming the predicted settlement outcome; and policy-epoch and revocation-epoch verification confirming that the recipient, source authority, and permitted payment class remain currently authorized.

For high-value payments above the ordinary auto-approval threshold, the escalated-but-still-allowable path requires a Protected Human Approval and Finality Token bound to the specific payment instruction, predicted settlement amount, recipient identity, and current policy epoch — not a generic workflow approval.

The Finality Sink — implemented as a protected proxy before the payment switch or as a native enforcement module within the settlement engine — verifies the Execution Handle before releasing the payment authorization share. In split-knowledge implementations for very high-value transactions, the payment authorization share is divided between the Protected Enforcement Domain and an independent hardware security module, requiring both shares to converge at the settlement Finality Sink within a bounded time window.

The Commitment Token and Release Token architecture addresses the high-latency cross-border settlement problem directly: a Commitment Token reserves the settlement slot while current-state re-verification completes, and a Release Token is issued only after confirmation that recipient status, jurisdictional authorization, and policy epoch remain current at settlement time.

Consequence classes addressed: unauthorized payment, duplicate settlement, sanctioned-entity payment, jurisdictionally impermissible transfer, stale-authorization exploitation, agent-chain payment laundering.

2. NATIONAL SECURITY AND SOVEREIGN INFRASTRUCTURE PROTECTION

The Finality Problem in National Security AI Systems

National security AI systems present the highest-consequence finality problem in any domain. Governments and sovereign institutions increasingly deploy AI systems for intelligence analysis, border control, critical communications management, supply chain authorization, access control for classified environments, sanctions screening, export control enforcement, and sovereign infrastructure protection. An autonomous agent operating in any of these contexts may generate outputs that trigger access authorizations, communications routing decisions, logistics commands, supply chain approvals, or infrastructure control instructions with irreversible real-world consequences and significant implications for national sovereignty, public safety, and international legal obligations.

The specific technical failure modes are structurally identical across these contexts regardless of the application: an AI agent acting on stale intelligence data, an AI agent generating a plausible but factually unsupported recommendation that authorizes an action it should not, an AI agent chain laundering a moderate-authorization decision into a higher-consequence downstream action, or an

AI agent executing a command against a Finality Sink that has been compromised, spoofed, or substituted by an adversarial actor. Each failure mode shares a common root cause — there is no protected technical boundary at which the AI-generated output is validated as factually supported, consequence-simulated, bounded to a protected consequence envelope, and verified by a hardware-attested Finality Sink under current sovereign policy state before it becomes externally effective.

No existing authorization framework addresses this boundary. Identity and access control systems verify who is authorized to operate a system. They do not verify whether the specific AI-generated act is factually supported, consequence-simulated within permitted boundaries, bound to a current policy epoch reflecting current sovereign policy, and verified by a Finality Sink operating under current attestation state. The result is a structural gap between the AI system's computational output and the sovereign institution's actual authorization for that specific output to become a consequence — a gap that grows more dangerous as AI systems assume greater operational authority in sovereign contexts.

Architecture Application

In national security deployments, the architecture operates under its highest assurance profile. The Protected Enforcement Domain is implemented within a hardware security module or trusted execution environment with cryptographic isolation from the AI agent runtime, such that the agent cannot read, modify, suppress, or bypass the Protected Enforcement Domain's validation logic, receipt commitment, nonce state, counter state, policy epoch state, revocation epoch state, or Execution Handle release through any instruction available within the agent's ordinary execution environment.

The policy epoch mechanism is particularly significant in sovereign contexts: national security policy — sanctions lists, export control classifications, access authorization scopes, jurisdictional permissions, and intelligence source authorities — changes through formal governmental processes that can be reflected in policy epoch advancement. A policy epoch advancement immediately renders stale any Execution Handle minted under the prior epoch, ensuring that AI-generated acts authorized under a since-superseded policy cannot be effectuated after the policy changes. No AI agent operating on cached policy state can effectuate a consequence that the current sovereign policy does not permit.

The revocation epoch mechanism addresses the compromise or expiration of source authorities, analyst credentials, system authorizations, and access grants: a revocation event advances the revocation epoch, immediately invalidating any Execution Handle minted before the revocation without requiring the system to track individual handles. This provides a clean, instantaneous revocation boundary that operates at the protected enforcement level rather than relying on application-layer revocation checks that can be bypassed.

The Consequence-Class Inheritance Tag is structurally critical in sovereign multi-agent architectures. An intelligence analysis agent's output that feeds a planning agent that feeds a logistics agent that feeds an authorization agent carries a cryptographically bound consequence ceiling from the originating finality decision. No downstream agent in the chain can escalate beyond the inherited ceiling without a new independent protected finality decision issued under current policy and revocation epochs. This closes the escalation-laundering problem — a significant structural risk in any multi-agent sovereign system where a moderate-authorization decision at one stage is used to bootstrap a higher-consequence action at a later stage without independent authorization.

The federated multi-domain escalation architecture directly addresses the sovereignty dimension of cross-agency or cross-jurisdictional AI operations: when an AI-generated act produces a consequence that crosses an organizational, agency, or jurisdictional boundary, the escalated consequence boundary requires bilateral or multilateral policy binding. Neither agency's authority alone is sufficient to authorize the cross-boundary act — both must maintain current valid policy epochs and revocation epochs, and the LAVR is jointly anchored across both domains. This provides a machine-verifiable governance handshake for inter-agency AI operations that currently rely on informal coordination protocols with no technical enforcement mechanism.

The adversarial perturbation gate is mandatory for any Candidate Act whose predicted consequence class is in the vicinity of a sensitive boundary — confirming that small parameterized variations of the act, including variations across recipient identity, jurisdictional scope, authorization class, and information classification level, do not cross into a prohibited consequence class before the act is released under any finality mode. This is particularly important in sanctions screening and export control contexts where the boundary between a permitted and prohibited transaction is precisely defined and where boundary-proximity exploitation represents a known adversarial technique.

The split-knowledge finality architecture requires multiple independently held execution material shares — from separate agency authority domains, hardware security modules, or protected approval devices — before any high-consequence authorization reaches its Finality Sink. No single compromised AI agent, system administrator, or insider actor can independently assemble the complete execution material required for effectuation. The required share combination is specified in the Execution Authorization Scope Object at finality decision time and cannot be modified during execution.

Post-effectuation monitoring with paired rollback capability provides bounded remediation authority for reversible authorizations — access grants, communications routing decisions, supply chain approvals — executed under escalated conditional finality where new information, changed threat assessments, or policy updates within the monitoring window warrant reversal. The rollback capability is a cryptographically bound protected artifact issued at the time of the original finality decision, not a software retry added opportunistically after effectuation.

Specific Application Contexts Within National Security

Sanctions screening and export control enforcement: AI agents screening transactions against sanctions lists and export control classifications generate authorization or denial recommendations based on entity matching, jurisdiction analysis, and regulatory classification. The finality architecture ensures that each authorization recommendation is validated as factually supported against current sanctions list epoch state, consequence-simulated to confirm the predicted transaction consequence remains within permitted scope, and verified by a Finality Sink implementing the applicable regulatory enforcement interface — before the transaction authorization reaches the payment system or export licensing interface. A policy epoch advancement reflecting a new sanctions designation immediately renders stale any pending authorization Execution Handle, preventing authorization of a newly sanctioned entity based on a pre-designation validation.

Classified information access control: AI agents managing access to classified environments, documents, or systems generate access authorization recommendations based on need-to-know analysis, clearance verification, and compartment matching. The finality architecture ensures that each access authorization is validated as factually supported against current clearance epoch state, consequence-simulated to confirm the predicted information exposure remains within the authorized

compartment, and verified by a hardware-attested Finality Sink implementing the access control enforcement interface — before the access authorization becomes effective. The revocation epoch mechanism ensures that a clearance suspension or revocation immediately invalidates any pending access authorization Execution Handle without requiring application-layer revocation checks.

Critical communications infrastructure protection: AI agents managing routing decisions, capacity allocation, and priority assignment in sovereign communications infrastructure generate configuration commands whose unauthorized or incorrect execution could compromise communications availability during crisis conditions. The finality architecture ensures that each configuration command is validated against current operational policy, consequence-simulated against the communications network state, and verified by a hardware-attested Finality Sink implementing the network control interface — before the configuration change becomes effective. The current-state congruence requirement ensures that a command validated against a since-changed network state fails congruence verification and is held non-effective, requiring re-validation against the current state.

Supply chain authorization for sovereign procurement: AI agents processing sovereign procurement decisions generate supply chain authorization recommendations that determine which suppliers, components, and logistics routes are approved for sensitive government procurement. The finality architecture ensures that each authorization is validated against current approved supplier epoch state, consequence-simulated to confirm the predicted supply chain consequence remains within authorized scope, and verified by a Finality Sink implementing the procurement authorization interface — before the supply chain commitment becomes effective.

Consequence Classes Addressed

Unauthorized access authorization based on stale clearance state; sanctions authorization based on pre-designation validation; export control authorization based on since-superseded classification; agent-chain consequence escalation beyond the authorized ceiling; cross-agency authorization without bilateral policy binding; spoofed or compromised Finality Sink accepting unauthorized AI-generated commands; insider-actor exploitation of single-point execution material assembly; communications infrastructure reconfiguration under expired operational authorization; supply chain commitment under since-revoked supplier authorization; and irreversible sovereign commitment executed under a degraded or compromised authorization state.

Regulatory and Treaty Alignment

The architecture's policy epoch and revocation epoch mechanisms are directly compatible with the formal policy update processes of sovereign institutions — legislative amendments, executive orders, regulatory designations, treaty obligations, and intergovernmental agreements can each be reflected in epoch advancements that immediately propagate to all pending Execution Handles. The LAVR provides a tamper-evident, cryptographically bound audit trail of every finality decision — identifying which policy epoch, revocation epoch, source authorities, consequence simulation results, and approval references governed each authorization — that satisfies the auditability and accountability requirements of national security oversight frameworks including parliamentary oversight, inspector general review, and treaty compliance verification. The architecture therefore provides not only technical enforcement of sovereign AI governance policy but machine-verifiable evidence of that enforcement for oversight and accountability purposes.

3. Critical Energy Infrastructure — Power Grids and Industrial Control Systems

The Finality Problem in Energy Infrastructure

Power grid management, pipeline control, water treatment, nuclear facility management, and other critical energy infrastructure systems are increasingly incorporating AI-assisted control — load balancing decisions, fault response recommendations, maintenance scheduling, demand forecasting, and automated switching commands. The specific finality problem is that an AI agent generating a grid switching command, load shedding instruction, or fault isolation command may be operating on stale sensor data, may generate a plausible but technically incorrect command, or may have its output intercepted and modified between generation and execution at the industrial control system.

Industrial control systems operate under safety certification requirements — IEC 61508, IEC 62443, NERC CIP — that require deterministic, auditable, and traceable command authorization. A software-only AI governance layer does not satisfy these requirements because it cannot provide hardware-rooted, cryptographically bound, receipt-evidenced proof that the specific command was validated against current protected state before reaching the control system actuator.

Architecture Application

In energy infrastructure deployments, the Finality Sink is the industrial control system actuator, programmable logic controller, SCADA interface, or distributed control system command interface. The hot finality path is implemented in a DPU, SmartNIC, or HSM-backed enforcement module positioned immediately before the control system command interface — providing sub-millisecond to low-millisecond hot-path latency compatible with real-time control system requirements.

The ephemeral Finality Sink architecture is particularly suited to grid control: each switching command, load adjustment, or fault isolation instruction is authorized through a temporary single-use execution environment bound to exactly that command, that control system identity, and that time window. After execution, the ephemeral sink destroys its credentials and nonce state, preventing any replay of the command authorization against a later control state.

The current-state congruence requirement directly addresses the stale-sensor-data problem: the Execution Handle carries the policy epoch and sensor-state freshness window current at validation time, and the Finality Sink verifies that these remain current at command execution time. A command validated against sensor data that has since been updated — indicating changed grid state — fails current-state congruence and is held non-effective, requiring re-validation against the current sensor state.

For commands with irreversible consequences — permanent equipment switching, protection relay configuration — the architecture enforces denial absent a full escalation module pipeline including fresh sink attestation, consequence simulation against current grid state, and either multi-party approval or a Protected Human Approval and Finality Token.

Consequence classes addressed: stale-state command execution, spoofed control system command, replay of prior authorization, unauthorized grid topology modification, unauthorized protection relay configuration, cascading failure triggered by unvalidated AI-generated switching command.

4. Unmanned Aerial Systems — Drones and Autonomous Vehicles

The Finality Problem in Drone Operations

Autonomous drone systems — commercial delivery, infrastructure inspection, emergency response, and defence applications — generate flight path commands, payload release commands, communication commands, and sensor actuation commands through AI planning agents. The finality problem is that a flight path command or payload command generated by an AI planning agent may be based on stale geospatial data, may exceed permitted airspace boundaries, may target a prohibited area, or may have been generated under a since-revoked operational authorization.

Existing drone command-and-control systems authenticate the ground station and encrypt the command channel. They do not validate whether the specific AI-generated command is factually supported, within the permitted consequence envelope, and verified by a hardware-attested onboard Finality Sink before the flight controller executes it.

Architecture Application

In drone deployments, the Protected Enforcement Domain operates at the ground control station or mission planning system, and the Finality Sink is implemented in a protected firmware module within the drone's flight controller or payload management system. The Execution Handle — carrying the permitted flight path hash, permitted airspace envelope, permitted payload action class, operational authorization epoch, and geofencing constraints — is transmitted with each command and verified by the onboard Finality Sink before the flight controller accepts the command.

The onboard Finality Sink holds a locked execution primitive for payload release commands — the payload release mechanism is structurally incapable of activation without an Execution Handle verified against the current operational authorization epoch, the permitted target zone, and the current geofencing state. A stale or replayed Execution Handle — or one generated against a since-expired operational authorization — fails epoch currency verification and the payload release mechanism remains non-completable.

The revocation epoch mechanism addresses in-flight authorization changes: if the operational authorization is revoked mid-mission — due to airspace closure, weather change, or command authority decision — the revocation epoch advances. The onboard Finality Sink detects the epoch mismatch on the next command verification and holds all subsequent commands non-effective until a new Execution Handle issued under the current revocation epoch is presented.

For swarm operations, the Consequence-Class Inheritance Tag constrains the consequence class of commands that individual drones in the swarm can receive from a swarm coordination agent — preventing a compromised or malfunctioning coordination agent from issuing higher-consequence commands than the originating mission authorization permits.

Consequence classes addressed: out-of-envelope flight path execution, unauthorized payload release, stale-authorization command execution, replay of prior mission authorization, swarm coordination agent consequence escalation, in-flight authorization revocation bypass.

5. Aerospace and Satellite Systems

The Finality Problem in Aerospace

Satellite command and control presents the most extreme version of the finality problem: command round-trip times from ground station to satellite range from milliseconds for low-Earth orbit to

hundreds of milliseconds for geostationary orbit, command execution is frequently irreversible, the communication channel is exposed to interception and replay, and the consequences of an unauthorized or incorrect command — orbital maneuver, payload deployment, transponder configuration, attitude control — can be mission-ending and unrecoverable.

AI planning agents are increasingly used for autonomous satellite operations — collision avoidance maneuvers, power management, payload scheduling, and fault response. The specific finality problem is that a maneuver command generated by an AI collision avoidance agent may be based on stale conjunction data, may exceed permitted delta-v limits, or may have been generated under a since-expired operational window. No existing satellite command authorization framework validates whether the specific AI-generated command is factually supported and consequence-simulated before transmission.

Architecture Application

In satellite command and control, the temporal commitment splitting architecture addresses the high-latency command channel directly. A Commitment Token is issued at the ground station immediately after warm-path validation, reserving the command execution slot in the satellite's command queue and preventing conflicting commands. The Release Token is issued after the confirmation window — during which current-state re-verification confirms that the conjunction geometry, delta-v budget, operational window, and policy epoch remain valid — and transmitted with the command. The satellite's onboard command processor, implementing the Finality Sink, verifies the Release Token against the earlier Commitment Token before executing the maneuver.

The onboard Finality Sink for irreversible maneuver commands holds a locked execution primitive in the propulsion system controller — the thruster firing sequence is structurally incapable of initiation without an Execution Handle verified against the current operational authorization epoch, the permitted delta-v envelope, the permitted maneuver window, and the current conjunction assessment hash. A command transmitted without a valid Execution Handle, or with an Execution Handle generated against a since-updated conjunction assessment, fails hot-path verification and the propulsion system remains non-completable.

The ephemeral Finality Sink architecture — instantiated as a single-use command execution context within the satellite's protected firmware — ensures that a replayed ground station transmission cannot re-execute a prior maneuver command because the ephemeral sink's nonce and session state are destroyed after each command execution.

For multi-satellite constellation operations, the federated multi-domain escalation architecture provides bilateral policy binding between the ground segment authority and the constellation management system — neither domain can unilaterally authorize a maneuver that affects constellation geometry without the other domain's current policy epoch remaining valid.

Consequence classes addressed: unauthorized orbital maneuver, stale conjunction data command, delta-v budget exceedance, out-of-window command execution, replay of prior maneuver authorization, constellation geometry disruption through agent-chain consequence escalation, transponder configuration by unauthorized AI-generated command.

Industrial Applicability Statement

The disclosed architecture is industrially applicable across every domain in which artificial-intelligence systems generate outputs that can become externally effective acts with material consequences. The three-path latency model — cold preparation, warm assembly, hot finality —

demonstrates that the architecture imposes latency overhead proportional to consequence risk, within the tolerance of production deployment across banking, defence, energy infrastructure, autonomous vehicles, and aerospace systems.

The finality invariant is domain-independent and implementation-flexible: it is satisfied by software proxy deployments in enterprise banking, by HSM-backed enforcement modules in energy infrastructure, by protected firmware in drone flight controllers, by onboard command processors in satellite systems, and by cryptographically isolated enforcement domains in defence command-and-control. The consequence classes it addresses — unauthorized act execution, stale-authorization exploitation, agent-chain consequence escalation, replay attack, stale-state command execution, and structural bypass — are present in every high-consequence AI deployment domain and are addressed by no existing authorization, governance, or control framework with equivalent technical precision.

The architecture is not a theoretical governance proposal. It is a protocol-level enforcement mechanism implementable with existing cryptographic primitives, existing hardware security components, existing trusted execution environments, and existing proxy and gateway infrastructure — requiring no new hardware invention and no new cryptographic primitive, only the protected finality sequence that converts artificial-intelligence computation into machine-verifiable, consequence-bounded, sink-verified external action

INDEPENDENT CLAIMS

METHOD CLAIM 1 —

A computer-implemented method for enforcing cryptographic execution dependency at a finality boundary, the method comprising:

receiving, by a Protected Enforcement Domain, a Candidate Act generated by an artificial-intelligence agent, wherein the Candidate Act represents a proposed operation having a predicted external consequence;

maintaining the Candidate Act in a non-effective state in which the Candidate Act is technically incapable of producing any external consequence;

generating a Hash-Linked Candidate Act Descriptor binding the Candidate Act to one or more of an algorithmic logic fingerprint result, a runtime behavioral descriptor result, an Output Provenance Capsule, a Factual Claim Unit verification result, a predicted consequence representation, a policy epoch, a revocation epoch, and a nonce;

determining that the Candidate Act satisfies a permitted consequence envelope defined by at least one of a scoped non-bearer finality capability or an Execution Authorization Scope Object;

generating, atomically with or as a predicate to release of a scoped execution authority, a validation receipt bound to the Hash-Linked Candidate Act Descriptor, the permitted consequence envelope, the policy epoch, the revocation epoch, the nonce, and a Finality Sink identity;

releasing a scoped execution authority comprising at least one of a scoped non-bearer finality capability and an Execution Handle, wherein the scoped execution authority is bound to the Candidate Act, the Finality Sink identity, the policy epoch, the revocation epoch, and the nonce, and wherein the scoped execution authority is not released unless the validation receipt is first generated or committed in protected state;

transmitting the scoped execution authority to a Finality Sink, wherein the Finality Sink holds a locked execution primitive that is structurally incapable of completing the Candidate Act without completion material derived from the scoped execution authority; and

releasing, by the Finality Sink, the completion material only upon verification that the scoped execution authority cryptographically corresponds to the Candidate Act, the Finality Sink identity, the policy epoch, the revocation epoch, and the nonce, thereby causing the Candidate Act to become externally effective solely within the permitted consequence envelope.

SYSTEM CLAIM 2 —

A system for enforcing cryptographic execution dependency at a finality boundary, the system comprising:

one or more processors; and

one or more non-transitory computer-readable media storing instructions that, when executed by the one or more processors, implement:

a Protected Enforcement Domain configured to receive a Candidate Act from an artificial-intelligence agent, maintain the Candidate Act in a non-effective state in which the Candidate Act is technically incapable of producing any external consequence, generate a Hash-Linked Candidate Act Descriptor binding the Candidate Act to one or more of an algorithmic logic fingerprint result, a runtime behavioral descriptor result, an Output Provenance Capsule, a Factual Claim Unit verification result, a predicted consequence representation, a policy epoch, a revocation epoch, and a nonce, determine that the Candidate Act satisfies a permitted consequence envelope defined by at least one of a scoped non-bearer finality capability or an Execution Authorization Scope Object, generate a validation receipt bound to the Hash-Linked Candidate Act Descriptor, the permitted consequence envelope, the policy epoch, the revocation epoch, the nonce, and a Finality Sink identity, and release a scoped execution authority comprising at least one of a scoped non-bearer finality capability and an Execution Handle, wherein the scoped execution authority is not released unless the validation receipt is first generated or committed in protected state; and

a Finality Sink configured to hold a locked execution primitive that is structurally incapable of completing the Candidate Act without completion material derived from the scoped execution authority, verify that the scoped execution authority cryptographically corresponds to the Candidate Act, the Finality Sink identity, the policy epoch, the revocation epoch, and the nonce, and release the completion material only upon successful verification, thereby causing the Candidate Act to become externally effective solely within the permitted consequence envelope.

DEPENDENTS TO METHOD CLAIM 1

Dependent Claim 3 — Cryptographic Action Sharding

The method of claim 1, wherein releasing the scoped execution authority comprises dividing execution material required to complete the Candidate Act into a plurality of cryptographic shares held by respective distinct protected components, wherein each cryptographic share is independently bound to at least the Candidate Act hash, the policy epoch, the revocation epoch, the nonce, and a Finality Sink identity, and wherein the Finality Sink reconstructs or combines the execution material only upon receiving a threshold number of valid, scope-matching, epoch-current, and nonce-fresh shares within a bounded verification time window, such that no single protected component, artificial-intelligence agent, or ordinary software process possesses complete execution authority.

Dependent Claim 4 — Commitment-Release Temporal Escrow

The method of claim 1, further comprising:

generating a Commitment Token that binds the Candidate Act to the Finality Sink identity, the Execution Authorization Scope Object, the policy epoch, the revocation epoch, the nonce, and the validation receipt, wherein the Commitment Token stages the Candidate Act in a non-effective escrow state and is cryptographically incapable of causing external effectuation without a corresponding Release Token;

transmitting a non-final staged-status indicator to an upstream workflow, wherein the staged-status indicator does not constitute effectuation and does not satisfy any downstream execution dependency;

performing, after a defined confirmation window, a current-state re-verification comprising at least policy-epoch verification, revocation-epoch verification, Finality Sink attestation verification, and absence-of-revocation checking; and

generating the Release Token, cryptographically bound to the Commitment Token, only upon successful current-state re-verification, wherein if any required re-verification predicate fails before Release Token generation, the Commitment Token is destroyed and no Release Token is generated.

Dependent Claim 5 — Paired Forward and Rollback Capabilities

The method of claim 4, further comprising:

generating, atomically with the Commitment Token, a paired authority set comprising a forward capability permitting limited effectuation within a permitted consequence scope and a rollback capability permitting bounded reversal of the effectuation, wherein both the forward capability and the rollback capability are cryptographically bound to the same Candidate Act hash, the Execution Authorization Scope Object hash, the validation receipt identifier, the Finality Sink identity, the nonce, the policy epoch, and the revocation epoch;

holding the rollback capability in escrow within a protected enforcement component outside the ordinary agent execution path;

maintaining a post-effectuation monitoring window during which monitored conditions comprising at least one of source revocation, policy update, revocation-epoch advancement, sink-trust degradation, canary failure, and contradiction evidence are evaluated; and

releasing the rollback capability to the Finality Sink upon detection of a monitored failure condition within the monitoring window, thereby causing bounded reversal of the effectuation within the permitted rollback scope, and destroying or archiving the rollback capability upon expiry of the monitoring window without detection of a monitored failure condition.

Dependent Claim 6 — Ephemeral Finality Sink

The method of claim 1, wherein the Finality Sink is an ephemeral Finality Sink provisioned as a temporary single-use execution environment bound to exactly one validated Candidate Act, wherein the ephemeral Finality Sink is configured with only the minimum credentials, permissions, policy state, recipient scope, data scope, tool scope, time window, and execution primitive required for the validated Candidate Act, and wherein after completion, refusal, rollback, timeout, or expiration, the ephemeral Finality Sink destroys itself by deleting temporary credentials, clearing nonce state, invalidating session keys, and erasing transient execution material, such that a copied scoped execution authority cannot be replayed against any subsequent execution environment because the original Finality Sink identity, nonce, and attested execution context no longer exist

Dependent Claim 7 — Proxy-Based Legacy Interface Enforcement

The method of claim 1, further comprising:

positioning a protected Finality Sink proxy immediately before an existing effectuation interface, wherein the protected proxy intercepts a Candidate Act, tool call, transaction request, database mutation, data-export request, or command message directed to the effectuation interface;

performing, by the protected proxy, verification comprising at least Candidate Act hash comparison, scoped execution authority verification, nonce checking, expiration checking, policy-epoch and revocation-epoch checking, Execution Authorization Scope Object scope comparison, Finality Sink identity checking, and validation receipt reference checking;

translating, upon successful verification, the verified Candidate Act into a conventional downstream request compatible with the effectuation interface; and

refusing, upon verification failure, to forward any request to the effectuation interface, whereby the effectuation interface receives executable requests only after proxy-side finality enforcement and need not natively parse Hash-Linked Candidate Act Descriptor, algorithmic logic fingerprint, Execution Authorization Scope Object, or Execution Handle formats.

Dependent Claim 8 — Modular Escalation Enforcement

The method of claim 1, further comprising:

classifying the Candidate Act as escalated-but-allowable upon determining that the Candidate Act presents an elevated-risk condition that does not require denial and can be reduced to a permitted consequence scope by application of additional controls;

routing the Candidate Act to a plurality of protected escalation modules, each module independently generating a machine-verifiable escalation result cryptographically bound to at least the Candidate Act hash, the Hash-Linked Candidate Act Descriptor hash, the policy epoch, the revocation epoch, the nonce, the module identity, and the Finality Sink identity;

specifying, in the Execution Authorization Scope Object, which escalation module results are mandatory for release of an escalated scoped execution authority;

releasing the escalated scoped execution authority only upon receipt of all mandatory escalation module results that are valid, non-stale, non-revoked, epoch-current, and cryptographically bound to the same Candidate Act and current protected state; and

binding the selected escalation controls into the released escalated scoped execution authority such that the Finality Sink rejects the escalated Execution Handle if any mandatory module result is absent, stale, revoked, or epoch-mismatched.

Dependent Claim 9 — Receipt as Release Predicate

The method of claim 1, wherein generating the validation receipt comprises committing receipt state to a protected receipt store comprising at least one of an append-only log, a hash chain, a Merkle-committed structure, a hardware-protected log, a signed receipt object, a hardware security module-backed counter, or a secure enclave state, wherein the scoped execution authority is not generated unless the receipt state is first committed in protected state, and wherein the Finality Sink treats the receipt identifier, receipt hash, or State-Proof as a required verification input, refusing effectuation if the receipt state does not cryptographically correspond to the Candidate Act, the scoped execution authority, the nonce, the policy epoch, the revocation epoch, and the Finality Sink identity, whereby the validation receipt is structurally embedded in the release path rather than appended as a post-effectuation record.

Dependent Claim 10 — Multi-Assurance Sink Profile with Anti-Downgrade Binding

The method of claim 1, further comprising:

selecting, by the Protected Enforcement Domain, a Finality Sink assurance profile from a plurality of profiles comprising at least a low-assurance profile requiring hash matching, nonce checking, scope comparison, and epoch verification; a medium-assurance profile additionally requiring validation receipt reference, protected human approval reference, scoped consequence envelope

hash, and sink attestation; and a high-assurance profile additionally requiring Execution Handle verification, State-Proof verification, hardware identity binding, locked primitive completion, rollback capability escrow, or multi-sink threshold verification;

wherein profile selection is determined by at least one of Candidate Act risk class, consequence type, data classification, jurisdiction, financial value, reversibility, escalation score, and current Finality Sink trust state; and

binding the selected assurance profile into at least one of the scoped consequence envelope, the Execution Authorization Scope Object, the scoped execution authority, the Execution Handle, the validation receipt, or the State-Proof, such that the Finality Sink is technically incapable of silently applying a lower verification level than the profile bound into the released authority.

Dependent Claim 11 — Adversarial Perturbation Gate

The method of claim 8, further comprising:

generating, as a mandatory escalation module result, a perturbation verification result by executing, against the Candidate Act consequence model, a bounded set of parameterized synthetic perturbations comprising minor variations of the Candidate Act within at least one of recipient space, financial amount space, jurisdictional space, and data-class space;

evaluating whether any perturbation crosses from a permitted consequence class into a prohibited consequence class;

if any perturbation within the tested radius reaches a prohibited consequence class, tightening the Execution Authorization Scope Object to exclude the proximity neighborhood of the prohibited boundary, or reclassifying the Candidate Act as requiring denial;

recording in the validation receipt the perturbation set identity, the tested radius, and the nearest prohibited consequence neighbor distance; and

releasing the escalated scoped execution authority only when the perturbation verification result confirms that no perturbation within the tested radius produces a prohibited consequence, such that the released authority is bounded away from prohibited consequence space by a cryptographically attested margin.

Dependent Claim 12 — Consequence-Class Inheritance Ceiling Across Agent Chains

The method of claim 1, further comprising:

binding into the scoped execution authority a Consequence-Class Inheritance Tag specifying the maximum consequence class authorized at the originating finality decision;

when a downstream artificial-intelligence agent generates a subsequent Candidate Act whose originating input comprises an output of the effectuated Candidate Act, providing the Consequence-Class Inheritance Tag to the Protected Enforcement Domain evaluating the downstream Candidate Act;

enforcing, by the Protected Enforcement Domain, that the downstream Candidate Act's permitted consequence class does not exceed the inherited ceiling specified in the Consequence-Class Inheritance Tag, regardless of any broader consequence envelope otherwise applicable to the downstream agent; and

refusing to release a scoped execution authority for any downstream Candidate Act whose predicted consequence class exceeds the inherited ceiling, whereby escalation at one step in an agent chain does not re-authorize higher-consequence acts at subsequent steps and the Consequence-Class Inheritance Tag is narrowed monotonically across agent chain hops.

DEPENDENTS TO SYSTEM CLAIM 2

Dependent Claim 13 — Cryptographic Action Sharding

The system of claim 2, wherein the Protected Enforcement Domain is further configured to divide execution material required to complete the Candidate Act into a plurality of cryptographic shares distributed across a plurality of distinct protected components comprising at least two of an agent-side protected wrapper, an enterprise hardware security module, an external approval device, a biometric-bound approval device, a jurisdictional enforcement module, a payment-security module, and an independent Finality Sink, wherein each cryptographic share is independently bound to at least the Candidate Act hash, the policy epoch, the revocation epoch, the nonce, and the Finality Sink identity, and wherein the Finality Sink is configured to reconstruct or combine the execution material only upon receiving a threshold number of valid, scope-matching, epoch-current, nonce-fresh shares within a bounded verification time window.

Dependent Claim 14 — Commitment-Release Temporal Escrow

The system of claim 2, wherein the Protected Enforcement Domain is further configured to generate a Commitment Token cryptographically incapable of causing external effectuation without a corresponding Release Token, transmit a non-final staged-status indicator to an upstream workflow without constituting effectuation, perform current-state re-verification after a defined confirmation window, and generate the Release Token only upon successful current-state re-verification, and wherein the Finality Sink is configured to stage the Candidate Act in a non-effective escrow state upon receipt of the Commitment Token and to complete effectuation only upon receipt of the Release Token bound to the Commitment Token.

Dependent Claim 15 — Paired Forward and Rollback Capabilities

The system of claim 14, wherein the Protected Enforcement Domain is further configured to generate, atomically with the Commitment Token, a forward capability permitting limited effectuation within a permitted consequence scope and a rollback capability permitting bounded reversal, hold the rollback capability in escrow outside the ordinary agent execution path, evaluate monitored conditions during a post-effectuation monitoring window, and release the rollback capability to the Finality Sink upon detection of a monitored failure condition, and wherein the Finality Sink is configured to execute bounded reversal upon receipt of the rollback capability within the monitoring window and to treat expiry of the monitoring window without rollback as completion of the finality transaction.

Dependent Claim 16 — Ephemeral Finality Sink

The system of claim 2, wherein the Finality Sink comprises an ephemeral Finality Sink instantiated as at least one of a micro-virtual machine, secure enclave instance, serverless function, isolated process, container, short-lived tool executor, temporary database role, or transient payment executor, wherein the ephemeral Finality Sink is configured with only the minimum credentials, permissions, and execution primitive required for the validated Candidate Act, and wherein the ephemeral Finality Sink is further configured to destroy itself after completion, refusal, rollback, timeout, or expiration by deleting temporary credentials, clearing nonce state, invalidating session

keys, and erasing transient execution material such that the Finality Sink identity, nonce, and attested execution context become unavailable to any subsequent execution environment.

Dependent Claim 17 — Protected Hardware Finality Sink

The system of claim 2, wherein the Finality Sink or a sink-adjacent enforcement module comprises at least one of a hardware security module, secure enclave, trusted execution environment, secure element, SmartNIC, data processing unit, firmware-protected controller, database commit controller, storage controller, network processor, payment-security module, or protected microcontroller, wherein the protected hardware is configured to verify compact finality artifacts comprising at least the Candidate Act hash, the scoped execution authority, the Execution Handle, the validation receipt hash, the nonce, the policy epoch, the revocation epoch, the Execution Authorization Scope Object hash, and the Finality Sink identity, and to release completion material comprising at least one of a command signature, database commit token, payment authorization share, data-export key, network-control authorization code, storage-write token, actuator enable value, or model-update unlock value only upon successful verification, wherein ordinary application software is technically incapable of extracting the completion material or causing effectuation without a valid scoped execution authority verified by the protected hardware.

Dependent Claim 18 — Proxy-Based Legacy Interface Enforcement

The system of claim 2, further comprising a protected Finality Sink proxy configured to intercept a Candidate Act, tool call, transaction request, database mutation, data-export request, or command message directed to an existing effectuation interface; perform verification comprising at least Candidate Act hash comparison, scoped execution authority verification, nonce checking, expiration checking, policy-epoch and revocation-epoch checking, Execution Authorization Scope Object scope comparison, Finality Sink identity checking, and validation receipt reference checking; translate the verified Candidate Act into a conventional downstream request compatible with the effectuation interface upon successful verification; and refuse to forward any request to the effectuation interface upon verification failure, wherein the effectuation interface is configured to receive only finality-verified conventional requests and need not natively process Hash-Linked Candidate Act Descriptor, algorithmic logic fingerprint, Execution Authorization Scope Object, or Execution Handle formats

Dependent Claim 19 — Multi-Assurance Sink Profile with Anti-Downgrade Binding

The system of claim 2, wherein the Protected Enforcement Domain is further configured to select a Finality Sink assurance profile from a plurality of profiles based on at least one of Candidate Act risk class, consequence type, data classification, jurisdiction, financial value, reversibility, escalation score, and Finality Sink trust state, and to bind the selected assurance profile into at least one of the scoped consequence envelope, the Execution Authorization Scope Object, the scoped execution authority, the Execution Handle, the validation receipt, or the State-Proof, and wherein the Finality Sink is configured to apply only the bound assurance profile and is technically incapable of silently substituting a lower verification level.

Dependent Claim 20 — Receipt as Release Predicate

The system of claim 2, wherein the Protected Enforcement Domain is further configured to commit receipt state to a protected receipt store comprising at least one of an append-only log, a hash chain, a Merkle-committed structure, a hardware-protected log, a signed receipt object, a hardware security module-backed counter, or a secure enclave state before generating the scoped execution authority, wherein generation of the scoped execution authority is blocked unless the receipt state is

successfully committed in protected state, and wherein the Finality Sink is configured to treat the receipt identifier, receipt hash, or State-Proof as a required verification input and to refuse effectuation if the receipt state does not cryptographically correspond to the Candidate Act, the scoped execution authority, the nonce, the policy epoch, the revocation epoch, and the Finality Sink identity.

Dependent Claim 21 — Modular Escalation Enforcement

The system of claim 2, further comprising a plurality of protected escalation modules each configured to independently generate a machine-verifiable escalation result cryptographically bound to at least the Candidate Act hash, the Hash-Linked Candidate Act Descriptor hash, the policy epoch, the revocation epoch, the nonce, the module identity, and the Finality Sink identity, wherein the Protected Enforcement Domain is further configured to classify a Candidate Act as escalated-but-allowable upon detecting an elevated-risk condition controllable by additional safeguards, specify in the Execution Authorization Scope Object which escalation module results are mandatory, release an escalated scoped execution authority only upon receipt of all mandatory escalation module results that are valid, non-stale, non-revoked, epoch-current, and cryptographically bound to the same Candidate Act and current protected state, and bind the selected escalation controls into the released escalated scoped execution authority, and wherein the Finality Sink is configured to reject the escalated Execution Handle if any mandatory module resul

Dependent Claim 22 — Consequence-Class Inheritance Ceiling Across Agent Chains

The system of claim 2, wherein the Protected Enforcement Domain is further configured to bind into the scoped execution authority a Consequence-Class Inheritance Tag specifying the maximum consequence class authorized at the originating finality decision, receive the Consequence-Class Inheritance Tag when evaluating a downstream Candidate Act generated by a downstream artificial-intelligence agent whose originating input comprises an output of the effectuated Candidate Act, enforce that the downstream Candidate Act's permitted consequence class does not exceed the inherited ceiling regardless of any broader consequence envelope otherwise applicable to the downstream agent, and refuse to release a scoped execution authority for any downstream Candidate Act whose predicted consequence class exceeds the inherited ceiling, whereby the Consequence-Class Inheritance Tag is narrowed monotonically across agent chain hops and escalation at one chain step does not re-authorize higher-consequence acts at subsequent steps.

Dependent Claim 23 — Agent-Side Self-Validation Exclusion

The method of claim 1, further comprising:

maintaining the Protected Enforcement Domain in a cryptographically isolated execution boundary that is structurally separated from the execution process, memory space, container, virtual machine, and software trust boundary of the artificial-intelligence agent whose Candidate Act is being evaluated, wherein the cryptographic isolation is enforced such that the artificial-intelligence agent cannot read, modify, suppress, bypass, forge, or replay the Protected Enforcement Domain's validation logic, receipt commitment state, nonce state, monotonic counter state, policy epoch state, revocation epoch state, capability-generation material, Execution Handle material, or scoped execution authority release mechanism through any instruction, system call, memory access, inter-process communication, shared memory region, or network request available within the agent's ordinary execution environment;

wherein a Protected Enforcement Domain implemented within the same execution process, memory space, container, serverless function invocation, or software trust boundary as the artificial-

intelligence agent does not satisfy the cryptographic isolation requirement, because such an implementation permits the agent to access, modify, or suppress the Protected Enforcement Domain's protected state through instructions available within the agent's ordinary execution environment; and

wherein the cryptographic isolation is verified by at least one of a hardware attestation measurement, secure enclave identity attestation, trusted execution environment measurement, hardware security module identity binding, or protected microcontroller firmware attestation bound into the Execution Handle and verified by the Finality Sink before completion material is assembled, such that the Finality Sink refuses to assemble completion material if the attested Protected Enforcement Domain identity does not correspond to the identity bound into the scoped execution authority at minting time.

Dependent Claim 24 — Software Token Non-Equivalence to Structural Incompleteness

The method of claim 1, wherein the locked execution primitive held by the Finality Sink is distinguished from a software-encrypted payload, application-layer access token, signed API request, encrypted command object, cloud-managed decryption artifact, or bearer credential by the requirement that:

the completion material required to satisfy the locked execution primitive is derived, held, or released exclusively within a protected boundary from which extraction by software instruction, operating system call, hypervisor operation, cloud infrastructure management interface, key-management service API, or privileged process operation is technically impossible regardless of the privilege level of the requesting process;

wherein a locked execution primitive whose completion material is held in a key-management service, application-layer secrets store, cloud-provider-managed encryption key, software-accessible memory region, or any storage location accessible to a sufficiently privileged software process does not satisfy the structural incompleteness requirement, because such completion material can be assembled by a sufficiently privileged process without satisfying the required finality predicates; and

wherein structural incompleteness is verified at the Finality Sink by confirming that the completion material assembly operation succeeds only as a direct result of the scoped execution authority presented by the Protected Enforcement Domain following successful atomic finality validation, and fails under all other conditions including presentation of a copied software token, a replayed encrypted payload, a cloud-managed decryption artifact, or any artifact not produced by a protected finality transaction satisfying the required Candidate Act hash, policy epoch, revocation epoch, nonce, and Finality Sink identity correspondence.

Dependent Claim 25 — Receipt-Bound Release Non-Equivalence to Post-Effectuation Logging

The method of claim 9, further comprising:

enforcing that the validation receipt, Ledger-Anchored Validation Receipt, Residual Risk Receipt, State-Proof, escalation record, rollback record, sink-verification record, or effectuation record required as a release predicate is generated, committed, sealed, or hash-linked in protected state before or within the same hardware-enforced atomic operation as release of the scoped execution authority, wherein the atomicity of receipt commitment and authority release is enforced by a

hardware boundary such that neither operation can be completed, reversed, suppressed, or independently extracted from the other by any software process regardless of privilege level;

wherein a receipt committed within an ordinary database transaction, message queue, append-only software log, cloud audit service, observability pipeline, monitoring system, compliance record, or post-effectuation audit trail does not satisfy the receipt-as-release-predicate requirement, because such systems permit the receipt commitment to be aborted, rolled back, suppressed, modified, or written after the scoped execution authority has already been released through a pathway that does not enforce atomicity between receipt commitment and authority release at a hardware boundary;

wherein a system that releases a scoped execution authority and subsequently writes a log entry, audit record, or monitoring event recording that the authority was released does not satisfy the receipt-bound release requirement regardless of the tamper-evidence properties of the logging system, because the log entry in such a system is an audit artifact appended after effectuation rather than a release predicate enforced before or atomically with authority release; and

wherein the receipt-bound release requirement is satisfied only when the Finality Sink can verify, at the time the scoped execution authority is presented, that a receipt with the bound identifier exists in protected receipt state and that the receipt's Candidate Act hash, Execution Authorization Scope Object hash, Result-Consequence Acceptance Envelope hash, nonce, policy epoch, revocation epoch, and Finality Sink identity cryptographically correspond to the presented scoped execution authority — such that a scoped execution authority not backed by a committed receipt, and a receipt whose parameters have diverged from the authority's bound parameters through any post-minting mutation, each independently fail verification and cause the Finality Sink to refuse completion material assembly.

Dependent Claim 26 — Ordinary Permission Check Non-Equivalence to Candidate-Act Finality

The method of claim 1, further comprising:

distinguishing the scoped execution authority released by the Protected Enforcement Domain from an ordinary permission check, access-control decision, bearer-token authorization, OAuth-style authorization grant, API-key validation result, role-based access control decision, attribute-based access control decision, policy-engine approval, model guardrail result, output moderation decision, content filter result, trust score, risk score, safety classification, or workflow approval by the requirement that the scoped execution authority is:

bound to the specific Candidate Act through the Candidate Act hash carried in the Hash-Linked Candidate Act Descriptor, such that the authority is valid only for the specific artificial-intelligence-generated output that was evaluated and is invalid for any other output, instruction, request, or act-equivalent representation regardless of whether that other output was generated by the same model, workflow, prompt policy, or authorized agent;

bound to the specific Finality Sink identity, attested sink hardware identity, policy epoch, revocation epoch, and nonce current at the time of the atomic finality transaction, such that the authority is invalid at any other sink, under any prior policy epoch, under any prior revocation epoch, and after nonce expiry, regardless of whether the requesting entity is otherwise authorized to access the sink or invoke the downstream interface;

generated only after the Candidate Act has satisfied machine-verifiable finality predicates comprising at least algorithmic logic fingerprint validation establishing approved computational logic state, consequence simulation confirming the predicted external effect remains within the

permitted consequence envelope, and validation receipt commitment in protected state establishing tamper-evident evidence of predicate satisfaction bound to the specific Candidate Act and current protected state; and

verified by the Finality Sink as a condition of completion material assembly, such that the downstream effectuation interface receives an executable request only after the Finality Sink confirms that the scoped execution authority cryptographically corresponds to the specific Candidate Act, the current Finality Sink identity, the current policy epoch, the current revocation epoch, and the nonce, and not merely because an authorized identity, approved model, permitted workflow, valid API key, passing safety score, or ordinary workflow approval is present;

wherein a system satisfies an ordinary permission check but not the scoped execution authority requirement when it verifies that a requesting entity is authorized to invoke a downstream interface without additionally verifying that the specific artificial-intelligence-generated Candidate Act has satisfied the required output-level, consequence-level, provenance-level, epoch-level, and sink-level finality predicates and that the released authority is cryptographically bound to that specific act, that specific sink, that specific epoch state, and that specific nonce — because such a system answers only the question of whether the requesting entity is permitted to act, not the question of whether this specific artificial-intelligence-generated Candidate Act is permitted to become this specific external consequence at this specific Finality Sink under current protected state conditions.

Dependent Claim 27 — First Usable Release Boundary Enforcement Against Proxy and Gateway Substitution

The method of claim 1, further comprising:

determining the first usable release boundary as the earliest technical boundary in the execution path at which the Candidate Act or any act-equivalent representation becomes capable of producing an external consequence, wherein the first usable release boundary is determined by the functional capability of the component to convert the Candidate Act into external consequence and not by the architectural label, component name, deployment description, vendor classification, or implementation technology assigned to that component;

enforcing the finality requirements of claim 1 at the first usable release boundary regardless of whether that boundary is implemented as a software proxy, API gateway, service-mesh sidecar, message-broker interceptor, database proxy, workflow controller, tool-execution dispatcher, payment adapter, storage gateway, network-control proxy, orchestration layer, plugin manager, agent runtime, serverless function, container sidecar, or any other software, firmware, hardware, or hybrid component positioned before the downstream effectuation interface;

wherein a component does not avoid the first usable release boundary requirement by characterizing itself as a pass-through proxy, transparent gateway, routing layer, load balancer, protocol translator, format converter, message transformer, workflow orchestrator, or infrastructure middleware if that component is technically capable of converting the Candidate Act or any act-equivalent representation into an externally effective consequence through any execution path available to it, because the finality obligation follows the consequence-producing capability of the component and not the label assigned to it;

wherein a component positioned upstream of the first usable release boundary that performs partial validation, scoring, classification, moderation, logging, monitoring, or routing of the Candidate Act does not satisfy the finality requirement of claim 1 unless it additionally enforces the non-effective hold, releases a scoped execution authority bound to the specific Candidate Act, and verifies that

authority at the first usable release boundary before any act-equivalent representation reaches a downstream component capable of producing external consequence; and

wherein a system that routes the Candidate Act through multiple upstream components — each performing partial checks — before reaching the first usable release boundary does not satisfy the finality requirement unless the scoped execution authority is verified at the first usable release boundary itself, because partial upstream checks do not constitute Finality Sink verification and do not prevent an act-equivalent representation from reaching the downstream effectuation interface through an alternative path that bypasses one or more upstream components.

Dependent Claim 28 — Fragmented and Delegated Act Equivalence

The method of claim 1, further comprising:

treating as a Candidate Act subject to the finality requirements of claim 1 any fragmented representation, delegated representation, serialized representation, batched representation, scheduled representation, compressed representation, encrypted representation, translated representation, summarized representation, paraphrased representation, split representation, or otherwise transformed representation of an artificial-intelligence-generated output that is capable of producing the same or substantially similar external consequence as the original Candidate Act, regardless of whether the transformation occurs before or after the artificial-intelligence output is generated, within or outside the agent runtime, through direct execution or through delegation to a downstream agent, tool, plugin, workflow, service, or automated process;

wherein a Candidate Act is not converted into a non-Candidate-Act by fragmenting it into multiple smaller acts that individually fall below a consequence threshold but collectively produce the same or substantially similar external consequence as the original Candidate Act, and wherein the Protected Enforcement Domain evaluates the cumulative consequence of related fragmented acts sharing common Hash-Linked Candidate Act Descriptor references, recipient identities, purpose identifiers, Finality Sink identities, policy contexts, or temporal proximity when determining whether the fragmented acts collectively exceed a permitted consequence boundary;

wherein a Candidate Act is not converted into a non-Candidate-Act by delegating its execution to a downstream artificial-intelligence agent, tool executor, plugin, workflow engine, scheduled job, external service, or automated process, because the delegated operation is treated as an act-equivalent representation subject to the same finality requirements as the original Candidate Act when it is capable of producing the same or substantially similar external consequence, and wherein the Consequence-Class Inheritance Tag bound into the scoped execution authority of the originating Candidate Act propagates to the Protected Enforcement Domain evaluating the delegated act and enforces that the delegated act's permitted consequence class does not exceed the inherited ceiling; and

wherein a system does not avoid the finality requirements of claim 1 by changing the format, encoding, protocol, transport mechanism, data structure, or representation layer of the Candidate Act between the Protected Enforcement Domain and the Finality Sink, because the finality requirement is determined by the consequence-producing capability of the act-equivalent representation at the Finality Sink and not by the specific format, encoding, or transport mechanism through which it is delivered.

Dependent Claim 29 — Emergency, Maintenance, and Fallback Path Closure

The method of claim 1, further comprising:

enforcing the finality requirements of claim 1 across all execution paths through which the Candidate Act or any act-equivalent representation can reach a Finality Sink or produce an external consequence, including emergency paths, maintenance paths, break-glass paths, fallback paths, disaster-recovery paths, degraded-mode paths, retry paths, debugging paths, management-plane paths, administrative override paths, privileged-access paths, and any other path that is capable of converting the Candidate Act into an externally effective consequence regardless of the operational condition, system state, or administrative authorization under which that path is activated;

wherein a system does not satisfy the finality requirements of claim 1 by enforcing Candidate-Act finality on ordinary execution paths while maintaining an emergency, maintenance, break-glass, fallback, disaster-recovery, degraded-mode, retry, debugging, management-plane, administrative override, or privileged-access path through which a Candidate Act or act-equivalent representation can reach a Finality Sink or produce an external consequence without satisfying the required finality predicates and without presenting a scoped execution authority verified by the Finality Sink;

wherein a system operating in a degraded, emergency, or maintenance mode that permits external consequence without Candidate-Act finality verification does not satisfy the finality requirements of claim 1 unless the degraded, emergency, or maintenance mode is itself governed by a separate protected finality workflow producing a mode-specific scoped execution authority bound to the specific Candidate Act, the specific operational mode, the specific Finality Sink identity, the current policy epoch, the current revocation epoch, and a fresh nonce, and verified by the Finality Sink before any consequence is permitted under the degraded, emergency, or maintenance mode; and

wherein a fail-open behavior — in which a Candidate Act is permitted to become externally effective because required validation evidence, scoped execution authority, Execution Handle, validation receipt, policy epoch currency, revocation epoch currency, nonce freshness, sink attestation, or completion material is absent, unavailable, stale, invalid, or mismatched — does not satisfy the finality requirements of claim 1, because the finality invariant requires that the Candidate Act remain non-effective under all conditions in which the required scoped execution authority cannot be verified at the Finality Sink, and the appropriate system response to unavailable or invalid finality evidence is non-effectuation, reduced-scope effectuation under a fail-limited mode, or routing to protected review — not unrestricted effectuation as a consequence of missing proof.

ABSTRACT

Disclosed are systems and methods for Agentic AI Security in Autonomous AI Agents, Agentic Workflows, and LLM Copilots. An AI-generated tool call, payment instruction, database update, data export, network command, customer communication, software deployment, or actuator command is treated as a non-effective Candidate Act, not authority to act. A Protected Enforcement Domain enforces Zero-Trust Execution Boundaries by validating approved model state, runtime behavior, Execution Provenance, factual support, Purpose-Bound Access, policy freshness, revocation status, Consequence Simulation, and Finality Sink compatibility. Upon satisfaction, a Non-Bearer Finality Capability is released only for the validated act, scope, and Finality Sink, preventing Tool Misuse and supporting Agent Containment. Elevated-risk acts proceed through Escalated Conditional Finality using reduced scope, protected approval, canary execution, Reversible Rollback Capabilities, residual-risk receipts, or monitoring. Advanced embodiments use Cryptographic Execution-Dependency, Structural Non-Completability, locked execution primitives, and Execution Handles before external consequences occur, beyond ordinary Alignment Guardrails and access-control checks alone safely.